Decision Making - Temperature Check for Outdoor Activities

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

You are organizing an outdoor event and need to decide whether it's safe for participants to go outside based on the current temperature. Temperatures below zero degrees Celsius are considered too cold and unsafe, while temperatures above zero are suitable for outdoor activities. Write a program to check if the given temperature is positive or negative.

Input Format

A single integer input representing the current temperature in degrees Celsius.

Constraints

The temperature can be any integer (positive, negative, or zero).

Output Format

- If the given temperature is positive, print "Safe for outdoor activities."
- If the temperature is negative or zero, print "Too cold for outdoor activities."

Sample Input 0

```
15
```

Sample Output 0

```
Safe for outdoor activities
```

Explanation 0

The given temperature is 15 degrees Celsius, which is above zero. Hence, the output is "Safe for outdoor activities."

Sample Input 1

```
-5
```

Sample Output 1

```
Too cold for outdoor activities
```

Explanation 1

The given temperature is -5 degrees Celsius, which is below zero. Hence, the output is "Too cold for outdoor activities."

Change Theme Language Java 7

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner ss=new Scanner(System.in);
        int a=ss.nextInt();
        if(a<=0){
            System.out.print("Too cold for outdoor activities");
        }
        else{
            System.out.print("Safe for outdoor activities");
        }
        }
        /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class sh
        be named Solution. */
    }

```

Line: 21 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Decision Making - Event Scheduling for Leap Year Celebrations

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a city, special events are scheduled every leap year to celebrate community milestones. Event organizers need a program that can help them identify whether a given year is a leap year, so they can schedule their events accordingly. If the year is a leap year, the program should notify the organizers to "Schedule Event"; otherwise, it should inform them "No Event This Year."

Write a program to determine if a given year is a leap year. If it is, print "Schedule Event"; otherwise, print "No Event This Year."

A year is considered a leap year if:

- It is divisible by 4,
- If it is a century year (i.e., divisible by 100), it must also be divisible by 400.

Input Format

A single integer representing the year.

Constraints

NA

Output Format

- Output "Schedule Event" if the year is a leap year.
- Output "No Event This Year" if it is not a leap year.

Sample Input 0

```
2024
```

Sample Output 0

```
Schedule Event
```

Explanation 0

The year 2024 is divisible by 4 and not a century year, so it is a leap year.

Sample Input 1

```
2023
```

Sample Output 1

No Event This Year

Explanation 1

The year 2023 is not divisible by 4, so it is not a leap year.

Change Theme Language Java 7

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner ss=new Scanner(System.in);
        int a=ss.nextInt();
        if ((a%4==0&& a%100!=0)  ||a%400==0){
            System.out.print("Schedule Event");
        }
        else{
            System.out.print("No Event This Year");
        }
        /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class sh
        be named Solution. */
    }
}

```

Line: 21 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Decision Making - Narnian Alphabet Helper

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In Narnia, Jeni is trying to help the young children learn the English alphabet while she is away. Since many of the Narnian children are new to English, Jeni wants a program that will help them identify whether a given character is a vowel, consonant, or not an English alphabet letter at all. This program will help children practice on their own and gain confidence in recognizing vowels and consonants.

Write a program that takes a single character as input and determines if it is a:

- Vowel (letters 'a', 'e', 'i', 'o', 'u' in both uppercase and lowercase),
- Consonant (any other English alphabet letter), or
- Not an alphabet (if the character is not an English letter).

Input Format

A single character (either lowercase or uppercase, or any other symbol).

Constraints

NA

Output Format

Print one of the following outputs based on the character input:

- "Vowel" if the character is a vowel.
- "Consonant" if the character is a consonant.
- "Not an alphabet" if the character is not a valid English alphabet letter.

Sample Input 0

```
e
```

Sample Output 0

```
Vowel
```

Explanation 0

The input character e is a vowel, so the output is "Vowel".

Sample Input 1

```
Z
```

Sample Output 1

Consonant

Explanation 1

The input character Z is a consonant, so the output is "Consonant".

Change Theme Language Java 7

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner ss=new Scanner(System.in);
        char ch=Character.toLowerCase(ss.next().charAt(0));
        if(ch=='a'||ch=='e'||ch=='i'||ch=='o'||ch=='u')
        {
            System.out.print("Vowel");
        }

        else if(ch>='a'&& ch<='z')
        {
            System.out.print("Consonant");

        }
        else{
            System.out.print("Not an alphabet");
        }

        /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class sh
        be named Solution. */
    }
}

```

Line: 29 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Decision Making - Rock, Paper, Scissors

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Two players A and B, are playing the game of Rock, Paper, Scissors. Player A chooses a move represented by a string value M: and the move can be one of the following: ‘rock’, ‘paper’, or 'scissors' where,

- rock beats scissors
- scissors beats paper
- paper beats rock

Your task is to find and return a string value representing the winning move for Player B.

Note The output is case sensitive

Input Format

A string value M representing the move chosen by Player A

Constraints

NA

Output Format

Return a string representing the winning move for Player B.

Sample Input 0

```
rock
```

Sample Output 0

```
Paper
```

Explanation 0

Here, Player A chooses rock. The winning move for Player B is 'paper, as 'paper' beats 'rock' in the game of Rock, Paper, Scissors Hence, paper is returned as the output.

Change Theme Language Java 7

```
1  import java.io.*;
2  import java.util.*;
3  import java.text.*;
```

```
4  import java.math.*;
5  import java.util.regex.*;
6
7  public class Solution {
8
9      public static void main(String[] args) {
10         Scanner ss=new Scanner(System.in);
11         String a=ss.nextLine();
12         if (a.equals("rock")){
13             System.out.print("Paper");
14         }
15         else if (a.equals("scissors")){
16             System.out.print("Rock");
17
18
19         }
20         else{
21             System.out.print("Scissors");
22         }
23
24         /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class sh
           be named Solution. */
25     }
26 }
27
```

Line: 27 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Decision Making - Joey's Quest for the Trendy Meatball Sandwich

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Joey Tribbiani, a huge fan of sandwiches, spots an irresistible offer at his favorite sandwich shop. The shop promises free sandwiches for life to anyone who can identify whether a given number is a "Trendy Number." Joey, excited at the prospect, wants to check various numbers to see if they qualify. A "Trendy Number" is defined as a 3-digit number where the middle digit is divisible by 3. Can you help Joey write a program that determines whether a given number is a "Trendy Number"?

Write a program that takes an integer n as input and checks the following:

- If the number is not a 3-digit number, it should print "Invalid Number".
- If the number is a 3-digit number and its middle digit is divisible by 3, it should print "Trendy Number".
- If the number is a 3-digit number but its middle digit is not divisible by 3, it should print "Not a Trendy Number".

Input Format

A single integer, n, representing the given number.

Constraints

NA

Output Format

- Print "Trendy Number" if the input satisfies the trendy number conditions.
- Print "Not a Trendy Number" if it's a 3-digit number but doesn't satisfy the middle-digit divisibility rule.
- Print "Invalid Number" if the input number is not a 3-digit number.

Sample Input 0

```
791
```

Sample Output 0

```
Trendy Number
```

Explanation 0

The input number 791 is a 3-digit number, and its middle digit 9 is divisible by 3, making it a "Trendy Number."

Sample Input 1

```
275
```

Sample Output 1

Not a Trendy Number

Explanation 1

The input number 275 is a 3-digit number, but its middle digit 7 is not divisible by 3, so it's "Not a Trendy Number".

Change Theme Language Java 7

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner ss=new Scanner(System.in);
        int a=ss.nextInt();
        int c=String.valueOf(a).length();
        if (c!=3){
            System.out.print("Invalid Number");
        }
        else{
        int d=a/10;
        int x=d%10;
        if(x%3==0){
            System.out.print("Trendy Number");
        }
        else{
            System.out.print("Not a Trendy Number");
        }
        }

        /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class sh
        be named Solution. */
    }
}
```

Line: 30 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Decision Making - Car Rental Charges for a Travel Agency

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine you're organizing a trip for a group of tourists, and you've partnered with a local travel agency to provide car rentals for the journey. The travel agency has a pricing policy where they charge differently for the first few hours of travel and then increase the hourly rate after a specified number of hours.

Your task is to calculate the total cost for a given travel time in minutes. The travel agency charges:

1. R1 rupees per hour for the first N hours of travel.
2. R2 rupees per hour for any additional time after the first N hours.

You need to compute the total cost based on the total travel time (in minutes) provided as input.

Input Format

- The first line contains an integer R1, representing the charge (in rupees) per hour for the first N hours.
- The second line contains an integer N, representing the number of hours for which the first rate R1 applies.
- The third line contains an integer R2, representing the charge (in rupees) per hour for additional hours after the first N hours.
- The fourth line contains an integer X, representing the total travel time in minutes.

Constraints

NA

Output Format

Output the total traveling cost in rupees.

Sample Input 0

```
20
4
40
300
```

Sample Output 0

```
120
```

Explanation 0

- Total travelling hours = 300÷60=5 hours (since we convert 300 minutes to hours).
- Rupees 20/hour for the first 4 hours: 20×4=80 rupees.
- Rupees 40/hour for the remaining 1 hour: 40×1=40 rupees.
- Total cost = 80+40=120 rupees.

Sample Input 1

```
30
5
35
500
```

Sample Output 1

```
290
```

Explanation 1

- Total travelling hours = 500÷60=8.33hours. The ceiling of 8.33 is 9 hours.
- Rupees 30/hour for the first 5 hours: 30×5=150 rupees.
- Rupees 35/hour for the remaining 4 hours: 35×4=140 rupees.
- Total cost = 150+140=290 rupees.

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner ss=new Scanner(System.in);
        int a=ss.nextInt();
        int b=ss.nextInt();
        int c=ss.nextInt();
        int d=ss.nextInt();
        int f=(int)Math.ceil(d/60.0);
        int cost;
         int e=a*b;
            int g=f-b;
        if (f<=b){
             cost=f*a;
        }
        else{
            cost=e+(g*c);
        }

        System.out.print(cost);

        /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class sh
        be named Solution. */
    }
```

Line: 31 Col: 1

Upload Code as File
Test against custom input
Run Code
Submit Code

Blog | Scoring | Environment | FAQ | About Us | Helpdesk | Careers | Terms Of Service | Privacy Policy

Decision Making - Choosing the Tallest Skyscraper

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

A city planning committee is considering three skyscraper designs for a new urban development project. To make the most impactful statement in the city's skyline, they need to select the tallest skyscraper from the proposed designs. Each skyscraper's height in meters is provided as input. Your task is to write a program that helps the committee determine which skyscraper is the tallest.

Input Format

Three integers, height1, height2, and height3, representing the heights of the three skyscrapers.

Constraints

NA

Output Format

Print the height of the tallest skyscraper as an integer.

Sample Input 0

```
250 300 275
```

Sample Output 0

```
300
```

Explanation 0

The heights of the skyscrapers are 250m, 300m, and 275m. The tallest one is 300m.

Sample Input 1

```
450 450 400
```

Sample Output 1

```
450
```

Explanation 1

The heights of the skyscrapers are 450m, 450m, and 400m. The tallest one is 450m.

Change Theme Language Java 7

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner ss=new Scanner(System.in);
        int a=ss.nextInt();
        int b=ss.nextInt();
        int c=ss.nextInt();
        if(a>b && a>c){
            System.out.print(a);
        }
        else if(b>=a && b>=c){
            System.out.print(b);
        }
        else if(c>=a && c>=b) {
            System.out.print(c);
        }
        /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class sh
        be named Solution. */
    }
}

```

Line: 26 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Decision Making - Selecting the Second Best Candidate

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

A university is organizing an annual awards ceremony to honor the top students in a prestigious academic competition. After evaluating the scores of the top three finalists, the committee wants to determine the student who secured the second-highest score. Your task is to write a program that, given the scores of the three students, identifies the second-highest score to ensure fair recognition of the top achievers.

Input Format

Three integers, score1, score2, and score3, representing the scores of the three students

Constraints

NA

Output Format

Print the second-highest score as an integer.

Sample Input 0

```
85 92 78
```

Sample Output 0

```
85
```

Explanation 0

The scores are 85, 92, and 78. The second-highest score is 85.

Sample Input 1

```
67 67 89
```

Sample Output 1

```
67
```

Explanation 1

The scores are 67, 67, and 89. The second-highest score is 67.

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner ss=new Scanner(System.in);
        int a=ss.nextInt();
        int b=ss.nextInt();
        int c=ss.nextInt();
        if(a>=b && a>=c){
            System.out.print(Math.max(b,c));
        }
        else if(b>=a && b>=c){
            System.out.print(Math.max(a,c));
        }
        else if(c>=a && c>=b){
            System.out.print(Math.max(a,b));
        }
        /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class sh
        be named Solution. */
    }
}

```

Line: 26 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Decision Making - Calculating Daily Profit or Loss Percentage

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Jammy and Lily run a small shop together. Each day, Jammy invests a certain amount of money, Rs. X, to set up and run the shop. At the end of the day, they calculate the total earnings, Rs. Y, from sales and other sources. Jammy asks Lily to calculate the profit or loss percentage compared to the amount invested. Your task is to write a program that takes the invested amount and the earned amount as inputs and outputs whether they made a profit or a loss, including the percentage value. However, if any input amount is negative, it should output "Invalid Input."

Input Format

1. The first input corresponds to the invested amount (Integer).
2. The second input corresponds to the earned amount (Integer).

Constraints

Input values must be non-negative integers; otherwise, output "Invalid Input."

Output Format

- If there is a profit, print: Profit - X%
- If there is a loss, print: Loss - X%
- If the investment and earnings are equal, print: No Profit, No Loss
- In case of negative inputs, print: Invalid Input

Sample Input 0

```
1200
1500
```

Sample Output 0

```
Profit - 25%
```

Explanation 0

The investment was Rs. 1200, and the earnings were Rs. 1500. The profit is Rs. 300, and the profit percentage is (300/1200) * 100 = 25%.

Sample Input 1

```
1500
1200
```

Sample Output 1

Loss - 20%

Explanation 1

The investment was Rs. 1500, and the earnings were Rs. 1200. The loss is Rs. 300, and the loss percentage is (300/1500) * 100 = 20%.

Sample Input 2

-1500
-1200

Sample Output 2

Invalid Input

Sample Input 3

1000
1000

Sample Output 3

No Profit, No Loss

Change Theme Language Java 7

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner ss=new Scanner(System.in);
        int a=ss.nextInt();
        int b=ss.nextInt();
        int c;
        c=b-a;
        int d=(Math.abs(c)*100/a);

        if (a<0 || b<0)
        {
            System.out.print("Invalid Input");
        }
        else{
            if(a==b)
            {
               System.out.print("No Profit, No Loss");
```

```
25              }
26              else if(c<0){
27                  System.out.printf("Loss - %d%% ",d);
28              }
29              else if(c>0){
30                  System.out.printf("Profit - %d%%" ,d);
```

Line: 38 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Decision Making - Movie Ticket Price Calculator

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

You work at a movie theater that offers different ticket prices for adults and children. Adults are those over 13 years of age, and the regular ticket price for an adult is $5.00, while the regular ticket price for a child is 2.00. The theater runs four daily shows at the following times:

- Morning Show: 10.15
- Matinee Show: 13.30
- Evening Show: 18.00
- Night Show: 22.00

All matinee shows have a special discounted rate, where both adults and children can watch the movie for $2.00. Write a program that calculates the movie ticket price based on the customer's age and the chosen show timing.

Input Format

1. An integer input representing the customer's age.
2. A floating-point number representing the show timing on a 24-hour clock.

Constraints

- Age will be a positive integer.
- Show timing will be given as a floating-point number in a 24-hour format.

Output Format

Print the price of the movie ticket in the format: $

Sample Input 0

```
10
13.30
```

Sample Output 0

```
$2.00
```

Explanation 0

The customer is 10 years old, which classifies them as a child. The chosen show timing is 13.30, which is a matinee show. Since matinee shows have a special price of $2.00 for everyone, the output is 2.00.

Sample Input 1

```
19
13.30
```

Sample Output 1

$2.00

Explanation 1

The customer is 19 years old, classifying them as an adult. The chosen show timing is 13.30, which is a matinee show. Even though the regular adult ticket price is $5.00, the special matinee price is 2.00 for everyone, so the output is $2.00.

Change Theme Language Java 7

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner ss=new Scanner(System.in);
        int a=ss.nextInt();
        String b=ss.next();
        if (b.equals("13.30")){
            System.out.print("$2.00");
        }
        else{
            if(a>=13)
            {
                System.out.print("$5.00");
            }
            else if (a<=13){
                System.out.print("$2.00");
            }
        }

        /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class sh
        be named Solution. */
    }
}

```

Line: 29 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Looping - Adam's Charity

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Adam decides to be generous and do some charity. Starting today, from day 1 until day n, he gives i2 coins to charity on day 'i' (1 <= i <= n).

Return the total coins he would give to charity.

Input Format

Input consists of one integer

Constraints

NA

Output Format

Output consists of one integer

Sample Input 0

```
2
```

Sample Output 0

```
5
```

Explanation 0

- There are 2 days.
- On the first day, he will give 1^2 = 1 coin.
- On the second day, he will give 2^2 = 4.
- Total no.of coins = 1 + 4 = 5

Sample Input 1

```
5
```

Sample Output 1

```
55
```

Explanation 1

- There are 5 days.
- On the first day, he will give 1^1 = 1 coin.
- On the second day, he will give 2^2 = 4 coins and therefore 9, 16 and 25 respectively on 3rd, 4th, 5th day.
- Total no of coins = 1 + 4 + 9 + 16 + 25 = 55

Change Theme Language Java 7

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int sum = 0;

        for (int i = 1; i <= n; i++)
            sum += i * i;

        System.out.println(sum);
    }
}

```

Line: 16 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Looping - Car Speed

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

You have an electric car whose speed is derived from the battery percentage

The battery percentage is denoted by an integer value N and the speed is the sum of all the factors of the battery percentage including 1. Your task is to find and return an integer value representing the speed of the car.

Input Format

An integer value N representing the battery percentage of the car.

Constraints

NA

Output Format

Return an integer value representing the speed of the car.

Sample Input 0

```
12
```

Sample Output 0

```
28
```

Explanation 0

Here, the given battery percent is 12 and the factors of 12 are 1, 2, 3, 4, 6, and 12. The sum of all the factors will be 1+2+3+4+6+12=28. Therefore 28 is returned as the output

Sample Input 1

```
6
```

Sample Output 1

```
12
```

Change Theme Language Java 7

```
1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
```

```
6
7           int n = sc.nextInt();
8           int sum = 0;
9
10          for (int i = 1; i <= n; i++) {
11              if (n % i == 0)
12                  sum += i;
13          }
14
15          System.out.println(sum);
16      }
17  }
18
```

Line: 18 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Looping - Next Leap Year

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

You are an app developer, and your company wants you to develop a new feature to engage more users by showing the next nearest leap year for a given year, represented by an integer value N. Your task is to find and return an integer value representing the next nearest leap year for the given year. If the given year itself is a leap year, then return that.

Input Format

An integer value N representing the given year.

Constraints

NA

Output Format

Return an integer value representing the next nearest leap year for a given year.

Sample Input 0

```
2021
```

Sample Output 0

```
2024
```

Explanation 0

Here, the given year is 2021 and the next upcoming leap year would be 2024. Therefore, 2024 is returned as the output.

Sample Input 1

```
2008
```

Sample Output 1

```
2008
```

Explanation 1

Here, the given year is 2008, which is itself a leap year. Therefore 2008 is returned as the output.

Change Theme Language Java 7

```
1  import java.util.Scanner;
2
```

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int year = sc.nextInt();

        while (true) {
            if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0)) {
                System.out.println(year);
                break;
            }
            year++;
        }
    }
}

```

Line: 17 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Looping - Quality Check for Premium Products

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a retail store, some products are considered "premium" if their product ID is a Prime Number. A Prime Number is a number greater than 1 that has no divisors other than 1 and itself. Your task is to create a program that checks whether the given product ID qualifies as "Premium" or not.

Imagine a store manager who needs to filter out premium products for special promotions. If a product ID is a Prime Number, it should be labeled as "Premium Product," otherwise, it should be labeled as "Regular Product."If the given product ID is 1 consider it as "Regular Product".

Input Format

The first line of input contains an integer N, which represents the product ID.

Constraints

NA

Output Format

- Print "Premium Product" if the product ID is a Prime Number.
- Print "Regular Product" if the product ID is not a Prime Number.

Sample Input 0

```
17
```

Sample Output 0

```
Premium Product
```

Explanation 0

- The number 17 is greater than 1 and has no divisors other than 1 and itself.
- Hence, the product ID 17 is a Prime Number, so the output is "Premium Product."

Sample Input 1

```
20
```

Sample Output 1

```
Regular Product
```

Explanation 1

- The number 20 is greater than 1 but has divisors other than 1 and itself (e.g., 2, 4, 5, 10).
- Hence, the product ID 20 is not a Prime Number, so the output is "Regular Product."

Change Theme Language Java 7

```java
import java.util.Scanner;

public class Solution {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if(n<=1){
        System.out.println("Regular Product");
        return;
        }

        boolean isPrime = true;

        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                isPrime = false;
                break;
            }
        }
```

Line: 29 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Looping - Coding Marathon

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

N number of people participated in a coding marathon where they were asked to solve some problems. Each problem carried 1 mark and at the end of the marathon, the total marks that each person achieved was calculated.

As an organizer, you have the list of the total marks that each person achieved. You have to calculate the sum of the marks of top K scorers from the list.

Input Format

1. An integer N representing the number of participants.
2. An integer K representing the number of top scorers to consider.
3. A space-separated list of N integers representing the marks of each participant.

Constraints

NA

Output Format

A single integer representing the sum of the marks of the top K scorers.

Sample Input 0

```
4
2
4 1 2 5
```

Sample Output 0

```
9
```

Explanation 0

Top 2 scores are 5 and 4. Sum = 5+4 = 9.

Sample Input 1

```
4
3
4 3 6 1
```

Sample Output 1

```
13
```

Explanation 1

Top 3 scores are 6, 4 and 3. Sum = 6+4+3 = 13.

Change Theme Language Java 7

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int k = sc.nextInt();

        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        Arrays.sort(arr);

        int sum = 0;
        for (int i = n - 1; i >= n - k; i--) {
            sum += arr[i];
```

Line: 9 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Looping - Optimizing Energy Division for Maximum Efficiency

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

A renewable energy company wants to distribute a given amount of energy, represented by an integer N, into smaller segments to maximize the total output. Each segment represents a unit that, when combined with others, maximizes the efficiency and overall power output.

The company's research indicates that splitting the energy into certain parts will yield a higher product (efficiency). For instance, if N is split into 2 and 3, or multiple 3's, it yields the highest efficiency. Your task is to help the company determine the best way to split the energy N into smaller segments to maximize the product.

Write a program that accepts a non-negative integer N and outputs the maximum possible product when N is split optimally.

Input Format

A single non-negative integer N.

Constraints

$1 \leq N \leq 100$

Output Format

A single positive integer representing the maximum product of the energy segments.

Sample Input 0

```
4
```

Sample Output 0

```
4
```

Explanation 0

When N = 4, the best split is 4 = 2 + 2, resulting in a product of $2 \times 2 = 4$

Sample Input 1

```
6
```

Sample Output 1

```
9
```

Explanation 1

When N = 6, the best split is 6 = 3 + 3, resulting in a product of $3 \times 3 = 9$

Change Theme Language Java 7

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        if (n == 2) {
            System.out.println(1);
        } else if (n == 3) {
            System.out.println(2);
        } else {
            int product = 1;

            while (n > 4) {
                product *= 3;
                n -= 3;
            }

            product *= n;
```

Line: 25 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Looping - Teaching Digit Positions

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a mathematics class, a number system is being taught to students. Before teaching them 10's and 100's place, they will be taught the number positions. The positions will be starting from sequence number 1, and the direction will be from left to right.

So if I want to find the second position of a digit in the number 90876, it will be 0. If the Kth digit exceeds the number position return -1.

Write a program to find the Kth digit in a given number.

Input Format

1. The first line of input contains an integer N
2. The second line of input contains an integer K

Constraints

NA

Output Format

The output prints an integer denoting the Kth digit in a given number

Sample Input 0

```
956781
3
```

Sample Output 0

```
6
```

Explanation 0

The input by the user is 956781, where 9 is the first digit, 5 is the second digit, 6 is the third digit and so on. The user is asking for the third (3rd) digit in the given number, which is 6.

Sample Input 1

```
12345
6
```

Sample Output 1

```
-1
```

Explanation 1

The given number is 12345. The number has only 5 digits. Since the user is asking for the sixth digit, which does not exist, the output is -1.

Change Theme Language Java 7

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String n = sc.next();
        int k = sc.nextInt();

        if (k > n.length() || k <= 0) {
            System.out.println(-1);
        } else {
            System.out.println(n.charAt(k - 1));
        }
    }
}

```

Line: 17 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Looping - Unique-Digit Numbers for Ticket Validation

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine that a theme park is issuing tickets with unique serial numbers. For a special promotion, the park wants to count how many tickets between two given serial numbers have no repeated digits. The theme park administration wants to automate this process to ensure accurate counts and streamline operations.

Your task is to write a program to determine the total number of tickets (or integers) between two given serial numbers n1 and n2 (both inclusive) that have all unique digits.

Input Format

1. The first line contains a non-negative integer n1.
2. The second line contains a non-negative integer n2 such that n1

Constraints

NA

Output Format

The output should be an integer representing the count of numbers between n1 and n2 that have no repeated digits.

Sample Input 0

```
10
50
```

Sample Output 0

```
37
```

Explanation 0

- The numbers between 10 and 50 that have no repeated digits are: 10, 12, 13, ..., 49, 50.
- There are 37 such numbers.

Sample Input 1

```
123
130
```

Sample Output 1

```
8
```

Explanation 1

- The numbers between 123 and 130 that have no repeated digits are: 123, 124, 125, 126, 127, 128.
- The number 129 and 130 are not counted because 129 has unique digits, but 130 has a repeated '0'.

Change Theme Language Java 7

```java
import java.util.*;

public class Main {

    static boolean uniqueDigits(int n) {
        boolean[] seen = new boolean[10];

        while (n > 0) {
            int digit = n % 10;
            if (seen[digit]) {
                return false;
            }
            seen[digit] = true;
            n /= 10;
        }
        return true;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

Line: 36 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Looping - Inventory Check for Rare Items

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a warehouse management system, rare items are considered special if their item ID is a Strong Number. A Strong Number is a number in which the sum of the factorial of each digit is equal to the number itself. Your task is to create a program that checks whether the given item ID is a Strong Number or not.

Imagine a warehouse manager who needs to quickly identify rare items for cataloging. If an item ID is a Strong Number, it should be flagged as "Rare Item," otherwise, it should be marked as "Common Item."

Input Format

The first line of input contains an integer N, which represents the item ID.

Constraints

NA

Output Format

Print "Rare Item" if the item ID is a Strong Number. Print "Common Item" if the item ID is not a Strong Number.

Sample Input 0

```
145
```

Sample Output 0

```
Rare Item
```

Explanation 0

- The digits of 145 are 1, 4, and 5.
- The factorial of 1 is 1, the factorial of 4 is 24, and the factorial of 5 is 120.
- The sum of factorials: 1 + 24 + 120 = 145, which is equal to the item ID.
- Hence, the item ID 145 is a Strong Number, so the output is "Rare Item."

Sample Input 1

```
123
```

Sample Output 1

```
Common Item
```

Explanation 1

- The digits of 123 are 1, 2, and 3.
- The factorial of 1 is 1, the factorial of 2 is 2, and the factorial of 3 is 6.
- The sum of factorials: 1 + 2 + 6 = 9, which is not equal to the item ID.
- Hence, the item ID 123 is not a Strong Number, so the output is "Common Item."

Change Theme Language Java 7

```java
import java.util.*;

public class Main {

    static int factorial(int n) {
        int fact = 1;
        for (int i = 1; i <= n; i++) {
            fact *= i;
        }
        return fact;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int temp = n;
        int sum = 0;

        while (temp > 0) {
```

Line: 33 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Looping - Calculating Plant Growth Stages

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

A biologist is studying the growth patterns of a rare plant species, which follows a unique pattern similar to the Fibonacci sequence. The plant's growth stages, in terms of the number of branches, can be predicted using this sequence: 0, 1, 1, 2, 3, 5, 8, 13, and so on. Given the number of months, the biologist wants to know how many branches the plant will have.

Your task is to write a program that, given the number of months N, computes the number of branches the plant will have using the Fibonacci sequence. The biologist needs this information to understand the plant's future growth and to plan the necessary resources.

Input Format

An integer N representing the number of months.

Constraints

NA

Output Format

Print the number of branches as an integer.

Sample Input 0

```
6
```

Sample Output 0

```
0 1 1 2 3 5 8
```

Explanation 0

- The Fibonacci sequence is 0, 1, 1, 2, 3, 5, 8...
- The 6th term in the Fibonacci sequence is 8, so after 6 months, the plant will have 8 branches.

Sample Input 1

```
9
```

Sample Output 1

```
0 1 1 2 3 5 8 13 21 34
```

Explanation 1

- The Fibonacci sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, 34...
- The 9th term in the Fibonacci sequence is 34, so after 9 months, the plant will have 34 branches

Change Theme Language Java 7

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int a = 0, b = 1;

        for (int i = 0; i <= n; i++) {
            System.out.print(a);

            if (i < n) {
                System.out.print(" ");
            }

            int c = a + b;
            a = b;
            b = c;
```

Line: 24 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Array 2D - Irene's Audience Arrangement Checker

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

Irene, one of the event organizers, has arranged the audience in a matrix format where boys are represented by 1s and girls by 0s. To win a prize, she must ensure that all girls (0s) are positioned below the main diagonal of the matrix. If she successfully arranges the audience according to this condition, she will be rewarded; otherwise, she will not.

Your task is to help the judges determine if Irene's arrangement is valid by checking if the matrix is an upper triangular matrix.

Problem Statement

Write a program that checks whether a given matrix is an upper triangular matrix. A matrix is considered upper triangular if all the elements below the main diagonal are zero.

Input Format

1. The first line contains an integer n, representing the number of rows and columns of the square matrix.
2. The next n lines contain space-separated integers representing the matrix.

Constraints

The matrix is a square matrix (i.e., the number of rows equals the number of columns).

Output Format

Print "Upper triangular matrix" if the condition is satisfied; otherwise, print "Not an Upper triangular matrix"

Sample Input 0

```
3
1 2 3
0 1 2
0 0 1
```

Sample Output 0

```
Upper triangular matrix
```

Explanation 0

In this input, all elements below the diagonal are 0, hence it is an upper triangular matrix.

Sample Input 1

```
3
0 0 0
0 8 0
4 9 7
```

Sample Output 1

```
Not an Upper triangular matrix
```

Explanation 1

In this input, the element 4 below the diagonal is not 0, hence it is not an upper triangular matrix.

Change Theme Language Java 7

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);

        int n = scan.nextInt();

        int[][] arr = new int[n][n];

        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                arr[i][j] = scan.nextInt();
            }
        }

        boolean isUpper = true;

        for(int i=1;i<n;i++){
            for(int j=0;j<i;j++){
                if(arr[i][j]!=0){
                    isUpper = false;
                    break;
                }
            }
            if(!isUpper){
                break;
```

Line: 43 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Arrays 2D - Robotic Warehouse Navigation - Snake Pattern Traversal

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

In a modern warehouse, a robot is tasked with scanning items in storage shelves arranged in a grid-like format. The robot moves in a snake-like pattern to ensure that all shelves are scanned efficiently. Specifically, the robot starts at the first row and scans from left to right, then moves to the next row and scans from right to left, and so on.

Write a program that outputs the order in which the robot scans the items in a given matrix in a snake pattern.

Input Format

1. The first line contains two integers, m (number of rows) and n (number of columns).
2. The next m lines contain n integers each, representing the item IDs on the storage shelves.

Constraints

- 1≤m,n≤1001 \leq m, n \leq 1001≤m,n≤100
- Item IDs are integers ranging from 111 to 100010001000.

Output Format

Print the item IDs in the order the robot scans them in a single line, separated by spaces.

Sample Input 0

```
3 4
1 2 3 4
5 6 7 8
9 10 11 12
```

Sample Output 0

```
1 2 3 4 8 7 6 5 9 10 11 12
```

Explanation 0

Input Matrix (Item IDs on Shelves)

1 2 3 4

5 6 7 8

9 10 11 12

The robot starts at the first row and moves left to right: 1 2 3 4 It then moves to the second row and scans from right to left: 8 7 6 5 Finally, it moves to the third row and scans from left to right: 9 10 11 12

Output

The order of item IDs in the snake pattern is 1 2 3 4 8 7 6 5 9 10 11 12.

Sample Input 1

```
2 3
1 2 3
```

```
4 5 6
```

Sample Output 1

```
1 2 3 6 5 4
```

Change Theme Language Java 7

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);

        int n = scan.nextInt();
        int m = scan.nextInt();

        int[][] arr = new int[n][m];
        for(int i=0;i<n;i++){
            for(int j=0;j<m;j++){
                arr[i][j] = scan.nextInt();
            }
        }

        for(int i=0;i<n;i++){
            if(i%2==0){
                for(int j=0;j<m;j++){
                    System.out.print(arr[i][j]+" ");
                }
            }
            else{
                for(int j=m-1;j>=0;j--){
                    System.out.print(arr[i][j]+" ");
                }
            }
```

Line: 36 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Arrays 2D - Identifying Maximum Defect Levels in Production Batches

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

In a manufacturing unit, a quality control team is tasked with monitoring defect levels across multiple production batches. The defect levels for each item in a batch are recorded in a matrix format, where each row represents a production batch and each column represents the defect level of an item within that batch.

The quality control team needs to find the maximum defect level in each production batch to identify the most defective item in each batch. Write a program to help them identify the maximum defect levels for all production batches.

Input Format

1. The first line contains two integers, m (number of production batches) and n (number of items in each batch).
2. The next m lines contain n integers each, representing the defect levels of items in each batch.

Constraints

NA

Output Format

Output m integers, each representing the maximum defect level in the corresponding production batch.

Sample Input 0

```
3 4
5 3 8 6
2 9 4 7
1 6 3 8
```

Sample Output 0

```
8
9
8
```

Explanation 0

Input Matrix (Defect Levels)

Copy code

5 3 8 6

2 9 4 7

1 6 3 8

Batch 1: The maximum defect level is 8.

Batch 2: The maximum defect level is 9.

Batch 3: The maximum defect level is 8.

Output

The program outputs the maximum defect level for each batch, which helps the quality control team quickly assess the severity of defects in each batch.

Sample Input 1

```
2 3
1 2 3
4 5 6
```

Sample Output 1

```
3
6
```

Change Theme Language Java 7

```
1  import java.io.*;
2  import java.util.*;
3  import java.text.*;
4  import java.math.*;
5  import java.util.regex.*;
6
7  public class Solution {
8
9      public static void main(String[] args) {
10         Scanner scan = new Scanner(System.in);
11
12         int n = scan.nextInt();
13         int m = scan.nextInt();
14
15         for(int i=0;i<n;i++){
16             int[] arr = new int[m];
17
18             for(int j=0;j<m;j++){
19                 arr[j] = scan.nextInt();
20             }
21
22             int max = arr[0];
23             for(int j=1;j<m;j++){
24                 if(arr[j]>max){
25                     max = arr[j];
26                 }
27             }
28             System.out.println(max);
29         }
30     }
31 }
```

Line: 32 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Arrays 2D - Counting Sorted Production Lines

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

In a manufacturing plant, items produced in assembly lines are arranged in rows for quality control. Each row represents a production line, and each item in the row has a quality score. The plant's quality assurance team needs to identify how many production lines have items sorted in non-decreasing order based on their quality scores.

Write a program that counts the number of rows in the matrix that are sorted in non-decreasing order.

Input Format

1. The first line contains two integers, m (number of rows) and n (number of columns).
2. The next m lines contain n integers each, representing the quality scores of items in each production line.

Constraints

- $1 \le m,n \le 100$
- The quality scores are integers ranging from −1000 to 1000.

Output Format

Print the total count of rows that are sorted in non-decreasing order.

Sample Input 0

```
4 5
10 20 30 40 50
15 10 5 0 -5
5 5 6 7 8
9 11 10 13 14
```

Sample Output 0

```
2
```

Explanation 0

Input Matrix (Quality Scores of Items in Production Lines)

10 20 30 40 50 -> Sorted

15 10 5 0 -5 -> Not sorted

5 5 6 7 8 -> Sorted

9 11 10 13 14 -> Not sorted

The 1st and 3rd rows are sorted in non-decreasing order, so the output is 2.

Output

The program counts the rows where quality scores are sorted in non-decreasing order.

Sample Input 1

```
3 4
1 2 3 4
```

```
5 4 3 2
7 8 8 9
```

Sample Output 1

```
2
```

Change Theme Language Java 7

```
public class Main {
    public static void main(String[] args) {
        int m = sc.nextInt();
        int n = sc.nextInt();

        int count = 0;

        for (int i = 0; i < m; i++) {
            int[] row = new int[n];

            for (int j = 0; j < n; j++) {
                row[j] = sc.nextInt();
            }

            boolean sorted = true;

            for (int j = 0; j < n - 1; j++) {
                if (row[j] > row[j + 1]) {
                    sorted = false;
                    break;
                }
            }

            if (sorted) {
                count++;
            }
        }

        System.out.println(count);
        sc.close();
    }
}

```

Line: 37 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Arrays 2D - Power Grid Monitoring - Computing Diagonal Load Balances

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a power distribution center, the load levels are monitored and recorded in a grid-like structure to analyze how power is being distributed across different regions. The grid is represented as an n×n matrix, where each element represents the load level at a specific point in the grid.

The engineers are interested in efficiently calculating the sum of load levels along the two main diagonals:

- Primary diagonal: From the top-left to the bottom-right.
- Secondary diagonal: From the top-right to the bottom-left.

Write a program to compute the sum of the load levels along both diagonals.

Input Format

1. The first line contains a single integer, n(the size of the square matrix).
2. The next nnn lines contain nnn integers each, representing the load levels in the grid.

Constraints

NA

Output Format

Print two integers: the sum of the primary diagonal and the sum of the secondary diagonal, separated by a space.

Sample Input 0

```
3
5 2 3
8 6 4
1 9 7
```

Sample Output 0

```
18 10
```

Explanation 0

Input Matrix (Load Levels):

5 2 3

8 6 4

1 9 7

Primary Diagonal: 5+6+7=18

Secondary Diagonal: 3+6+1=10 Output

The program outputs 18 10, which represents the sum of the primary and secondary diagonals, respectively.

Sample Input 1

```
4
10 2 3 4
```

```
5 11 6 7
8 9 12 13
14 15 16 17
```

Sample Output 1

```
50 33
```

Change Theme Language Java 7

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);

        int n = scan.nextInt();
        int[][] arr = new int[n][n];

        int sum1 = 0;
        int sum2 = 0;
        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                arr[i][j] = scan.nextInt();

                if(i==j){
                    sum1 += arr[i][j];
                }

                if(i+j==n-1){
                    sum2 += arr[i][j];
                }
            }
        }
        System.out.println(sum1+" "+sum2);
    }
}
```

Line: 33 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Array 2D - Security Camera Image Processing System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

A security system captures images of an area in a binary format using a grid of pixels, where 1 represents a white pixel (presence of light) and 0 represents a black pixel (absence of light). The images are used to monitor and analyze the area for unusual activity.

However, to simplify the analysis, the system needs to process each image in two steps:

- Flip the Image Horizontally: Reverse each row of the image.
- Invert the Image: Replace each 0 with 1 and each 1 with 0.

Write a program to process the images using the described operations and return the modified images.

Input Format

1. The first line contains an integer n, representing the dimensions of the square image matrix.
2. The next n lines contain n binary integers (0 or 1) each, representing the pixel values of the image.

Constraints

NA

Output Format

Output the modified image matrix after flipping and inverting.

Sample Input 0

```
3
1 1 0
1 0 1
0 0 0
```

Sample Output 0

```
1 0 0
0 1 0
1 1 1
```

Explanation 0

Original Image

1 1 0

1 0 1

0 0 0

Step 1 - Flip Horizontally

Row 1: [1, 1, 0] -> [0, 1, 1]

Row 2: [1, 0, 1] -> [1, 0, 1]

Row 3: [0, 0, 0] -> [0, 0, 0]

Result after flipping:

0 1 1

1 0 1

0 0 0

Step 2 - Invert

Row 1: [0, 1, 1] -> [1, 0, 0]

Row 2: [1, 0, 1] -> [0, 1, 0]

Row 3: [0, 0, 0] -> [1, 1, 1]

Result after inverting:

1 0 0

0 1 0

1 1 1

Sample Input 1

```
2
1 0
0 1
```

Sample Output 1

```
1 0
0 1
```

Change Theme Language Java 7

```
1  import java.io.*;
2  import java.util.*;
3  import java.text.*;
4  import java.math.*;
5  import java.util.regex.*;
6
7  public class Solution {
8
9      public static void main(String[] args) {
10         Scanner scan = new Scanner(System.in);
11
12         int n = scan.nextInt();
13
14         int[][] arr = new int[n][n];
15
16         // Read matrix
17         for (int i = 0; i < n; i++) {
18             for (int j = 0; j < n; j++) {
19                 arr[i][j] = scan.nextInt();
20             }
21         }
22
23         // Flip horizontally + invert
24         for (int i = 0; i < n; i++) {
25
26             int left = 0;
27             int right = n - 1;
28
29             while (left <= right) {
30
31                 // Swap and invert at the same time
```

```
        int temp = arr[i][left];
```

Line: 51 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Arrays 2D - Warehouse Inventory Sum Calculation

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a warehouse, products are stored in a grid-like arrangement where each cell in the grid represents the quantity of a particular product in a specific section. To ensure inventory accuracy and efficient restocking, the warehouse management system needs to calculate the total quantity of products for each row and each column of the grid.

Write a program to help the warehouse staff calculate the sum of quantities for each row and each column in the grid.

Input Format

1. The first line of input contains an integer n, which represents the number of rows and columns in the grid (n x n grid).
2. The next n lines each contain n integers, representing the quantity of products in each cell of the grid.

Constraints

NA

Output Format

The output should include:

- The sum of each row, printed on a new line.
- The sum of each column, printed on a new line after the row sums.

Sample Input 0

```
3
2 3 4
5 6 7
8 9 10
```

Sample Output 0

```
Row sums:
9
18
27
Column sums:
15
18
21
```

Explanation 0

Input Details

The warehouse has a 3x3 grid with the quantities:

2 3 4

5 6 7

8 9 10

Row Sums

Row 1: 2 + 3 + 4 = 9

Row 2: 5 + 6 + 7 = 18

Row 3: 8 + 9 + 10 = 27

Column Sums

Column 1: 2 + 5 + 8 = 15

Column 2: 3 + 6 + 9 = 18

Column 3: 4 + 7 + 10 = 21

Sample Input 1

```
2
1 2
3 4
```

Sample Output 1

```
Row sums:
3
7
Column sums:
4
6
```

Explanation 1

Input Details

The warehouse has a 2x2 grid with the quantities:

1 2

3 4

Row Sums

Row 1: 1 + 2 = 3

Row 2: 3 + 4 = 7

Column Sums

Column 1: 1 + 3 = 4

Column 2: 2 + 4 = 6

Change Theme Language Java 7

```
1  import java.io.*;
2  import java.util.*;
3  import java.text.*;
4  import java.math.*;
5  import java.util.regex.*;
6
7  public class Solution {
8
9      public static void main(String[] args) {
10         Scanner scan = new Scanner(System.in);
11         int n = scan.nextInt();
12
13         int[][] arr = new int[n][n];
14         for(int i=0;i<n;i++){
15             for(int j=0;j<n;j++){
```

```
16                  arr[i][j] = scan.nextInt();
17              }
18          }
19          System.out.println("Row sums:");
20          for(int i=0;i<n;i++){
21              int row = 0;
22              for(int j=0;j<n;j++){
23                  row += arr[i][j];
24              }
25              System.out.println(row);
26          }
27          System.out.println("Column sums:");
28          for(int j=0;j<n;j++){
29              int col = 0;
30              for(int i=0;i<n;i++){
31                  col += arr[i][j];
32              }
```

Line: 37 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Arrays 2D - Warehouse Shelf Max Quantity Finder

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

In a large warehouse, products are organized on a series of vertical shelves arranged in rows and columns. The warehouse manager needs to know the maximum quantity of items available in each column to make better decisions for restocking. Write a program that calculates the maximum quantity of items present in each column.

Input Format

1. The first line of input contains an integer n, which represents the number of rows and columns in the grid (n x n grid).
2. The next n lines each contain n integers, representing the quantity of items in each cell of the grid.

Constraints

NA

Output Format

Print the maximum quantity for each column on a new line.

Sample Input 0

```
3
5 1 9
2 8 6
7 4 3
```

Sample Output 0

```
7
8
9
```

Explanation 0

Input Details

The warehouse has a 3x3 grid representing the quantity of items:

5 1 9

2 8 6

7 4 3

Maximum Quantity in Each Column

Column 1: The values are 5, 2, and 7. The maximum is 7.

Column 2: The values are 1, 8, and 4. The maximum is 8.

Column 3: The values are 9, 6, and 3. The maximum is 9.

Sample Input 1

```
2
10 15
```

```
20 5
```

Sample Output 1

```
20
15
```

Explanation 1

Input Details

The warehouse has a 2x2 grid representing the quantity of items:

10 15

20 5

Maximum Quantity in Each Column

Column 1: The values are 10 and 20. The maximum is 20.

Column 2: The values are 15 and 5. The maximum is 15.

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);

        int n = scan.nextInt();

        int[][] arr = new int[n][n];
        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                arr[i][j] = scan.nextInt();
            }
        }
        for(int j=0;j<n;j++){
            int max = arr[0][j];
            for(int i=0;i<n;i++){
                if(arr[i][j]>max){
                    max = arr[i][j];
                }
            }
            System.out.println(max);
        }
    }
}

```

Line: 31 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Blog | Scoring | Environment | FAQ | About Us | Helpdesk | Careers | Terms Of Service | Privacy Policy

Strings - Seat Allocation Check for Graduation Ceremony

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

During the annual graduation ceremony, the organizers have arranged 26 seats, labeled from A to Z, for special guests. Each seat should ideally be placed in the order of the English alphabet, corresponding to the ordinal number of the letter (i.e., A at position 1, B at position 2, and so on up to Z at position 26). Due to a last-minute shuffle, the seats have been arranged randomly, and the organizer needs your help to check how many seats are correctly positioned.

Write a program that takes a string of randomly arranged seat labels and determines how many of these seats are in the correct alphabetical position.

Input Format

A single string representing the current arrangement of seat labels.

Constraints

- The string contains only uppercase or lowercase English letters.
- Length of the string≤26\text{Length of the string} \leq 26Length of the string≤26
- The same alphabetical positions apply for both uppercase and lowercase letters.

Output Format

A single integer representing the number of seats that are correctly positioned.

Sample Input 0

```
abcxyzgh
```

Sample Output 0

```
5
```

Explanation 0

In this input:

- Letters a, b, c, g, and h are correctly positioned at 1, 2, 3, 7, and 8 respectively.
- The rest of the letters are not in their correct positions.

Sample Input 1

```
abcxyz
```

Sample Output 1

```
3
```

Explanation 1

In this input:

- Letters a, b, and c are correctly positioned at positions 1, 2, and 3, respectively.
- The other letters x, y, and z are not in their correct positions.

Change Theme Language Java 7

```
1  import java.util.*;
2  public class Main {
3      public static void main(String[] args) {
4          Scanner sc = new Scanner(System.in);
5          String s = sc.nextLine();
6          int count = 0;
7          for (int i = 0; i < s.length(); i++) {
8              char ch = Character.toLowerCase(s.charAt(i));
9              if (ch - 'a' + 1 == i + 1) {
10                 count++;
11             }
12         }
13         System.out.println(count);
14     }
15 }
16
```

Line: 4 Col: 45

Upload Code as File | Test against custom input | Run Code | Submit Code

Strings - Reversing Words for a Speech Teleprompter

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

Imagine you are working on developing software for a teleprompter used during speeches. Sometimes, speakers want to practice their lines by reading the sentences in reverse order to improve their memory and articulation skills. Your task is to write a program that reverses the order of words in a given sentence without using the split method. This will help the speakers rehearse their lines unconventionally.

You are given a string S containing a sentence. Write a program to reverse the order of occurrence of words in the sentence while keeping the individual words unchanged. Do not use the split method. Ensure that all words in your output are followed by a single space.

Input Format

A string S containing a sentence with multiple words separated by spaces.

Constraints

- Words in the input are separated by a single space.
- There are no leading or trailing spaces in the input.
- The input string S will contain only alphabetic characters and spaces.

Output Format

Print the sentence with the order of words reversed, separated by single spaces.

Sample Input 0

```
Hello world
```

Sample Output 0

```
world Hello
```

Explanation 0

The words "Hello" and "world" have been reversed to "world Hello".

Sample Input 1

```
Coding is awesome
```

Sample Output 1

```
awesome is Coding
```

Explanation 1

The words "Coding", "is", and "awesome" have been reversed to "awesome is Coding".

Change Theme Language Java 7

```java
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int end = s.length() - 1;
        for (int i = s.length() - 1; i >= 0; i--) {
            if (s.charAt(i) == ' ') {
                for (int j = i + 1; j <= end; j++) {
                    System.out.print(s.charAt(j));
                }
                System.out.print(" ");
                end = i - 1;
            }
        }
        for (int i = 0; i <= end; i++) {
            System.out.print(s.charAt(i));
        }
    }
}

```

Line: 21 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Strings - Lower Case Count

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Lewis is given a string S of length N. He likes lowercase letters more than the upper case letters and wants to find their count. Your task is to help him find and return a string representing the lowercase letters and the total count.

Input Format

- A string
- An integer representing the length of string

Constraints

NA

Output Format

Return a string representing the lowercase letters and the total count.

Sample Input 0

```
HelloWorld
10
```

Sample Output 0

```
elloorld 8
```

Sample Input 1

```
JavaProgramming
15
```

Sample Output 1

```
avarogramming 13
```

Change Theme Language Java 7

```
1  import java.util.*;
2  public class Main {
```

```
2   public class Main {
3       public static void main(String[] args) {
4           Scanner sc = new Scanner(System.in);
5           String S = sc.nextLine();
6           int N = sc.nextInt();
7           String result = "";
8           int count = 0;
9           for (int i = 0; i < N; i++) {
10              char ch = S.charAt(i);
11              if (Character.isLowerCase(ch)) {
12                  result += ch;
13                  count++;
14              }
15          }
16          System.out.println(result + " " + count);
17      }
18  }
19
```

Line: 11 Col: 45

Upload Code as File | Test against custom input | Run Code | Submit Code

Strings - Kindergarten Sentence Building

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

An English school teacher is helping her kindergarten students learn to construct sentences. As part of the learning process, she asks the students to identify the word in each sentence that has the maximum number of alphabets. Your task is to write a program that takes a sentence as input and outputs the number of characters in the longest word.

Note

The sentence can contain uppercase letters, lowercase letters, special characters, and numbers, but only alphabetic characters in the words are counted.

Input Format

The input consists of a single string, which is the sentence.

Constraints

NA

Output Format

The output should be an integer representing the length of the longest word in the sentence.

Sample Input 0

```
Knowledge is the greatest gift
```

Sample Output 0

```
9
```

Explanation 0

Words and their number of alphabetic characters:

- Knowledge: 9
- is: 2
- the: 3
- greatest: 8
- gift: 4

The longest word is "Knowledge" with 9 characters.

Sample Input 1

```
Length of the longest word
```

Sample Output 1

```
7
```

Explanation 1

Words and their number of alphabetic characters:

- Length: 6
- of: 2
- the: 3
- longest: 7
- word: 4

The longest word is "longest" with 7 characters.

Change Theme Language Java 7

```
1  import java.util.*;
2  public class Main {
3      public static void main(String[] args) {
4          Scanner sc = new Scanner(System.in);
5          String s = sc.nextLine();
6          int max = 0, count = 0;
7          for (int i = 0; i < s.length(); i++) {
8              char ch = s.charAt(i);
9              if (Character.isLetter(ch)) {
10                 count++;
11             } else {
12                 if (count > max) {
13                     max = count;
14                 }
15                 count = 0;
16             }
17         }
18         if (count > max) {
19             max = count;
20         }
21         System.out.println(max);
22     }
23 }
24
```

Line: 17 Col: 10

Upload Code as File Test against custom input Run Code Submit Code

Blog | Scoring | Environment | FAQ | About Us | Helpdesk | Careers | Terms Of Service | Privacy Policy

Strings - Fannys occurrence

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

Fanny is given a string along with the string which contains single character x. She has to remove the character x from the given string. Help her write a function to remove all occurrences of x character from the given string.

Input Format

1. Input string s
2. String containing any character x

Constraints

NA

Output Format

String without the occurrence of character x

Sample Input 0

```
welcome to mettl
l
```

Sample Output 0

```
wecome to mett
```

Explanation 0

As l is the character which is required to be removed, therefore all the occurrences of l are removed, keeping all other characters.

Sample Input 1

```
Lord out of Rings
o
```

Sample Output 1

```
Lrd ut f Rings
```

Explanation 1

As o is the character which is required to be removed, therefore all the occurrences of o are removed, keeping all other characters.

Change Theme Language Java 7

```java
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        char x = sc.nextLine().charAt(0);
        String result = "";
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) != x) {
                result += s.charAt(i);
            }
        }
        System.out.println(result);
    }
}

```

Line: 16 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Strings - Data Compression for Server Log Optimization

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a large IT company, server logs are generated continuously. To save storage space, these logs need to be compressed efficiently. One common method is to replace multiple consecutive occurrences of the same character with a single occurrence. Additionally, since logs are often analyzed in reverse chronological order, the compressed logs must be presented in reverse order.

Your task is to write a program that takes a server log string, compresses it by removing consecutive duplicates, and then outputs the compressed string in reverse order.

Input Format

A string S representing the server log, with a maximum length of 50 characters.

Constraints

NA

Output Format

Print the compressed string in reverse order.

Sample Input 0

```
aaabbccddeffggg
```

Sample Output 0

```
gfedcba
```

Explanation 0

- The input string aaabbccddeffggg is first compressed to abcdefg by replacing multiple consecutive occurrences of each character with a single occurrence.
- The compressed string is then reversed to give gfedcba.

Sample Input 1

```
xxxyyyzzzppqqqrrrss
```

Sample Output 1

```
srqpzyx
```

Explanation 1

- The input string xxxyyyzzzppqqqrrrss is first compressed to sqrpzyx.
- The compressed string is then reversed to give srqpzyx.

Change Theme Language Java 7

```
1  import java.util.*;
2  public class Main {
3      public static void main(String[] args) {
4          Scanner sc = new Scanner(System.in);
5          String s = sc.nextLine();
6          String compressed = "";
7          compressed += s.charAt(0);
8          for (int i = 1; i < s.length(); i++) {
9              if (s.charAt(i) != s.charAt(i - 1)) {
10                 compressed += s.charAt(i);
11             }
12         }
13         for (int i = compressed.length() - 1; i >= 0; i--) {
14             System.out.print(compressed.charAt(i));
15         }
16     }
17 }
18
```

Line: 18 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Strings - Character Count

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

You are given a string S of length N. Your friend wants to know the number of times his favorite letter C occurs in the string. Your task is to help your friend find and return an integer value representing the number of times a character occurs in a particular string.

Note

All the characters in the strings are in lowercase.

Input Format

1. A string S
2. An integer N, representing the length of string
3. A character C

Constraints

NA

Output Format

Return an integer value representing the number of times a character occurs in a particular string

Sample Input 0

```
helloworld
10
l
```

Sample Output 0

```
3
```

Explanation 0

Here the given string is "helloworld" and the length of the string is 10 and the character whose count is to be found is "l". The number of times "l" occurs in the string is 3. Therefore, 3 is returned as the output.

Sample Input 1

```
mercermettl
11
t
```

Sample Output 1

2

Explanation 1

Here the given string is "mercermettl" and the length of the string is 11 and the AOS character whose count is to be found is "t". The number of times "t" occurs in the string is 2. Therefore, 2 is returned as the output.

Change Theme Language Java 7

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String S = sc.nextLine();
        int N = sc.nextInt();
        char C = sc.next().charAt(0);
        int count = 0;
        for (int i = 0; i < N; i++) {
            if (S.charAt(i) == C) {
                count++;
            }
        }
        System.out.println(count);
    }
}

```

Line: 17 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Strings - Character Count 1

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Mario wants to determine whether any letter in a given word is repeated more than three times. If any letter appears more than three times, Mario should discard the word. Write a program that takes a single word as input and outputs the highest number of times any letter appears within that word. If this count exceeds three, Mario can discard the word.

Input Format

A single string value consisting of lowercase alphabets (a-z).

Constraints

The input string will only contain lowercase alphabets: str = {a-z}.

Output Format

A positive integer representing the highest number of times any letter appears in the input word.

Sample Input 0

```
trumpet
```

Sample Output 0

```
2
```

Explanation 0

- The word "trumpet" has the letter 't' repeated 2 times, and all other letters appear only once.
- The highest number of repeating letters is 2.

Sample Input 1

```
reiterate
```

Sample Output 1

```
3
```

Explanation 1

- The word "reiterate" has the letter 'e' repeated 3 times, and other letters appear once or twice.
- The highest number of repeating letters is 3.

Change Theme Language Java 7

```java
import java.util.*;
public class Main{
    public static void main(String[] args){
        Scanner ss = new Scanner(System.in);
        String s = ss.nextLine();
        int max = 0;
        for(char ch='a';ch<='z';ch++){
            int count= 0;
            for(int i=0;i<s.length();i++){
                if(s.charAt(i)==ch){
                    count++;
                }
            }
            if(count>max){
                max=count;
            }
        }
        System.out.println(max);
    }
}

```

Line: 21 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Strings - Analyzing Sentence Complexity in Content Writing

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine you are developing a content evaluation tool for writers. The tool assesses the readability and complexity of the text they create, providing a "difficulty quotient" to help gauge whether the content is suitable for a target audience. Words that are considered "hard" might indicate a higher complexity level, while "easy" words suggest a more readable passage. You are tasked with implementing the algorithm that calculates this difficulty quotient based on predefined rules.

You are given a string containing multiple words separated by spaces. A word is considered "hard" if:

1. The number of consonants is greater than the number of vowels.
2. There are three or more consecutive consonants in the word.

Otherwise, the word is considered "easy." Implement a function to count the number of hard and easy words in the string and calculate the difficulty quotient using the formula:

Difficulty quotient = (5 * hard) - (2 * easy)

Return 0 if the input string is null.

Input Format

1. A single string str containing multiple words separated by spaces.
2. The string contains only lowercase English letters and spaces.

Constraints

- The output value lies within the integral range.
- The input string may contain only lowercase alphabets and spaces.
- If the string is null or empty, the output should be 0.

Output Format

An integer representing the difficulty quotient.

Sample Input 0

```
qlewldoaa life ace by fantasy
```

Sample Output 0

```
11
```

Explanation 0

- qlewldoaa: Hard (more consonants than vowels)
- life: Easy

- ace: Easy
- by: Hard (3 consecutive consonants)
- fantasy: Hard (more consonants than vowels)

Calculating Difficulty Quotient:

Difficulty quotient = (5 * 3) - (2 * 2) = 15 - 4 = 11

Sample Input 1

```
openai is awesome
```

Sample Output 1

```
-6
```

Explanation 1

- openai: Easy
- is: Easy
- awesome: Easy

Calculating Difficulty Quotient:

Difficulty quotient = (5 * 0) - (2 * 3) = 0 - 6 = -6

Change Theme Language Java 7

```java
import java.util.Scanner;
public class Main {
    static int difficultyQuotient(String str) {
        if (str == null || str.isEmpty())
            return 0;
        String[] words = str.split(" ");
        int hard = 0;
        int easy = 0;
        for (String word : words) {
            int vowels = 0;
            int consonants = 0;
            int consecutiveConsonants = 0;
            boolean isHard = false;
            for (int i = 0; i < word.length(); i++) {
                char ch = word.charAt(i);

                if (ch == 'a' || ch == 'e' || ch == 'i' ||
                    ch == 'o' || ch == 'u') {
                    vowels++;
                    consecutiveConsonants = 0;
                } else {
                    consonants++;
                    consecutiveConsonants++;

                    if (consecutiveConsonants >= 3) {
                        isHard = true;
```

```
27                                                    }
28                                                ]
```

Line: 43 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Strings - Analyzing Alphanumeric Characters in Official Documents

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine you're working as a software developer for a company that specializes in document management systems. One of the features requested by the clients is a tool to analyze official documents and reports. The tool should be able to count and display the total number of alphanumeric characters in a given document. This is especially useful for formatting and readability analysis or even to ensure compliance with character count limitations set by regulatory bodies.

Your task is to write a program that reads a string (representing the document content) and counts the total number of alphanumeric characters in it, including both letters and numbers. The system should handle the input efficiently and return meaningful results for content analysis.

Problem Statement

Write a program that takes a string as input and counts the total number of alphanumeric characters present in the string. If the input string is empty, return -1. Multiple occurrences of each alphanumeric character should be counted separately.

Input Format

The input consists of a single string str.

Constraints

NA

Output Format

- Output a single integer representing the total count of alphanumeric characters.
- If the input string is empty, output -1.

Sample Input 0

```
Ram scored 91.62% in class 12 Board Exams.
```

Sample Output 0

```
32
```

Explanation 0

- Total alphabetic characters: 26
- Total numeric characters: 6
- Total alphanumeric characters: 26 + 6 = 32

Sample Input 1

Happy New Year 2016!!

Sample Output 1

16

Explanation 1

- Total alphabetic characters: 14
- Total numeric characters: 2
- Total alphanumeric characters: 14 + 2 = 16

Change Theme Language Java 7

```java
import java.util.Scanner;
public class Main {
    static int countAlphanumeric(String str) {
        if (str == null || str.isEmpty())
            return -1;
        int count = 0;
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (Character.isLetterOrDigit(ch)) {
                count++;
            }
        }
        return count;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();
        System.out.println(countAlphanumeric(str));
    }
}

```

Line: 21 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Recursion - Calculating the Total Revenue of Orders

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In an e-commerce platform, you are tasked with calculating the total revenue generated from the first n orders. Each order is associated with a price that corresponds to its order number (i.e., the first order costs 1 unit, the second order costs 2 units, and so on). The company wants to implement a feature where the total revenue from the first n orders is computed using recursion.

Problem

You are given a positive integer n which represents the number of orders. You need to calculate the total revenue generated from the first n orders using recursion.

Input Format

The input consists of a single integer n ($1 \le n \le 1000$), representing the number of orders.

Constraints

NA

Output Format

The output should be a single integer representing the total revenue generated from the first n orders.

Sample Input 0

```
5
```

Sample Output 0

```
15
```

Explanation 0

The total revenue from the first 5 orders can be calculated as the sum of natural numbers from 1 to 5:

Revenue = 1 + 2 + 3 + 4 + 5 = 15

Sample Input 1

```
3
```

Sample Output 1

```
6
```

Explanation 1

The total revenue from the first 3 orders is:

Revenue = 1 + 2 + 3 = 6

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {
     public static int revenue(int n){
        if(n==0){
            return 0;
        }
        return n+revenue(n-1);
    }

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();
        int result = revenue(n);
        System.out.println(result);
    }
}

```

Line: 22 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Recursion - The Fibonacci Sequence for Future Engineers

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In the year 2150, a group of engineering students at the Galactic Institute of Technology is studying algorithms and their applications in computational design. As part of their curriculum, they are learning about recursive functions and their power in solving complex problems.

To enhance their understanding, the instructor has assigned a project where students must calculate Fibonacci numbers using recursion. The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, usually starting with 0 and 1. The task is to write a program that computes the n-th Fibonacci number using a recursive approach.

Write a program that takes a non-negative integer nnn as input and returns the n-th Fibonacci number using recursion.

Input Format

The input consists of a single non-negative integer n.

Constraints

$0 \le n \le 30$

Output Format

Print the n-th Fibonacci number.

Sample Input 0

```
5
```

Sample Output 0

```
5
```

Explanation 0

The Fibonacci sequence up to the 5th term is:

0,1,1,2,3,5

Thus, the 5th Fibonacci number is 5.

Sample Input 1

```
10
```

Sample Output 1

```
55
```

Explanation 1

The Fibonacci sequence up to the 10th term is:

0,1,1,2,3,5,8,13,21,34,55

Thus, the 10th Fibonacci number is 55.

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {
    static int fib(int n){
        if(n<=1){
            return n;
        }
        return fib(n-1)+fib(n-2);
    }

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();
        System.out.println(fib(n));
    }
}

```

Line: 21 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Recursion - Finding the Greatest Common Divisor in Ancient Math

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In the year 3000 BC, in the ancient city of Mesopotamia, mathematicians were discovering the wonders of numbers. Among their many studies, they found the need to simplify fractions, which led to the exploration of the Greatest Common Divisor (GCD). The GCD of two numbers is the largest positive integer that divides both numbers without leaving a remainder.

To help the young mathematicians master this concept, their wise teacher has assigned them a project to calculate the GCD of two positive integers using a recursive method. This task not only enhances their understanding of numbers but also builds their problem-solving skills.

Write a program that takes two positive integers as input and returns their GCD using recursion.

Input Format

The input consists of two positive integers, a and b.

Constraints

$1 \le a,b \le 10^6$

Output Format

Print the GCD of the two integers.

Sample Input 0

```
48 18
```

Sample Output 0

```
6
```

Explanation 0

The GCD of 48 and 18 is 6, as 6 is the largest number that divides both 48 and 18 without leaving a remainder.

Sample Input 1

```
56 98
```

Sample Output 1

```
14
```

Explanation 1

The GCD of 56 and 98 is 14, as 14 is the largest number that divides both 56 and 98 without leaving a remainder

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    static int gcd(int n, int m) {
        if (m == 0) {
            return n;
        }
        return gcd(m, n % m);
    }

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);

        int n = scan.nextInt();
        int m = scan.nextInt();

        System.out.println(gcd(n, m));
    }
}

```

Line: 25 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

CONTEST ENDS IN

1 day

Recursion - Finding the Mighty Element in the Forest

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

In the enchanting forest of Eldoria, the wise old owl named Orin has the unique ability to communicate with the animals. One day, Orin decided to organize a competition among the animals to find the mightiest creature based on their sizes. To do this, he devised a plan to represent the sizes of the animals in an array and asked the clever rabbit, Ruby, to help him find the largest size using a special method: recursion.

Write a program that takes an array of positive integers representing the sizes of different animals and finds the maximum size using recursion.

Input Format

1. The first line contains an integer nnn (the number of animals).
2. The second line contains n positive integers representing the sizes of the animals.

Constraints

- $1 \le n \le 1000$
- Sizes are positive integers.

Output Format

Output the maximum size value from the array

Sample Input 0

```
5
23 12 45 67 34
```

Sample Output 0

```
67
```

Explanation 0

In the array [23, 12, 45, 67, 34], the maximum size is 67, which represents the largest animal.

Sample Input 1

```
4
2 8 1 5
```

Sample Output 1

```
8
```

Explanation 1

In the array [2, 8, 1, 5], the maximum size is 8, which represents the largest animal.

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    static int fmax(int[] arr,int n){
        if(n==0){
            return arr[0];
        }

        int largest = fmax(arr,n-1);

        return Math.max(arr[n-1],largest);
    }

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = scan.nextInt();
        }
        System.out.println(fmax(arr,n));
    }
}

```

Line: 30 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Recursion - Counting Active Bits in a Device Monitoring System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

Imagine a device monitoring system that tracks the state of sensors in a factory. Each sensor is either active or inactive, and the state of all sensors is represented by a binary number. If a sensor is active, its corresponding bit is set to 1; otherwise, it is set to 0. To monitor the overall activity in the factory, you are required to count the number of active sensors (set bits) using a recursive approach.

Write a recursive function int countSetBits(int number) that takes an integer number (representing the state of the sensors in binary form) and returns the count of set bits in the binary representation of number.

Input Format

A single integer number.

Constraints

0≤number≤10^6

Output Format

An integer representing the count of set bits.

Sample Input 0

```
13
```

Sample Output 0

```
3
```

Explanation 0

The binary representation of 13 is 1101, which has three set bits (1s). Therefore, the output is 3.

Sample Input 1

```
8
```

Sample Output 1

```
1
```

Explanation 1

The binary representation of 8 is 1000, which has one set bit (1). Therefore, the output is 1.

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    static int countSetBits(int number) {
        if (number == 0) {
            return 0;
        }

        return (number % 2) + countSetBits(number / 2);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int number = sc.nextInt();

        System.out.println(countSetBits(number));
    }
}

```

Line: 25 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Recursion - Recursive String Length Calculator

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a digital document management system, each document is represented as a string. As part of the system's features, you need to determine the length of a string using a recursive approach. This feature is essential for efficient document analysis without relying on built-in methods.

Your task is to write a program that calculates the length of a given string using recursion.

Input Format

A single string S representing the content of the document.

Constraints

NA

Output Format

An integer representing the length of the string.

Sample Input 0

```
hello
```

Sample Output 0

```
5
```

Explanation 0

The string "hello" has 5 characters, so the output is 5.

Sample Input 1

```
recursion
```

Sample Output 1

```
9
```

Explanation 1

The string "recursion" has 9 characters, so the output is 9.

Change Theme Language Java 7

```
1  import java.io.*;
2  import java.util.*;
```

```
3  import java.text.*;
4  import java.math.*;
5  import java.util.regex.*;
6
7  public class Solution {
8      static int len(String str){
9          if(str.length()==0){
10             return 0;
11         }
12         return 1+len(str.substring(1));
13     }
14
15     public static void main(String[] args) {
16         Scanner scan = new Scanner(System.in);
17         String str = scan.nextLine();
18         int r = len(str);
19
20         System.out.println(r);    }
21 }
22
```

Line: 22 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Recursion - Calculating Factorials for the Space Expedition

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

In the futuristic world of 2140, a team of astronauts is preparing for a journey to Mars. As part of their training, they are required to calculate various mathematical functions, one of which is the factorial of a number. The factorial of a non-negative integer n is the product of all positive integers less than or equal to n.

To streamline their calculations, the team decided to implement a program that uses recursion to compute the factorial. Your task is to help them write a program that calculates the factorial of a given number using a recursive approach.

Write a program that takes a non-negative integer as input and returns its factorial using recursion.

Input Format

The input consists of a single non-negative integer n.

Constraints

NA

Output Format

Print the factorial of the number.

Sample Input 0

```
5
```

Sample Output 0

```
120
```

Explanation 0

The factorial of 5 is calculated as:

$5!=5\times4\times3\times2\times1=120$

Sample Input 1

```
0
```

Sample Output 1

```
1
```

Explanation 1

The factorial of 0 is defined as:

$0!=1$

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static int fact(int n){
        if(n==0){
            return 1;
        }
        return n*fact(n-1);
    }

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);

        int n = scan.nextInt();
        int r = fact(n);
        System.out.println(r);
    }
}

```

Line: 24 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Recursion - Recursive Sum of Array Elements

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a digital library, the system maintains an array of book ratings to analyze the popularity of books. You are tasked with implementing a feature that calculates the total sum of ratings using recursion. This feature will help in the dynamic calculation of total ratings without using traditional iteration methods, enhancing the system's ability to process large data sets efficiently.

Write a program to calculate the sum of elements in an array using recursion.

Input Format

1. A single integer N representing the number of elements in the array.
2. An array of N integers representing the ratings of the books in the library.

Constraints

NA

Output Format

A single integer representing the sum of the array elements.

Sample Input 0

```
5
1 3 2 4 5
```

Sample Output 0

```
15
```

Explanation 0

The sum of the array elements [1, 3, 2, 4, 5] is 1 + 3 + 2 + 4 + 5 = 15.

Sample Input 1

```
3
7 8 10
```

Sample Output 1

```
25
```

Explanation 1

The sum of the array elements [7, 8, 10] is 7 + 8 + 10 = 25.

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    static int add(int[] arr,int n,int sum){
        if (n==0){
            return sum;
        }
        return add(arr,n-1,sum+arr[n-1]);
    }

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);

        int n = scan.nextInt();
        int[] arr = new int[n];

        for(int i=0;i<n;i++){
            arr[i] = scan.nextInt();
        }

        System.out.println(add(arr,n,0));
    }
}

```

Line: 29 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Class and Objects - Wicket Tracking System

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In the world of cricket analysis, match analysts and commentators need precise and organized information about each wicket that falls during the game. The process of recording wicket details is essential for detailed commentary and post-match analysis. This Wicket Tracking System will allow cricket analysts to input the details of every wicket (such as over, ball, wicket type, dismissed player, and bowler) and then display that information in a well-structured format for easy reference.

In this scenario, imagine you are building a system for a cricket commentary team to log each wicket as the game progresses. The system will capture the details of the wicket, like which over and ball it occurred, who the player dismissed was, the type of dismissal, and the bowler responsible.

The data will be captured using objects and stored in memory to allow the commentary team to recall it during and after the game for analysis.

Program Specifications

Class Wicket: Attributes:

- over: Stores the over in which the wicket occurred.
- ball: Stores the ball within the over when the wicket was taken.
- wicketType: Stores the type of dismissal (e.g., LBW, Bowled, Stumped).
- playerName: Stores the name of the player who got out.
- bowlerName: Stores the name of the bowler who took the wicket.

Constructors:

- A 5-argument constructor to initialize all member variables.
- A default constructor with no arguments.

Class Main: Main Method:

- Reads input for the number of wickets.
- Reads each wicket's details using a comma-separated string (split using String.split()).
- Uses an array of Wicket objects to store the information for multiple wickets.
- Outputs the wicket details in a formatted manner.

Input Format

1. The first line contains an integer representing the number of wickets.
2. For each wicket, the user will input the details in the following format:
   - over,ball,wicketType,playerName,bowlerName
   - The details should be separated by commas with no leading or trailing spaces.

Constraints

NA

Output Format

1. The output begins with "Wicket Details".
2. For each wicket, the details will be printed in the following format:
   - Over:
   - Ball:
   - Wicket Type:
   - Player Name:
   - Bowler Name:

Sample Input 0

```
3
5,2,LBW,Gautam Gambir,Aswin
6,6,Bowled,Brad Hogg,Dwayne Bravo
7,3,Stumped,Robin Uthappa,Ravindra Jadeja
```

Sample Output 0

```
Enter the number of wickets
Enter the details of wicket 1
Enter the details of wicket 2
Enter the details of wicket 3
Wicket Details
Over: 5
Ball: 2
Wicket Type: LBW
Player Name: Gautam Gambir
Bowler Name: Aswin
Over: 6
Ball: 6
Wicket Type: Bowled
Player Name: Brad Hogg
Bowler Name: Dwayne Bravo
Over: 7
Ball: 3
Wicket Type: Stumped
Player Name: Robin Uthappa
Bowler Name: Ravindra Jadeja
```

Sample Input 1

```
1
10,4,Caught,AB de Villiers,Moeen Ali
```

Sample Output 1

```
Enter the number of wickets
Enter the details of wicket 1
Wicket Details
Over: 10
Ball: 4
Wicket Type: Caught
Player Name: AB de Villiers
Bowler Name: Moeen Ali
```

Change Theme Language Java 7

```
1  import java.util.Scanner;
2
3  class Wicket {
4      int over;
5      int ball;
6      String wicketType;
7      String playerName;
8      String bowlerName;
9
10     Wicket() {
```

```
10      Wicket() {
11      }
12
13      Wicket(int over, int ball, String wicketType, String playerName, String
        bowlerName) {
14          this.over = over;
15          this.ball = ball;
16          this.wicketType = wicketType;
17          this.playerName = playerName;
18          this.bowlerName = bowlerName;
19      }
```

Line: 59 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Class and Objects - Cricket Match Score Management System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine you are working on a project for a cricket score management system. One of the key features is to record and display details of each inning played in a cricket match. The data includes the team batting in that inning and the runs scored by the team. You need to implement this using object-oriented programming, where each inning's details are stored in an object. The system then prints out the details for both innings in a match.

Program Specifications

Class Name: Innings

- Member Variables:
- String battingTeam — Stores the name of the team batting in the inning.
- Long runs — Stores the total runs scored by the team in the inning.

Constructor:

- A 2-argument constructor that initializes battingTeam and runs.
- A default constructor (no arguments).

Main Class:

- Reads the details of two innings from user input using an array of objects.
- Prints the output in the main method without using getters and setters.

Input Format

1. The first line contains the prompt: "Enter the values for Innings 1."
2. The second line contains the prompt: "Enter the BattingTeam," followed by the user input for the first inning's batting team (e.g., RCB).
3. The third line contains the prompt: "Enter the runs scored," followed by the user input for the first inning's runs (e.g., 190).
4. The fourth line contains the prompt: "Enter the values for Innings 2."
5. The fifth line contains the prompt: "Enter the BattingTeam," followed by the user input for the second inning's batting team (e.g., CSK).
6. The sixth line contains the prompt: "Enter the runs scored," followed by the user input for the second inning's runs (e.g., 190).

Constraints

NA

Output Format

- The first line of the output contains "Innings 1 Details".
- The second line contains "BattingTeam: " followed by the batting team name for the first inning (e.g., "RCB").
- The third line contains "Runs scored: " followed by the runs scored by the first inning's team (e.g., 190).
- The fourth line of the output contains "Innings 2 Details".
- The fifth line contains "BattingTeam: " followed by the batting team name for the second inning (e.g., "CSK").
- The sixth line contains "Runs scored: " followed by the runs scored by the second inning's team (e.g., 190).

Sample Input 0

```
MI
180
KKR
175
```

Sample Output 0

```
Enter the values for Innings 1
Enter the BattingTeam
Enter the runs scored
Enter the values for Innings 2
Enter the BattingTeam
Enter the runs scored
Innings 1 Details
BattingTeam: MI
Runs scored: 180
Innings 2 Details
BattingTeam: KKR
Runs scored: 175
```

Sample Input 1

```
SRH
225
DC
220
```

Sample Output 1

```
Enter the values for Innings 1
Enter the BattingTeam
Enter the runs scored
Enter the values for Innings 2
Enter the BattingTeam
Enter the runs scored
Innings 1 Details
BattingTeam: SRH
Runs scored: 225
Innings 2 Details
BattingTeam: DC
Runs scored: 220
```

Change Theme Language Java 7

```java
import java.util.Scanner;

class Innings {
    String battingTeam;
    Long runs;

    Innings() {
    }

    Innings(String battingTeam, Long runs) {
        this.battingTeam = battingTeam;
        this.runs = runs;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Innings[] innings = new Innings[2];
```

Line: 50 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Class and Objects - Car Dealership Management System

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

A car dealership needs a management system to store and manage details about the cars available for sale. This system will allow users to enter information such as the car's brand, model, year of manufacture, price, and engine capacity. The objective is to maintain an organized inventory of cars and display their details clearly.

Program Specifications

Create a class named Car: Attributes:

- String brand: Stores the brand of the car.
- String model: Stores the model of the car.
- int year: Stores the year of manufacture of the car.
- double price: Stores the price of the car.
- double engineCapacity: Stores the engine capacity of the car in liters.

Constructors:

- Parameterized Constructor:
- Car(String brand, String model, int year, double price, double engineCapacity): Initializes the car's attributes with the provided values.

Methods:

- void displayCarDetails(): Displays the car's information in a structured format.

Create a class named Main:

- Purpose: Captures user input to create a Car object using the constructor and then displays the car's details using the displayCarDetails() method.

Input Format

The program should prompt the user to enter the following details:

1.First input: Brand of the car (String).

2.Second input: Model of the car (String).

3.Third input: Year of manufacture (int).

4.Fourth input: Price of the car (double).

5.Fifth input: Engine capacity (double).

Constraints

NA

Output Format

If the input is valid, the output should display the car details in the following format: Car Details:

- Brand: [brand]
- Model: [model]
- Year: [year]
- Price: $[price]
- Engine Capacity: [engineCapacity] L

If any input is invalid (e.g., year should be greater than 1885, price and engineCapacity should be greater than zero), the output should display: Invalid Input

Sample Input 0

Toyota
Camry
2020
30000.50
2.5

Sample Output 0

Enter the brand of the car:
Enter the model of the car:
Enter the year of manufacture:
Enter the price of the car:
Enter the engine capacity of the car:
Car Details:
Brand: Toyota
Model: Camry
Year: 2020
Price: $30000.5
Engine Capacity: 2.5 L

Sample Input 1

Ford
Mustang
1800
-15000
5.0

Sample Output 1

Enter the brand of the car:
Enter the model of the car:
Enter the year of manufacture:
Enter the price of the car:
Enter the engine capacity of the car:
Invalid Input

Change Theme Language Java 7

```
import java.util.Scanner;

class Car {
    String brand;
    String model;
    int year;
    double price;
    double engineCapacity;

    Car(String brand, String model, int year, double price, double
    engineCapacity) {
        this.brand = brand;
        this.model = model;
        this.year = year;
        this.price = price;
        this.engineCapacity = engineCapacity;
    }

    void displayCarDetails() {
```

```
18      void displayCarDetails() {
19          System.out.println("Car Details: ");
```

Line: 55 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Class and Objects - Laptop Purchase Decision

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Alex and his friend Alicia are planning to purchase a new laptop. They want to compare different laptop models based on key specifications such as brand, processor, operating system, processor range, and RAM. To help them make an informed decision, we will create a program that captures the details of a laptop and displays it in a structured format. This will allow them to easily review the laptops they're considering.

Program Specifications

Create a class named Laptop: Attributes:

- String brandName: Stores the brand of the laptop.
- String processorName: Stores the processor name of the laptop.
- String os: Stores the operating system of the laptop.
- long processorRange: Stores the processor range (in bits).
- long ram: Stores the RAM size (in GB).

Methods:

- void displayLaptopDetails(): Displays the laptop's information in a structured format.

Create a class named Main:

- Purpose: Captures user input to create a Laptop object and then displays the laptop's details using the displayLaptopDetails() method.

Input Format

The program should prompt the user to enter the following details:

- First input: Brand of the laptop (String).
- Second input: Processor name of the laptop (String).
- Third input: OS of the laptop (String).
- Fourth input: Processor range of the laptop (long).
- Fifth input: RAM size of the laptop (long).

Constraints

NA

Output Format

- If the input is valid, the output should display the laptop details in the following format: Laptop Details:

Laptop Brand: [brandName]

Processor Name: [processorName]

OS: [os]

Processor Range: [processorRange] bit

Ram: [ram] GB

- If any input is invalid (e.g., processorRange and ram must be greater than zero), the output should display:Invalid Input

Sample Input 0

```
Dell
Intel Core i5-5675C
Windows
```

64
8

Sample Output 0

Enter the brand of the laptop:
Enter the processor of the laptop:
Enter the OS of the laptop:
Enter the processor range of the laptop:
Enter the ram size of the laptop:
Laptop Details:
Laptop Brand: Dell
Processor Name: Intel Core i5-5675C
OS: Windows
Processor Range: 64 bit
Ram: 8 GB

Sample Input 1

HP
AMD Ryzen 5
Linux
-32
4

Sample Output 1

Enter the brand of the laptop:
Enter the processor of the laptop:
Enter the OS of the laptop:
Enter the processor range of the laptop:
Enter the ram size of the laptop:
Invalid Input

Change Theme Language Java 7

```
1  import java.util.Scanner;
2
3  class Laptop {
4      String brandName;
5      String processorName;
6      String os;
7      long processorRange;
8      long ram;
9
10     Laptop(String brandName, String processorName, String os, long
       processorRange, long ram) {
11         this.brandName = brandName;
12         this.processorName = processorName;
13         this.os = os;
14         this.processorRange = processorRange;
15         this.ram = ram;
16     }
17
18     void displayLaptopDetails() {
19         System.out.println("Laptop Details:");
```

Line: 62 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Class and Objects - Cricket Team Management System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine you are the manager of a professional cricket team. You need a system to keep track of essential information about your team, including the team's name, coach's name, location, players, and the captain. You want to ensure that this information can be entered easily and displayed clearly. This program will allow you to input team details in a structured format and retrieve that information when needed.

Program Specifications

Class Name: Team Attributes:

- String name - Name of the team
- String coach - Coach of the team
- String location - Location of the team
- String players - List of players in the team (comma-separated)
- String captain - Name of the captain

Constructors:

- 5-Argument Constructor: Initializes all member variables.
- Default Constructor: Initializes the object with default values.

Class Name: Main Methods:

- main: Entry point for the program to read team details, create a Team object, and print the details.

Input Format

1. The user is prompted to enter the team details in a single line.
2. The details should be separated by the # character, in the following order:
    - Team name (String)
    - Coach name (String)
    - Location (String)
    - Players (comma-separated String)
    - Captain name (String)

Constraints

NA

Output Format

1. The output displays the details of the team in a structured format.
2. Each piece of information is labelled and printed on a separate line as follows:
    - Team: [Team Name]
    - Coach: [Coach Name]
    - Location: [Location]
    - Players: [List of Players]
    - Captain: [Captain Name]

Sample Input 0

CSK#Stephen Fleming#Chennai#MS Dhoni, Ashwin, Raina, Hussey, Maxwell, Bravo, Morkel, Jadeja, Mohit Sharma, Hayden, du Plessis, Abhinav Muk

Sample Output 0

Enter the team details
Team: CSK
Coach: Stephen Fleming
Location: Chennai
Players: MS Dhoni, Ashwin, Raina, Hussey, Maxwell, Bravo, Morkel, Jadeja, Mohit Sharma, Hayden, du Plessis, Abhinav Mukund
Captain: MS Dhoni

Sample Input 1

MI#Mahela Jayawardene#Mumbai#Rohit Sharma, Hardik Pandya, Krunal Pandya, Pollard, Boult, Bumrah, Ishan Kishan, de Kock, Surya Kumar Yadav

Sample Output 1

Enter the team details
Team: MI
Coach: Mahela Jayawardene
Location: Mumbai
Players: Rohit Sharma, Hardik Pandya, Krunal Pandya, Pollard, Boult, Bumrah, Ishan Kishan, de Kock, Surya Kumar Yadav, Jayant Yadav
Captain: Rohit Sharma

Change Theme Language Java 7

```
1
2  import java.util.*;
3
4  class Team {
5      String name;
6      String coach;
7      String location;
8      String players;
9      String captain;
10
11     void setDetails(String n, String c, String l, String p, String cap) {
12         name = n;
13         coach = c;
14         location = l;
15         players = p;
16         captain = cap;
17     }
18
19     void displayDetails() {
20         System.out.println("Team: " + name);
```

Line: 48 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Blog | Scoring | Environment | FAQ | About Us | Helpdesk | Careers | Terms Of Service | Privacy Policy

Class and Objects - Cricket Delivery Tracker

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

You are tasked with developing a program for a cricket statistics application. The application tracks deliveries in a cricket match. Each delivery in cricket is associated with details such as the over number, ball number, number of runs scored, the batsman who faced the ball, the bowler who bowled the ball, and the non-striker at the other end. To model this, you will create a Delivery class to store the details of a particular delivery, and a Main class to test and display these details.

Program Specifications

Delivery Class:

- Private member variables: over, ball, runs, batsman, bowler, nonStriker.
- Constructors:
- Default constructor.
- Parameterized constructor to initialize all member variables.
- Getters for all member variables (no setters required as the values are initialized through the constructor).

Main Class:

- The main method will handle input and output.
- It will create an instance of Delivery using the parameterized constructor and print the details in the required format.

Input Format

The input will be provided in the following order:

1. Enter the over: An integer value representing the over number.
2. Enter the ball: An integer value representing the ball number within the over.
3. Enter the runs: An integer value representing the number of runs scored in the delivery.
4. Enter the batsman name: A string representing the name of the batsman facing the ball.
5. Enter the bowler name: A string representing the name of the bowler delivering the ball.
6. Enter the nonStriker name: A string representing the name of the non-striker (the other batsman at the crease).

Constraints

NA

Output Format

The output will be displayed in the following order:

1. Over: The over number.
2. Ball: The ball number.
3. Runs: The number of runs scored.
4. Batsman: The name of the batsman.
5. Bowler: The name of the bowler.
6. NonStriker: The name of the non-striker.

Sample Input 0

```
1
1
4
MS Dhoni
```

Dale Steyn
Suresh Raina

Sample Output 0

Enter the over
Enter the ball
Enter the runs
Enter the batsman name
Enter the bowler name
Enter the nonStriker name
Over: 1
Ball: 1
Runs: 4
Batsman: MS Dhoni
Bowler: Dale Steyn
NonStriker: Suresh Raina

Sample Input 1

3
5
6
Virat Kohli
Mitchell Starc
Rohit Sharma

Sample Output 1

Enter the over
Enter the ball
Enter the runs
Enter the batsman name
Enter the bowler name
Enter the nonStriker name
Over: 3
Ball: 5
Runs: 6
Batsman: Virat Kohli
Bowler: Mitchell Starc
NonStriker: Rohit Sharma

Change Theme Language Java 7

```
import java.util.*;
public class Main{
    public static void main(String[]ars){
        Scanner sc = new Scanner(System.in);
        int Over;
        int Ball;
        int Runs;
        String Batsman;
        String Bowler;
        String Nonstriker;
        Over = sc.nextInt();
        Ball = sc.nextInt();
        Runs = sc.nextInt();
        sc.nextLine();
        Batsman = sc.nextLine();
```

```
16          Bowler = sc.nextLine();
17          Nonstriker = sc.nextLine();
18          System.out.println("Enter the over");
19          System.out.println("Enter the ball");
20          System.out.println("Enter the runs");
```

Line: 33 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Class and Objects - Sports Venue Information System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine you are developing a Sports Venue Information System for sports enthusiasts, event organizers, and fans. This system allows users to input details about various sports venues, such as stadiums and arenas, in a simple and structured format. Users can quickly retrieve and view the information about venues where their favorite teams play or where upcoming events are scheduled.

In this application, each venue is represented as an object that holds its name and city. Users can input venue details in a comma-separated format, and the application will utilize the String.split() function to parse and display this information in a user-friendly manner.

Program Specifications

Class Name: Venue

- Member Variables:
- String name (to store the venue's name).
- String city (to store the city where the venue is located).

Class Name: Main

- This class contains the main method to interact with the user and test the Venue class.

Input Format

The user is prompted to enter the venue name (String).

1. This input should be a single line containing the full name of the venue. The user is then prompted to enter the city name (String).
2. This input should also be a single line containing the name of the city.

Constraints

NA

Output Format

- The output displays the details of the venue in a structured format, clearly labeled as "Venue Details."
- Each piece of information is displayed with a label (Venue Name, City Name) followed by its corresponding value.
- The label and its corresponding value should be separated by a colon (:) and a space.
- Each detail is printed on a separate line.

Sample Input 0

```
Wankhede Stadium,Mumbai
```

Sample Output 0

```
Enter the venue details
Venue Details
Venue Name : Wankhede Stadium
City Name : Mumbai
```

Sample Input 1

```
Eden Gardens,Kolkata
```

Sample Output 1

```
Enter the venue details
Venue Details
Venue Name : Eden Gardens
City Name : Kolkata
```


Blog | Scoring | Environment | FAQ | About Us | Helpdesk | Careers | Terms Of Service | Privacy Policy

Class and Objects - Cricket Player Information Tracker

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine you are developing a Cricket Player Information Tracker application that helps coaches, analysts, and fans easily access detailed information about various cricket players. The application will allow users to input player details in a simple format, and then parse that information to display it in a structured way. This will be particularly useful for sports analysts who need quick access to player profiles during matches or training sessions.

In this application, each player is represented as an object that holds their name, country, and skill set. The user can input player details in a comma-separated format, and the application will utilize the String.split() function to parse and display these details cleanly.

Program Specifications

Class Name: Player

- Member Variables:
- String name (to store the player's name).
- String country (to store the player's country).
- String skill (to store the player's skill).

Class Name: Main

- This class contains the main method to interact with the user and test the Player class.

Input Format

The user is prompted to enter the player details as a single line of tet, with each detail separated by a comma.

- The first detail is the player's name (String).
- The second detail is the player's country (String).
- The third detail is the player's skill (String).

Constraints

NA

Output Format

- The output displays the details of the player in a structured format, with each piece of information labelled (Player Name, Country Name, Skill).
- Each label and its corresponding value should be separated by a colon (:) and a space.
- The output should be printed on separate lines, clearly presenting each detail.

Sample Input 0

```
Virat Kohli,India,Batsman
```

Sample Output 0

```
Enter the player details
Player Details
Player Name : Virat Kohli
Country Name : India
Skill : Batsman
```

Sample Input 1

Joe Root,England,Batsman

Sample Output 1

```
Enter the player details
Player Details
Player Name : Joe Root
Country Name : England
Skill : Batsman
```

Change Theme Language Java 7

```java
import java.util.Scanner;

class Player {
    String name;
    String country;
    String skill;

    Player(String name, String country, String skill) {
        this.name = name;
        this.country = country;
        this.skill = skill;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the player details");
        String input = sc.nextLine();
```

Line: 34 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Class and Objects - Cricket Match Score Tracker

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine you are developing a Cricket Match Score Tracker application for a cricket organization. The application is designed to help scorekeepers input and track innings details for various matches. The goal is to easily record the batting team's name, the runs scored in an innings, and the inning number, which helps coaches, players, and fans keep track of the match's progress.

In this application, each inning is represented as an object that holds essential details, making it easier to manage and display information about the ongoing cricket match.

Program Specifications

Class Name: Innings

- Member Variables:
- String number (to store the innings number).
- String battingTeam (to store the name of the batting team).
- Long runs (to store the runs scored by the batting team).
- Method:
- void displayInningsDetails() (to display the details of the innings).

Class Name: Main

- This class contains the main method to interact with the user and test the Innings class.

Input Format

1. The user is prompted to enter the innings number (String).
2. The user is then asked to input the batting team name (String).
3. Finally, the user is prompted to enter the runs scored (Long).

Constraints

NA

Output Format

1. The output displays the details of the innings in a structured format, with each piece of information labeled (Innings number, Batting Team, Runs scored).
2. Each label and its corresponding value should be separated by a colon (:) and a space.
3. The output should be printed on separate lines.

Sample Input 0

```
Firstinnings
CSK
200
```

Sample Output 0

```
Enter the innings number
Enter the BattingTeam
Enter the runs scored
Innings Details :
Innings number : Firstinnings
```

BattingTeam :CSK
Runs scored :200

Sample Input 1

Secondinnings
MI
150

Sample Output 1

Enter the innings number
Enter the BattingTeam
Enter the runs scored
Innings Details :
Innings number : Secondinnings
BattingTeam :MI
Runs scored :150

Change Theme Language Java 7

```java
import java.util.Scanner;

class Innings {
    String number;
    String battingTeam;
    Long runs;

    Innings(String number, String battingTeam, Long runs) {
        this.number = number;
        this.battingTeam = battingTeam;
        this.runs = runs;
    }

    void displayInningsDetails() {
        System.out.println("Innings Details :");
        System.out.println("Innings number : " + number);
        System.out.println("BattingTeam :" + battingTeam);
        System.out.println("Runs scored :" + runs);
    }
}
```

Line: 42 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Class and Objects - Cricket Match Delivery Tracker

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine you're developing a Cricket Match Delivery Tracker system for a sports analytics company. This system helps track and display key details of each delivery (ball) bowled in a cricket match. For each delivery, the system records details like the over number, ball number, the number of runs scored, and the players involved (batsman, bowler, and non-striker).

This tool is useful for match commentators, statisticians, and coaches who want to log every ball and analyze the match in real-time. Your task is to develop an object-oriented program to input delivery details and display them in a user-friendly format.

Program Specifications

Class Name: Delivery

- Member Variables:
- Long Over (to store the over number).
- Long Ball (to store the ball number).
- Long Runs (to store the runs scored on that delivery).
- String Batsman (to store the name of the batsman).
- String Bowler (to store the name of the bowler).
- String NonStriker (to store the name of the non-striker).
- Method:
- void displayDeliveryDetails() (to display the details of the delivery).

Class Name: Main

- This class contains the main method to interact with the user and test the Delivery class.

Input Format

1. The user is prompted to enter the over number (Long).
2. Next, the user is asked to input the ball number (Long).
3. The user is then prompted to enter the runs scored (Long).
4. The user is asked to input the batsman's name (String).
5. The user is prompted to enter the bowler's name (String).
6. Finally, the user is asked to input the non-striker's name (String).

Constraints

NA

Output Format

1. The output should display the details of the delivery in a structured format, with each piece of information labeled (Over, Ball, Runs, Batsman, Bowler, NonStriker).
2. Each label and its corresponding value should be separated by a colon (:) and a space.
3. The output should be printed on separate lines.

Sample Input 0

```
1
1
0
Virat Kohli
```

Jasprit Bumrah
Rohit Sharma

Sample Output 0

Enter the over
Enter the ball
Enter the runs
Enter the batsman name
Enter the bowler name
Enter the nonStriker name
Delivery Details :
Over : 1
Ball : 1
Runs : 0
Batsman : Virat Kohli
Bowler : Jasprit Bumrah
NonStriker : Rohit Sharma

Sample Input 1

5
3
6
AB de Villiers
Pat Cummins
David Warner

Sample Output 1

Enter the over
Enter the ball
Enter the runs
Enter the batsman name
Enter the bowler name
Enter the nonStriker name
Delivery Details :
Over : 5
Ball : 3
Runs : 6
Batsman : AB de Villiers
Bowler : Pat Cummins
NonStriker : David Warner

Change Theme Language Java 7

```
1  import java.util.Scanner;
2
3  class Delivery {
4      private Long Over;
5      private Long Ball;
6      private Long Runs;
7      private String Batsman;
8      private String Bowler;
9      private String NonStriker;
10
11     public Delivery(Long Over, Long Ball, Long Runs, String Batsman,
12                     String Bowler, String NonStriker) {
13         this.Over = Over;
```

```
        this.Ball = Ball;
        this.Runs = Runs;
        this.Batsman = Batsman;
        this.Bowler = Bowler;
        this.NonStriker = NonStriker;
    }

```

Line: 61 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Class and Objects - Sports Event Venue Management System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine you're tasked with developing a Sports Event Venue Management System for a sports event organization. The company is responsible for organizing major sporting events like cricket, football, and other tournaments at different stadiums across the country. Each venue where the events take place needs to be stored in the system with details such as the venue's name and the city where it is located.

The system should be simple to use, allowing the event organizers to input venue information and retrieve the stored details to ensure proper event planning and logistics.

Program Specifications

1. Create a class named Venue to store details about a venue, such as the venue name and city.
2. Create a Main class to capture user input and display the venue's information in a structured format.
3. Adhere strictly to object-oriented principles and use default access for member variables.

Input Format

1. First, the user is prompted to enter the venue name (String).
2. Next, the user is asked to input the city name (String).

Constraints

NA

Output Format

- The output should display the details of the venue in a structured format, with each piece of information labeled (Venue Name, City Name).
- Each label and its corresponding value should be separated by a colon (:) and a space.
- The output should be printed on separate lines.

Sample Input 0

```
M. A. Chidambaram Stadium
Chennai
```

Sample Output 0

```
Enter the venue name
Enter the city name
Venue Details
Venue Name : M. A. Chidambaram Stadium
City Name : Chennai
M. A. Chidambaram Stadium
```

Sample Input 1

```
Wankhede Stadium
Mumbai
```

Sample Output 1

```
Enter the venue name
Enter the city name
Venue Details
Venue Name : Wankhede Stadium
City Name : Mumbai
Wankhede Stadium
```

Change Theme Language Java 7

```java
import java.util.Scanner;

class Venue {
    String name;
    String city;

    void display() {
        System.out.println("Venue Details");
        System.out.println("Venue Name : " + name);
        System.out.println("City Name : " + city);
        System.out.println(name);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Venue v = new Venue();

        System.out.println("Enter the venue name");
```

Line: 30 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Class and Objects - Player Information System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine you're building a Player Information System for a sports management company. The system is designed to store and display the details of players from different sports teams. Every player has a name, the country they represent, and a specific skill set such as a batsman, bowler, or all-rounder.

The company's goal is to maintain a record of their players and easily retrieve and display their information as needed. To accomplish this, you're tasked with developing an object-oriented program that captures the player's details and prints them in a structured format.

Program Specifications

1. Create a class named Player to store player information like name, country, and skill.
2. Create a Main class to capture user input and display the player's information.
3. Adhere strictly to object-oriented principles by encapsulating the player's data within the Player class and using another class for testing.

Input Format

1. First, the user is prompted to enter the player's name (String).
2. Next, the user is asked to input the player's country name (String).
3. Finally, the user is asked to input the player's skill (String).

Constraints

NA

Output Format

- The output should display the details of the player in a structured format, with each piece of information labeled (Player Name, Country Name, Skill).
- Each label and its corresponding value should be separated by a colon (:) and a space.
- The output should be printed on separate lines.

Sample Input 0

```
MS Dhoni
India
All Rounder
```

Sample Output 0

```
Enter the player name
Enter the country name
Enter the skill
Player Details:
Player Name : MS Dhoni
Country Name : India
Skill : All Rounder
```

Sample Input 1

```
Virat Kohli
India
Batsman
```

Sample Output 1

```
Enter the player name
Enter the country name
Enter the skill
Player Details:
Player Name : Virat Kohli
Country Name : India
Skill : Batsman
```

Change Theme Language Java 7

```
1
2  import java.util.*;
3  public class main {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          String name;
7          String country;
8          String skill;
9          name = sc.nextLine();
10         country = sc.nextLine();
11         skill = sc.nextLine();
12         System.out.println("Enter the player name");
13         System.out.println("Enter the country name");
14         System.out.println("Enter the skill");
15         System.out.println("Player Details: ");
16         System.out.println("Player Name : "+name);
17         System.out.println("Country Name : "+country);
18         System.out.println("Skill : "+skill);
19     sc.close();
20     }
```

Line: 22 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Encapsulation - Library Book Organizer

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

A library needs a system to manage its book inventory. Each book in the library has details including the title, author, ISBN number, publication year, and genre. The system should encapsulate these details within a class to restrict direct access and display the book details in a structured format using toString() and String.format().

Program Specifications

Create a class named Book:

Attributes (Private):

- String title: Stores the title of the book.
- String author: Stores the author of the book.
- String isbn: Stores the ISBN number of the book.
- int publicationYear: Stores the year the book was published.
- String genre: Stores the genre of the book.

Constructors:

- Parameterized Constructor: Initializes all attributes when a new book is created.

Methods:

- public String getTitle(): Returns the title of the book.
- public String getAuthor(): Returns the author's name.
- public String getIsbn(): Returns the ISBN of the book.
- public int getPublicationYear(): Returns the publication year.
- public String getGenre(): Returns the genre of the book.
- public String toString(): Overrides toString() to display book details using String.format() in a structured format.

Create a class named Main:

- Purpose: Captures user input to create a Book object and displays the book's details using toString().

Input Format

The program should prompt the user to enter the following details:

1.Book Title (String).

2.Author's Name (String).

3.ISBN Number (String).

4.Publication Year (int).

5.Genre (String).

Constraints

NA

Output Format

Display the book details using the toString() method, with each detail formatted on a new line.

The format should be:

Book Details:

Title: [title]

Author: [author]

ISBN: [isbn]

Publication Year: [publicationYear]

Genre: [genre]

Sample Input 0

```
Pride and Prejudice
Jane Austen
9780141199078
1813
Fiction
```

Sample Output 0

```
Enter the book title:
Enter the author's name:
Enter the ISBN number:
Enter the publication year:
Enter the genre:
Book Details:
Title: Pride and Prejudice
Author: Jane Austen
ISBN: 9780141199078
Publication Year: 1813
Genre: Fiction
```

Sample Input 1

```
Dune
Frank Herbert
9780441013593
1965
Science Fiction
```

Sample Output 1

```
Enter the book title:
Enter the author's name:
Enter the ISBN number:
Enter the publication year:
Enter the genre:
Book Details:
Title: Dune
Author: Frank Herbert
ISBN: 9780441013593
Publication Year: 1965
Genre: Science Fiction
```

Change Theme Language Java 7

```java
import java.util.Scanner;

class Book {
    private String title;
    private String author;
    private String isbn;
    private int publicationYear;
    private String genre;

    public Book(String title, String author, String isbn, int publicationYear, String genre)
        this.title = title;
        this.author = author;
        this.isbn = isbn;
        this.publicationYear = publicationYear;
        this.genre = genre;
    }

    public String getTitle() {
        return title;
    }

    public String getAuthor() {
        return author;
    }

    public String getIsbn() {
        return isbn;
    }
```

Line: 79 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Encapsulation - Cricket Innings Management

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

You are developing a cricket score management system for a tournament. In a cricket match, there are two innings where each team gets a chance to bat. The system needs to record the details of both innings (the batting team and the total runs scored) and display the results at the end of the match.

Your task is to design the solution using Object-Oriented Programming (OOP) principles to handle and display the innings details. The program will collect data for both innings, store the information using an array of objects, and display the results.

Program Specifications

Player Class:

Private member variables:

- String BattingTeam
- Long runs

Provides a 2-argument constructor for initializing these variables.

Getters and setters for the member variables using appropriate naming conventions.

Override the toString() method to format the innings details in the specified format.

InningsBO Class:

- Contains a method displayAllInningsDetails(Innings[] inningsList) which prints the details of all innings passed in the array.

Main Class:

- Handles user input to create the Innings objects and uses the InningsBO class to display the details.

Input Format

1. Enter the values for Firstinnings: Prompt for entering the first innings data.
2. Enter the BattingTeam: A string representing the batting team.
3. Enter the runs scored: A long integer representing the runs scored.
4. Enter the values for Secondinnings: Prompt for entering the second innings data.
5. Enter the BattingTeam: A string representing the batting team.
6. Enter the runs scored: A long integer representing the runs scored.

Constraints

NA

Output Format

The details of both innings formatted as:(%-20s%-20s)

Innings details

```
<BattingTeam>      <Runs>
```

Sample Input 0

```
Australia
334
India
335
```

Sample Output 0

```
Enter the values for FirstInnings
Enter the BattingTeam
Enter the runs scored
Enter the values for SecondInnings
Enter the BattingTeam
Enter the runs scored
Innings Details
Australia 334
India 335
```

Sample Input 1

```
Bangladesh
112
Afghanistan
113
```

Sample Output 1

```
Enter the values for FirstInnings
Enter the BattingTeam
Enter the runs scored
Enter the values for SecondInnings
Enter the BattingTeam
Enter the runs scored
Innings Details
Bangladesh 112
Afghanistan 113
```

Change Theme Language Java 7

```
1  import java.util.Scanner;
2  class Innings {
3      private String battingTeam;
4      private Long runs;
5      public Innings(String battingTeam, Long runs) {
6          this.battingTeam = battingTeam;
7          this.runs = runs;
8      }
9      public String getBattingTeam() {
10         return battingTeam;
11     }
```

```
12      public void setBattingTeam(String battingTeam) {
13          this.battingTeam = battingTeam;
14      }
15      public Long getRuns() {
16          return runs;
17      }
18      public void setRuns(Long runs) {
19          this.runs = runs;
20      }
21      public String toString() {
22          return String.format("%-20s%-20s", battingTeam, runs);
23      }
24  }
25  class InningsBO {
26      public void displayAllInningsDetails(Innings[] inningsList) {
27          System.out.println("Innings Details");
28          for (Innings innings : inningsList) {
```

Line: 58 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Encapsulation - Cricket Player Management

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

You are working on a sports management system for a cricket league. The system needs to handle player information such as name, country, and skill. Your task is to develop a solution using Object-Oriented Programming (OOP) concepts to store and display details of multiple players. The program must prompt for input of player details and display the details in a formatted table.

The system will use:

1. A Player class to hold individual player information.
2. A PlayerBO class to handle operations related to players, such as displaying player details.
3. An array of Player objects to store information for multiple players, ensuring a structured and organized representation.

Program Specifications

Player Class:

Private member variables:

- String Name
- String Country
- String Skill

Provides a 3-argument constructor for initializing these variables.

Getters and setters for the member variables using appropriate naming conventions.

Override the toString() method to format the player details as per the requirement.

PlayerBO Class:

- Contains a method displayPlayerDetails(Player[] playerList) which prints the details of all players passed in the array.

Main Class:

- Handles user input to create multiple Player objects and uses the PlayerBO class to display player details in the specified format.

Input Format

1. Enter the number of players: An integer representing the number of players.
2. Enter the player name: A string representing the player's name.
3. Enter the country name: A string representing the player's country.
4. Enter the skill: A string representing the player's skill (e.g., "Batsman", "Bowler", "All Rounder").

Constraints

NA

Output Format

The details of each player formatted in the specified format:(%-15s%-15s%-15s)

Player Details

```
<Player Name>    <Country>    <Skill>
```

Sample Input 0

```
2
Steve Smith
Australia
Batsman
Kagiso Rabada
South Africa
Bowler
```

Sample Output 0

```
Enter the number of players
Enter the player name
Enter the country name
Enter the skill
Enter the player name
Enter the country name
Enter the skill
Player Details
Steve Smith    Australia      Batsman
Kagiso Rabada  South Africa   Bowler
```

Sample Input 1

```
3
Ben Stokes
England
All Rounder
Moeen Ali
England
All Rounder
Chris Woakes
England
All Rounder
```

Sample Output 1

```
Enter the number of players
Enter the player name
Enter the country name
Enter the skill
Enter the player name
Enter the country name
Enter the skill
Enter the player name
Enter the country name
Enter the skill
Player Details
Ben Stokes     England        All Rounder
Moeen Ali      England        All Rounder
Chris Woakes   England        All Rounder
```

Change Theme Language Java 7

```java
import java.util.Scanner;

class Player {
    private String name;
    private String country;
    private String skill;

    public Player(String name, String country, String skill) {
        this.name = name;
        this.country = country;
        this.skill = skill;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCountry() {
        return country;
    }

    public void setCountry(String country) {
        this.country = country;
```

Line: 83 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Imagine you are working on an application that tracks the details of every ball delivered in a cricket match. For each delivery, you need to store the over number, ball number, batsman, bowler, and non-striker. You are required to implement this using object-oriented programming principles and ensure that the delivery details can be displayed in a formatted way. The program must have a Delivery class to hold the delivery details and a DeliveryBO class to manage and display the list of deliveries.

Program Specifications

Delivery Class:

Private member variables:

- Long Over
- Long ball
- String Batsman
- String bowler
- String NonStriker

Provides a 5-argument constructor to initialize these variables.

Getters and setters for the member variables using appropriate naming conventions.

Override the toString() method to format the delivery details as specified.

DeliveryBO Class:

- Contains a method displayAllDeliveryDetails(Delivery[] deliveryList) that prints the details of all deliveries passed in the array.

Main Class:

- Handles input from the user to create multiple Delivery objects and uses the DeliveryBO class to display the details of each delivery in the specified format.

Input Format

1. Enter the number of deliveries: An integer representing the number of deliveries.
2. Enter the over: A long integer representing the over number.
3. Enter the ball: A long integer representing the ball number.
4. Enter the batsman: A string representing the batsman's name.
5. Enter the bowler: A string representing the bowler's name.

Change Theme Language Java 7

```
1  import java.util.Scanner;
2  class Delivery {
3      private Long over;
4      private Long ball;
5      private String batsman;
6      private String bowler;
7      private String nonStriker;
8      public Delivery(Long over, Long ball, String
9                      String bowler, String nonStr
10         this.over = over;
11         this.ball = ball;
12         this.batsman = batsman;
13         this.bowler = bowler;
14         this.nonStriker = nonStriker;
15     }
16     public Long getOver() {
17         return over;
18     }
19     public void setOver(Long over) {
20         this.over = over;
21     }
22     public Long getBall() {
23         return ball;
24     }
25     public void setBall(Long ball) {
```

Line: 90 Col: 1

Upload Code as File Run Code Submit Code

Test against custom input

Encapsulation - Cricket Venue Management

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine you are working on an application that manages cricket tournament venues. For each venue, you need to store its name and the city where it is located. You are required to store this information in an object-oriented way, ensuring that venue details are displayed in a formatted output using a specific format. The program needs to have a Venue class for storing venue data and a VenueBO class for displaying the venue details.

Program Specifications

Venue Class:

- Private member variables: Name (String) and city (String).
- Provides a default constructor and a 2-argument constructor to initialize the variables.
- Getters and setters for the member variables using appropriate naming conventions.
- Override the toString() method to format the output in the specified format: ", ".

VenueBO Class:

- Contains a method displayVenueDetails(Venue venue) that prints the venue details using the toString() method of the Venue class.

Main Class:

- Handles input from the user to create a Venue object and uses the VenueBO class to display the venue's details in the required format.

Input Format

1. Enter the venue name: A string representing the venue's name.
2. Enter the city name: A string representing the city's name.

Constraints

NA

Output Format

- Venue Details: This text is printed inside the VenueBO class's displayVenueDetails method.
- The venue name and city are printed in the format ", ".

Sample Input 0

```
Eden Gardens
Kolkata
```

Sample Output 0

```
Enter the venue name
Enter the city name
Venue Details
Eden Gardens,Kolkata
```

Sample Input 1

```
Wankhede Stadium
Mumbai
```

Sample Output 1

```
Enter the venue name
Enter the city name
Venue Details
Wankhede Stadium,Mumbai
```

Change Theme Language Java 7

```
1   import java.util.Scanner;
2   class Venue {
3       private String name;
4       private String city;
5       public Venue() {
6       }
7       public Venue(String name, String city) {
8           this.name = name;
9           this.city = city;
10      }
11      public String getName() {
12          return name;
13      }
14      public String getCity() {
15          return city;
16      }
17      public void setName(String name) {
18          this.name = name;
19      }
20      public void setCity(String city) {
21          this.city = city;
22      }
23      public String toString() {
24          return name + "," + city;
25      }
26  }
27  class VenueBO {
```

Line: 46 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Encapsulation - Cricket Player Profile Management

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

You are building a system for a cricket league where you need to store information about players. Each player has a name, country, and skill (e.g., Batsman, Bowler, All-rounder). You need to store this information and display it in a formatted way using object-oriented principles. The task requires overriding the toString() method in the Player class to format the player's details and then display them using a separate PlayerBO class.

Program Specifications

Player Class:

- Private member variables: Name, Country, and Skill.
- Provides a default constructor and a 3-argument constructor.
- Getters and setters for the member variables using appropriate naming conventions.
- Override the toString() method to format the output in the specified format using String.format("%-15s %-15s %-15s", name, country, skill).

PlayerBO Class:

- Contains a method displayPlayerDetails(Player player) that prints the player's details using the toString() method of the Player class.

Main Class:

- Handles input from the user to create a Player object and uses the PlayerBO class to display the player's details in the required format.

Input Format

1. Enter the player name: A string representing the player's name.
2. Enter the country name: A string representing the player's country.
3. Enter the skill: A string representing the player's skill.

Constraints

NA

Output Format

- Player Details: This text is printed inside the PlayerBO class's displayPlayerDetails method.
- Player name, country, and skill are printed in a formatted way with fixed spacing.

Sample Input 0

```
Virat Kohli
India
Batsman
```

Sample Output 0

```
Enter the player name
Enter the country
Enter the skill
```

```
Player Details
Virat Kohli     India         Batsman
```

Sample Input 1

```
Steve Smith
Australia
Batsman
```

Sample Output 1

```
Enter the player name
Enter the country
Enter the skill
Player Details
Steve Smith     Australia       Batsman
```

Change Theme Language Java 7

```
1  import java.util.Scanner;
2  class Player {
3      private String name;
4      private String country;
5      private String skill;
6      public Player() {
7      }
8      public Player(String name, String country, String skill) {
9          this.name = name;
10         this.country = country;
11         this.skill = skill;
12     }
13     public String getName() {
14         return name;
15     }
16     public void setName(String name) {
17         this.name = name;
18     }
19     public String getCountry() {
20         return country;
21     }
22     public void setCountry(String country) {
23         this.country = country;
24     }
25     public String getSkill() {
26         return skill;
27     }
28     public void setSkill(String skill) {
```

Line: 56 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Blog | Scoring | Environment | FAQ | About Us | Helpdesk | Careers | Terms Of Service | Privacy Policy

Encapsulation - Cricket Venue Manager

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine you are working on a cricket tournament management application, where you need to store information about multiple venues where matches will be played. Each venue has a name and the city where it is located. You are required to design a Venue class to store this information, using appropriate encapsulation with getters and setters. The user will input venue details in a comma-separated format, and your application will split the input to extract the venue name and city.

Program Specifications

Venue Class

- Private member variables: name and city.
- A 2-argument constructor to initialize these variables.
- A default constructor.
- Getters and setters following the given naming convention (getName, setName, getCity, setCity).

Main Class

- Accepts the number of venues as input.
- Uses a list to store multiple Venue objects.
- Splits the input string to extract the venue name and city, and prints the details in the specified format.

Input Format

1. Enter the number of venues: An integer value representing the number of venues.
2. For each venue, enter the details in the following format:
   - Enter the details of venue X: A string containing the venue name and city separated by a comma (,).

Constraints

NA

Output Format

1. Venue Details: A heading to indicate the start of the output.
2. For each venue:
   - Venue Name: The name of the venue, exactly as input, except if there are spaces after the comma (like "M. Chinnasamy Stadium"), the first comma should not be printed directly next to the venue's name (e.g., "M, Chinnasamy Stadium" instead of "M. Chinnasamy Stadium").
   - City Name: The city of the venue.

Sample Input 0

```
2
Stadium A,New York
Hall B,Los Angeles
```

Sample Output 0

```
Enter the number of venues
Enter the details of venue 1
Enter the details of venue 2
Venue Details
Venue Name : Stadium A
City Name : New York
Venue Name : Hall B
City Name : Los Angeles
```

Sample Input 1

```
1
Convention Center,Chicago
```

Sample Output 1

```
Enter the number of venues
Enter the details of venue 1
Venue Details
Venue Name : Convention Center
City Name : Chicago
```

Change Theme Language Java 7

```java
import java.util.Scanner;
import java.util.ArrayList;
class Venue {
    private String name;
    private String city;
    Venue() {
    }
    Venue(String name, String city) {
        this.name = name;
        this.city = city;
    }
    public String getName() {
        return name;
    }
    public String getCity() {
        return city;
    }
    public void setName(String name) {
        this.name = name;
    }
    public void setCity(String city) {
        this.city = city;
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number of venues");
```

Line: 48 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Encapsulation - Cricket Player Statistics Management

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

Imagine a simple cricket system where we need to store and manage a cricketer's name, total runs, and total matches played. To ensure data security, we use encapsulation to keep these details private and allow access through public methods. The player's data should be updated and retrieved only through these methods.

Program Specifications

Create a class CricketPlayer with private fields:

- name (String)
- totalRuns (int)
- matchesPlayed (int)

Provide public getter and setter methods to:

- Get the player's name, total runs, and matches played.
- Set the player's name, total runs, and matches played.

Input Format

1. The first input line will ask for the player's name as a string.
2. The second input line will ask for the player's total runs as an integer.
3. The third input line will ask for the number of matches played by the player as an integer.

Constraints

NA

Output Format

The program will output the cricket player details in the following format:

- The player's name.
- The total runs scored by the player.
- The number of matches played by the player.

Sample Input 0

```
Virat Kohli
12000
250
```

Sample Output 0

```
Enter Player Name:
Enter Total Runs:
Enter Matches Played:
```

```
Cricket Player Details:
Name: Virat Kohli
Total Runs: 12000
Matches Played: 250
```

Sample Input 1

```
Joe Root
8500
-50
```

Sample Output 1

```
Enter Player Name:
Enter Total Runs:
Enter Matches Played:

Cricket Player Details:
Name: Joe Root
Total Runs: 8500
Matches Played: -50
```

Change Theme Language Java 7

```java
import java.util.Scanner;
class CricketPlayer {
    private String name;
    private int totalRuns;
    private int matchesPlayed;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getTotalRuns() {
        return totalRuns;
    }
    public void setTotalRuns(int totalRuns) {
        this.totalRuns = totalRuns;
    }
    public int getMatchesPlayed() {
        return matchesPlayed;
    }
    public void setMatchesPlayed(int matchesPlayed) {
        this.matchesPlayed = matchesPlayed;
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter Player Name:");
```

Line: 45 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Encapsulation - Cricket Player Profile and Performance Tracker

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

A cricket team management system needs to store and manage player information. The details include each player's name, age, role (e.g., Batsman, Bowler, All-Rounder), total matches played, and total runs scored. The management team wants a system where they can add player statistics, view player details, and securely manage sensitive player data by preventing direct access to the player's attributes.

Program Specifications

Create a class named CricketPlayer:

Attributes (Private):

- String name: Stores the player's name.
- int age: Stores the player's age.
- String role: Stores the player's role (e.g., Batsman, Bowler, All-Rounder).
- int matchesPlayed: Stores the total number of matches played by the player.
- int runsScored: Stores the total runs scored by the player.

Constructors: - Parameterized Constructor: Initializes all attributes when a new player is created.

Methods:

- public String getName(): Returns the player's name.
- public int getAge(): Returns the player's age.
- public String getRole(): Returns the player's role.
- public int getMatchesPlayed(): Returns the number of matches played.
- public int getRunsScored(): Returns the runs scored by the player.
- public void addRuns(int runs): Adds runs to the player's total (if runs is positive).
- public void addMatches(int matches): Increases matches played (if matches is positive).
- public void displayPlayerDetails(): Displays the player's information in a structured format.

Create a class named Main:

- Purpose: Captures user input to create a CricketPlayer object, allows the addition of runs and matches, and displays the player's details.

Input Format

The program should prompt the user to enter the following details:

1.Player's Name (String).

2.Player's Age (int).

3.Player's Role (String).

4.Matches Played (int).

5.Runs Scored (int).

Constraints

NA

Output Format

Displays the player details as well as the updated statistics after adding runs or matches.

The format should be:

Player Details:

Name: [name]

Age: [age]

Role: [role]

Matches Played: [matchesPlayed]

Runs Scored: [runsScored]

If an invalid action is attempted (e.g., negative runs/matches), display: Invalid Input

Sample Input 0

```
Rohit Sharma
36
Batsman
250
9500
200
2
```

Sample Output 0

```
Enter the player's name:
Enter the player's age:
Enter the player's role:
Enter matches played:
Enter runs scored:
Player Details:
Name: Rohit Sharma
Age: 36
Role: Batsman
Matches Played: 250
Runs Scored: 9500
Enter runs to add:
Runs added successfully. Updated Runs Scored: 9700
Enter matches to add:
Matches added successfully. Updated Matches Played: 252
Player Details:
Name: Rohit Sharma
Age: 36
Role: Batsman
Matches Played: 252
Runs Scored: 9700
```

Sample Input 1

```
Rishabh Pant
25
Wicketkeeper
50
1500
```

-100
0

Sample Output 1

```
Enter the player's name:
Enter the player's age:
Enter the player's role:
Enter matches played:
Enter runs scored:
Player Details:
Name: Rishabh Pant
Age: 25
Role: Wicketkeeper
Matches Played: 50
Runs Scored: 1500
Enter runs to add:
Invalid Input
Enter matches to add:
Invalid Input
Player Details:
Name: Rishabh Pant
Age: 25
Role: Wicketkeeper
Matches Played: 50
Runs Scored: 1500
```

Change Theme Language Java 7

```java
import java.util.Scanner;
class CricketPlayer {
    private String name;
    private int age;
    private String role;
    private int matchesPlayed;
    private int runsScored;
    CricketPlayer(String name, int age, String role, int matchesPlayed, int runsScored) {
        this.name = name;
        this.age = age;
        this.role = role;
        this.matchesPlayed = matchesPlayed;
        this.runsScored = runsScored;
    }
    public String getName() {
        return name;
    }
    public int getAge() {
        return age;
    }
    public String getRole() {
        return role;
    }
    public int getMatchesPlayed() {
        return matchesPlayed;
    }
```

```
27      public int getRunsScored() {
28          return runsScored;
```

Line: 86 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Blog | Scoring | Environment | FAQ | About Us | Helpdesk | Careers | Terms Of Service | Privacy Policy

Encapsulation - Bank Account Management System

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

A bank wants a system to manage customer bank accounts. To maintain security, account details such as account number, account holder's name, account balance, and account type should not be accessible or modifiable directly. Only authorized methods should be able to view or update the balance. This is achieved by encapsulating the data within a BankAccount class and providing secure access through getter and setter methods.

Program Specifications

Create a class named BankAccount: Attributes (Private):

- String accountNumber: Stores the bank account number.
- String accountHolderName: Stores the account holder's name.
- double balance: Stores the account balance.
- String accountType: Stores the type of account (e.g., "Savings" or "Checking").

Constructors: - Parameterized Constructor: Initializes all attributes when a new account is created.

Methods:

- public String getAccountNumber(): Returns the account number.
- public String getAccountHolderName(): Returns the account holder's name.
- public String getAccountType(): Returns the account type.
- public double getBalance(): Returns the current balance.
- public void deposit(double amount): Allows adding money to the balance (if amount is positive).
- public void withdraw(double amount): Allows withdrawing money from the balance (if amount is positive and does not exceed the balance).
- public void displayAccountDetails(): Displays the account's information in a structured format.

Create a class named Main:

- Purpose: Captures user input to create a BankAccount object, allows deposits, withdrawals, and displays the account's details.

Input Format

The program should prompt the user to enter the following details:

1.Account Number (String).

2.Account Holder's Name (String).

3.Initial Balance (double).

4.Account Type (String).

Constraints

NA

Output Format

Displays the account details as well as the balance after any deposit or withdrawal. The format should be:

Account Details:

Account Number: [accountNumber]

Account Holder Name: [accountHolderName]

Account Type: [accountType]

Balance: $[balance]

If an invalid transaction is attempted (e.g., negative deposit/withdrawal or withdrawal exceeding balance), display:

Invalid Transaction

Sample Input 0

```
1234567890
John Doe
500.0
Savings
200.0
150.0
```

Sample Output 0

```
Enter the account number:
Enter the account holder's name:
Enter the initial balance:
Enter the account type:
Account Details:
Account Number: 1234567890
Account Holder Name: John Doe
Account Type: Savings
Balance: $500.0
Enter deposit amount:
Deposit Successful. Updated Balance: $700.0
Enter withdrawal amount:
Withdrawal Successful. Updated Balance: $550.0
Account Details:
Account Number: 1234567890
Account Holder Name: John Doe
Account Type: Savings
Balance: $550.0
```

Sample Input 1

```
9876543210
Jane Smith
1000.0
Checking
 -50.0
200.0
```

Sample Output 1

```
Enter the account number:
Enter the account holder's name:
Enter the initial balance:
Enter the account type:
Account Details:
Account Number: 9876543210
Account Holder Name: Jane Smith
```

```
Account Type: Checking
Balance: $1000.0
Enter deposit amount:
Invalid Transaction
Enter withdrawal amount:
Withdrawal Successful. Updated Balance: $800.0
Account Details:
Account Number: 9876543210
Account Holder Name: Jane Smith
Account Type: Checking
Balance: $800.0
```

Change Theme Language Java 7

```
import java.util.Scanner;
class BankAccount {
    private String accountNumber;
    private String accountHolderName;
    private double balance;
    private String accountType;
    BankAccount(String accountNumber, String accountHolderName,
                double balance, String accountType) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
        this.accountType = accountType;
    }
    public String getAccountNumber() {
        return accountNumber;
    }
    public String getAccountHolderName() {
        return accountHolderName;
    }
    public double getBalance() {
        return balance;
    }
    public String getAccountType() {
        return accountType;
    }
    public void deposit(double amount) {
        if (amount > 0) {
            balance = balance + amount;
```

Line: 78 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Inheritance - Room Surface Area Calculator for Paint Estimation

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

An interior decorator wants to estimate the amount of paint required to cover the walls and ceiling in various rooms. Each room may have a different shape (rectangular, square, or circular). The decorator inputs the dimensions of each room to calculate its surface area. The area calculator application will prompt the user to select the shape of the room and input the relevant dimensions, then output the surface area.

Program Specifications

Base Class: Shape

Attributes:

- protected String shapeName: The name of the shape.

Methods:

- public Shape(String shapeName): Constructor to initialize the shape name.
- public Double calculateArea(): Returns 0 by default.

Derived Class: Square

Attributes:

- private Integer side: Length of one side of the square.

Methods:

- public Square(int side): Constructor to initialize the side and set the shape name.
- public Double calculateArea(): Calculates and returns the area of the square.

Derived Class: Rectangle

Attributes:

- private Integer length: Length of the rectangle.
- private Integer breadth: Breadth of the rectangle.

Methods:

- public Rectangle(int length, int breadth): Constructor to initialize the dimensions and set the shape name.
- public Double calculateArea(): Calculates and returns the area of the rectangle.

Derived Class: Circle

Attributes:

- private Integer radius: Radius of the circle.

Methods:

- public Circle(int radius): Constructor to initialize the radius and set the shape name.
- public Double calculateArea(): Calculates and returns the area of the circle.

Input Format

1. User selects the shape by entering the corresponding number.
2. For rectangles, the user inputs length and breadth.
3. For squares, the user inputs the side length.
4. For circles, the user inputs the radius.

Constraints

NA

Output Format

Displays the calculated area formatted to two decimal places.

Sample Input 0

```
1
150
75
```

Sample Output 0

```
1. Rectangle
2. Square
3. Circle
Area Calculator --- Choose your shape: Enter length and breadth: Area of Rectangle is: 11250.00
```

Sample Input 1

```
2
10
```

Sample Output 1

```
1. Rectangle
2. Square
3. Circle
Area Calculator --- Choose your shape: Enter side: Area of Square is: 100.00
```

Sample Input 2

```
3
12
```

Sample Output 2

```
1. Rectangle
2. Square
3. Circle
Area Calculator --- Choose your shape: Enter Radius: Area of Circle is: 452.39
```

Change Theme Language Java 7

```
1  import java.io.*;
2  import java.util.*;
3  import java.text.*;
4  import java.math.*;
5  import java.util.regex.*;
6
7  class Shape{
8      protected String name;
9
10     Shape(String name){
```

```
11          this.name = name;
12      }
13
14      public double Area(){
15          return 0.0;
16      }
17  }
18
19  class Square extends Shape{
20      private int side;
21
22      Square(int side){
23          super("Square");
24          this.side = side;
25      }
26
27      public double Area(){
28          return (double) side*side;
29      }
30  }
31
32  class Rectangle extends Shape{
```

Line: 107 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Inheritance - Cricket Squad Information System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In this Cricket Squad Information System, we aim to manage information about different players in a cricket team. The system allows users to enter details about various types of players, specifically Batsmen and Bowlers. The design uses hierarchical inheritance, where we have a base class called Player that captures common attributes of all players, and two derived classes, Batsman and Bowler, that inherit from Player and add their specific properties.

Program Specifications

Base Class: Player

Attributes:

- name: String (the name of the player)
- age: int (the age of the player)
- team: String (the team the player belongs to)

Methods:

- displayDetails(): Displays the player's name, age, and team.

Derived Class: Batsman

Additional Attributes:

- runsScored: int (total runs scored by the batsman)

Methods:

- displayBatsmanDetails(): Displays batsman-specific details along with the inherited player details.

Derived Class: Bowler

Additional Attributes:

- wicketsTaken: int (total wickets taken by the bowler)

Methods:

- displayBowlerDetails(): Displays bowler-specific details along with the inherited player details.

Input Format

1. For Batsman: The user will be prompted to enter the name, age, team, and runs scored.
2. For Bowler: The user will be prompted to enter the name, age, team, and wickets taken.

Constraints

NA

Output Format

The program will display the details of the entered batsman and bowler in a structured format, including both common and specific attributes.

Sample Input 0

```
Virat Kohli
35
India
12345
Jasprit Bumrah
30
```

```
India
120
```

Sample Output 0

```
Enter details for Batsman:
Name: Age: Team: Runs Scored:
Enter details for Bowler:
Name: Age: Team: Wickets Taken:
Batsman Details:
Name: Virat Kohli
Age: 35
Team: India
Runs Scored: 12345

Bowler Details:
Name: Jasprit Bumrah
Age: 30
Team: India
Wickets Taken: 120
```

Sample Input 1

```
Joe Root
33
England
10567
James Anderson
42
England
690
```

Sample Output 1

```
Enter details for Batsman:
Name: Age: Team: Runs Scored:
Enter details for Bowler:
Name: Age: Team: Wickets Taken:
Batsman Details:
Name: Joe Root
Age: 33
Team: England
Runs Scored: 10567

Bowler Details:
Name: James Anderson
Age: 42
Team: England
Wickets Taken: 690
```

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

class Player{
```

```
    String name,team;
    int age;
}

class Batsman extends Player{
    int runs;

    void display1(){
        System.out.println("Batsman Details:");
        System.out.println("Name: "+name);
        System.out.println("Age: "+age);
        System.out.println("Team: "+team);
        System.out.println("Runs Scored: "+runs);
    }
}

class Bowler extends Player{
    int wickets;

    void display2(){
        System.out.println("Bowler Details:");
        System.out.println("Name: "+name);
        System.out.println("Age: "+age);
        System.out.println("Team: "+team);
```

Line: 67 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Inheritance - Automobile Inventory Management System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In this Automobile Inventory Management System, we aim to manage different types of vehicles in an inventory. The system allows users to enter details about various vehicles, specifically cars and trucks, and then displays this information in a structured format. The design leverages hierarchical inheritance, where we have a base class called Vehicle that captures common attributes of vehicles, and two derived classes, Car and Truck, that inherit from Vehicle and add their specific properties.

Program Specifications

Base Class: Vehicle

Attributes:

- make: String (the manufacturer of the vehicle)
- model: String (the model name of the vehicle)
- year: int (the year the vehicle was manufactured)

Methods:

- displayDetails(): Displays the vehicle's make, model, and year.

Derived Class: Car

Additional Attributes:

- numberOfDoors: int (the number of doors in the car)

Methods:

- displayCarDetails(): Displays the car's specific details along with the inherited vehicle details.

Derived Class: Truck

Additional Attributes:

- loadCapacity: double (the maximum load capacity of the truck in tons)

Methods:

- displayTruckDetails(): Displays the truck's specific details along with the inherited vehicle details.

Input Format

1. For Car: The user will be prompted to enter the make, model, year, and number of doors.
2. For Truck: The user will be prompted to enter the make, model, year, and load capacity.

Constraints

NA

Output Format

The program will display the details of the entered car and truck in a clear format, including both common and specific attributes.

Sample Input 0

```
Honda
Accord
2021
4
Chevrolet
Silverado
```

2022
1.0

Sample Output 0

```
Enter details for Car:
Make: Model: Year: Number of Doors:
Enter details for Truck:
Make: Model: Year: Load Capacity (in tons): Car Details:
Make: Honda
Model: Accord
Year: 2021
Number of Doors: 4

Truck Details:
Make: Chevrolet
Model: Silverado
Year: 2022
Load Capacity: 1.0 tons
```

Sample Input 1

```
Hyundai
i20
2020
5
Volvo
FH16
2019
20.0
```

Sample Output 1

```
Enter details for Car:
Make: Model: Year: Number of Doors:
Enter details for Truck:
Make: Model: Year: Load Capacity (in tons): Car Details:
Make: Hyundai
Model: i20
Year: 2020
Number of Doors: 5

Truck Details:
Make: Volvo
Model: FH16
Year: 2019
Load Capacity: 20.0 tons
```

Change Theme Language Java 7

```
1  import java.io.*;
2  import java.util.*;
3  import java.text.*;
4  import java.math.*;
5  import java.util.regex.*;
6
7  class Vehicle{
8      String make;
9      String model;
```

```
10      int year;
11
12      void displayV(){
13          System.out.println("Make: "+make);
14          System.out.println("Model: "+model);
15          System.out.println("Year: "+year);
16      }
17  }
18
19  class Car extends Vehicle{
20      int doors;
21
22      void displayC(){
23          System.out.println("Number of Doors: "+doors);
24      }
25  }
26
27  class Truck extends Vehicle{
28      double load;
29
30      void displayT(){
31          System.out.println("Load Capacity: "+load+" tons");
32      }
```

Line: 69 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Inheritance - Bowler Profile Management System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

The cricket association wants to classify players based on their general profile, then further categorize them as either bowlers or specific types of fast bowlers. This classification helps manage player details at different levels of expertise.

This system will use multilevel inheritance:

- The Player class will store basic details about any player.
- The Bowler class will extend Player to add attributes specific to bowlers.
- The FastBowler class will extend Bowler to add details unique to fast bowlers.

Program Specifications

Create a Player class with the following attributes:

- String name: Name of the player.
- String team: Team of the player.
- int age: Age of the player.
- Include a method displayPlayerInfo() to print general player information.

Create a Bowler class extending Player with additional attributes:

- int wicketsTaken: Number of wickets taken by the bowler.
- double bowlingAverage: The bowling average of the bowler.
- Include a method displayBowlerInfo() to print bowler-specific details.

Create a FastBowler class extending Bowler with an additional attribute:

- int averageSpeed: Average bowling speed (km/h).
- Include a method displayFastBowlerInfo() to print fast bowler-specific details.

Create a Main class to capture user input and display player details at each level of inheritance.

Input Format

1. First, the user is prompted to enter the player's name.
2. Next, the player's team name.
3. Player's age.
4. Number of wickets taken.
5. Bowling average.
6. Average speed of bowling.

Constraints

NA

Output Format

The output should display the player's general details first, followed by bowler-specific details, and finally the fast bowler-specific details.

Sample Input 0

```
Mitchell Starc
Australia
33
280
```

```
25.3
145
```

Sample Output 0

```
Enter the player's name:
Enter the player's team:
Enter the player's age:
Enter the number of wickets taken:
Enter the bowling average:
Enter the average speed:
Player Details:
Name: Mitchell Starc
Team: Australia
Age: 33
Bowler Details:
Wickets Taken: 280
Bowling Average: 25.3
Fast Bowler Details:
Average Speed: 145 km/h
```

Sample Input 1

```
Kagiso Rabada
South Africa
28
210
23.1
142
```

Sample Output 1

```
Enter the player's name:
Enter the player's team:
Enter the player's age:
Enter the number of wickets taken:
Enter the bowling average:
Enter the average speed:
Player Details:
Name: Kagiso Rabada
Team: South Africa
Age: 28
Bowler Details:
Wickets Taken: 210
Bowling Average: 23.1
Fast Bowler Details:
Average Speed: 142 km/h
```

Change Theme Language Java 7

```
1  import java.io.*;
2  import java.util.*;
3  import java.text.*;
4  import java.math.*;
5  import java.util.regex.*;
6
7  class Player{
8      String name;
9      String team;
```

```
10      int age;
11
12      void displayP(){
13          System.out.println("Player Details:");
14          System.out.println("Name: "+name);
15          System.out.println("Team: "+team);
16          System.out.println("Age: "+age);
17      }
18  }
19
20  class Bowler extends Player{
21      int wickets;
22      double avg;
23
24      void displayB(){
25          System.out.println("Bowler Details:");
26          System.out.println("Wickets Taken: "+wickets);
27          System.out.println("Bowling Average: "+avg);
28      }
29  }
30
31  class FastBowler extends Bowler{
```

Line: 70 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Inheritance - Vehicle Management System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

A car rental company needs a system to organize its fleet of vehicles by tracking general vehicle details, car-specific information, and features for electric cars. This system allows rental agents to view detailed information on each vehicle type and assists in customer recommendations based on the vehicle type.

Using multilevel inheritance, the system has the following levels:

1. Vehicle Class for storing basic vehicle information.
2. Car Class extends Vehicle to include car-specific details.
3. ElectricCar Class extends Car to add details specific to electric cars.

Program Specifications

Create a Vehicle class with the following attributes:

- String make: The make of the vehicle (e.g., Toyota).
- String model: The model of the vehicle (e.g., Camry).
- int year: The manufacturing year of the vehicle.
- Method displayVehicleInfo(): Displays general vehicle information.

Create a Car class extending Vehicle with additional attributes:

- int numberOfDoors: Number of doors on the car.
- String fuelType: Type of fuel (e.g., gasoline, diesel).
- Method displayCarInfo(): Displays car-specific information.

Create an ElectricCar class extending Car with additional attributes:

- int batteryCapacity: Battery capacity in kWh.
- int rangePerCharge: Range per charge in kilometers.
- Method displayElectricCarInfo(): Displays electric car-specific information.

Create a Main class to capture user input and display the vehicle details across inheritance levels.

Input Format

1. First, the user is prompted to enter the make of the vehicle.
2. The model of the vehicle.
3. The manufacturing year.
4. The number of doors.
5. The fuel type (if the vehicle is an electric car, this should be "electric").
6. The battery capacity in kWh.
7. The range per charge in kilometers.

Constraints

NA

Output Format

The output should display:

- The general vehicle details.
- Car-specific details.
- Electric car-specific details (if applicable).

Sample Input 0

```
Toyota
Corolla
2020
4
gasoline
```

Sample Output 0

```
Enter the make of the vehicle:
Enter the model of the vehicle:
Enter the manufacturing year:
Enter the number of doors:
Enter the fuel type:
Vehicle Details:
Make: Toyota
Model: Corolla
Year: 2020
Car Details:
Number of Doors: 4
Fuel Type: gasoline
```

Sample Input 1

```
Nissan
Leaf
2019
4
electric
40
240
```

Sample Output 1

```
Enter the make of the vehicle:
Enter the model of the vehicle:
Enter the manufacturing year:
Enter the number of doors:
Enter the fuel type:
Enter the battery capacity in kWh:
Enter the range per charge in km:
Vehicle Details:
Make: Nissan
Model: Leaf
Year: 2019
Car Details:
Number of Doors: 4
Fuel Type: electric
Electric Car Details:
Battery Capacity: 40 kWh
Range Per Charge: 240 km
```

Change Theme Language Java 7

```
1  import java.io.*;
2  import java.util.*;
3  import java.text.*;
```

```
4  import java.math.*;
5  import java.util.regex.*;
6
7  class Vehicle{
8      String make;
9      String model;
10     int year;
11
12     void displayV(){
13         System.out.println("Vehicle Details:");
14         System.out.println("Make: "+make);
15         System.out.println("Model: "+model);
16         System.out.println("Year: "+year);
17     }
18 }
19
20 class Car extends Vehicle{
21     int doors;
22     String fuel;
23
24     void displayC(){
25         System.out.println("Car Details:");
26         System.out.println("Number of Doors: "+doors);
27         System.out.println("Fuel Type: "+fuel);
28     }
29 }
30
31 class ElectricCar extends Car{
32     int battery;
```

Line: 99 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Inheritance - University Student Information System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

The university wants to maintain basic details of students along with specific information about graduate students. The base class, Student, holds general information about all students, while the derived class GraduateStudent adds specific information relevant to graduate students.

Program Specifications

Create a class named Student to store general student information:

String name: Name of the student.

- int id: Student ID number.
- String department: Department of the student.
- Include a method named displayStudentInfo() to print the student's general information.

Create a class named GraduateStudent that extends Student with an additional attribute:

- String thesisTopic: The topic of the thesis for graduate students.
- Include a method named displayThesisTopic() to print the graduate student's thesis topic.

Create a Main class to capture user input for a graduate student, including their thesis topic, and display their details.

Input Format

1. First, the user is prompted to enter the student's name (String).
2. Next, the user is asked to enter the student ID (Integer).
3. The user is prompted to enter the student's department (String).
4. Finally, the user is asked to enter the thesis topic of the graduate student (String).

Constraints

NA

Output Format

- The output should display the student's general details first, followed by the thesis topic.
- Each label and its corresponding value should be separated by a colon (:) and a space.
- The output should print on separate lines.

Sample Input 0

```
John Doe
10001
Biology
Genetics and Evolution
```

Sample Output 0

```
Enter the student's name:
Enter the student ID:
Enter the student's department:
Enter the thesis topic:
Student Details:
Name: John Doe
ID: 10001
```

Department: Biology
Thesis Topic: Genetics and Evolution

Sample Input 1

```
Mary Smith
20002
Mathematics
Algebraic Topology Applications
```

Sample Output 1

```
Enter the student's name:
Enter the student ID:
Enter the student's department:
Enter the thesis topic:
Student Details:
Name: Mary Smith
ID: 20002
Department: Mathematics
Thesis Topic: Algebraic Topology Applications
```

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

class Student{
    String name;
    int id;
    String dept;

    void displayS(){
        System.out.println("Student Details:");
        System.out.println("Name: "+name);
        System.out.println("ID: "+id);
        System.out.println("Department: "+dept);
    }
}

class GraduateStudent extends Student{
    String thesisTopic;

    void displayGS(){
        System.out.println("Thesis Topic: "+thesisTopic);
    }
}

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
```

Line: 52 Col: 1

Run Code Submit Code

Upload Code as File

Test against custom input

Inheritance - Cricket Player Statistics System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

The cricket association wants to keep track of both general player details and specific details about bowlers. The base class, Player, will store general information about all cricket players, while the derived class, Bowler, will add details specific to bowlers.

Program Specifications

Create a class named Player to store general cricket player information:

- String name: Name of the player.
- String team: Team of the player.
- int age: Age of the player.
- Include a method named displayPlayerInfo() to print the player's general information.

Create a class named Bowler that extends Player with additional attributes:

- int wicketsTaken: Number of wickets taken by the bowler.
- double bowlingAverage: The bowling average of the bowler.
- Include a method named displayBowlingStats() to print the bowler's specific statistics.

Create a Main class to capture user input for a bowler and display their details.

Input Format

1. First, the user is prompted to enter the player's name (String).
2. Next, the user is asked to enter the player's team (String).
3. The user is prompted to enter the player's age (Integer).
4. Then, the user is asked to enter the number of wickets taken (Integer).
5. Finally, the user is asked to enter the bowling average (Double).

Constraints

NA

Output Format

- The output should display the player's general details first, followed by the bowling statistics.
- Each label and its corresponding value should be separated by a colon (:) and a space.
- The output should print each detail on a new line.

Sample Input 0

```
Jasprit Bumrah
India
29
300
24.5
```

Sample Output 0

```
Enter the player's name:
Enter the player's team:
Enter the player's age:
Enter the number of wickets taken:
```

```
Enter the bowling average:
Player Details:
Name: Jasprit Bumrah
Team: India
Age: 29
Bowling Statistics:
Wickets Taken: 300
Bowling Average: 24.5
```

Change Theme Language Java 7

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

class Player{
    String name,team;
    int age;

    void displayP(){
        System.out.println("Player Details:");
        System.out.println("Name: "+name);
        System.out.println("Team: "+team);
        System.out.println("Age: "+age);
    }
}

class Bowler extends Player{
    int wickets;
    double avg;

    void displayB(){
        System.out.println("Bowling Statistics:");
        System.out.println("Wickets Taken: "+wickets);
        System.out.println("Bowling Average: "+avg);
    }
}

public class Solution {

    public static void main(String[] args) {
```

Line: 56 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Polymorphism - Cricket Match Delivery Tracking System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In this scenario, we will develop a Cricket Match Delivery Tracking System that allows users to log and display details about specific deliveries during a cricket match. The system will utilize method overloading to handle different aspects of a delivery, such as player details (bowler and batsman) and run details.

Program Specifications

Class: Delivery

Methods:

- void displayDeliveryDetails(String bowler, String batsman): Displays the last names of the bowler and batsman for a particular delivery.
- void displayDeliveryDetails(Long runs): Displays run details for that delivery and indicates if it's a boundary or a six based on the runs scored.

Class: Main

Methods:

- public static void main(String[] args): Contains the menu-driven interface to test the Delivery class.

Input Format

1. The user can choose to log player details (bowler and batsman) or run details.
2. The user inputs names and run values as specified.

Constraints

NA

Output Format

The system will display the last names of the bowler and batsman or the run details based on user input.

Sample Input 0

```
1
Ravichandran Ashwin
Virat Kohli
```

Sample Output 0

```
Menu
1. Player details of the delivery
2. Run details of the delivery
Enter the bowler name:
Enter the batsman name:
Player details of the delivery:
Bowler: Ashwin
Batsman: Kohli
```

Sample Input 1

```
2
6
```

Sample Output 1

```
Menu
1. Player details of the delivery
2. Run details of the delivery
Enter the number of runs:
Number of runs scored in the delivery: 6
It is a Sixer.
```

Sample Input 2

```
2
4
```

Sample Output 2

```
Menu
1. Player details of the delivery
2. Run details of the delivery
Enter the number of runs:
Number of runs scored in the delivery: 4
It is a boundary.
```

Change Theme Language Java 7

```
class Delivery {
    void displayDeliveryDetails(String bowler, String batsman) {
        System.out.println("Bowler: " + b[b.length - 1]);
        System.out.println("Batsman: " + bat[bat.length - 1]);
    }

    void displayDeliveryDetails(Long runs) {
        System.out.println("Number of runs scored in the delivery: " + runs);

        if (runs == 6) {
            System.out.println("It is a Sixer.");
        } else if (runs == 4) {
            System.out.println("It is a boundary.");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Delivery d = new Delivery();

        System.out.println("Menu");
        System.out.println("1. Player details of the delivery");
        System.out.println("2. Run details of the delivery");

        int choice = Integer.parseInt(sc.nextLine().trim());

        if (choice == 1) {
```

Line: 55 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Blog | Scoring | Environment | FAQ | About Us | Helpdesk | Careers | Terms Of Service | Privacy Policy

Polymorphism - Simple Calculator

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In this scenario, we will create a Menu-Driven Simple Calculator application using method overloading in Java.

The calculator performs addition, subtraction, and multiplication on two numbers entered by the user.

The calculator supports:

- Operations on two integers
- Operations on two decimal (double) values

Based on the type of input provided by the user, the appropriate overloaded method is invoked automatically.

If the user enters decimal values, the calculation is performed using double methods.

If the user enters whole numbers, the calculation is performed using int methods.

The program continues to execute until the user chooses the Exit option from the menu.

Program Specifications

Class: Calculator

Methods:

- int add(int a, int b): Returns the sum of two integers.
- double add(double a, double b): Returns the sum of two doubles.
- int subtract(int a, int b): Returns the difference of two integers.
- double subtract(double a, double b): Returns the difference of two doubles.
- int multiply(int a, int b): Returns the product of two integers.
- double multiply(double a, double b): Returns the product of two doubles.

Class: Main

Methods:

- public static void main(String[] args): Contains the menu-driven interface to test the Calculator class.

Input Format

1. The user can choose an arithmetic operation (addition, subtraction, multiplication).
2. The user inputs two numbers for the selected operation.

Constraints

NA

Output Format

The system will display the result of the operation.

Sample Input 0

```
1
10
20
4
```

Sample Output 0

Menu:
1. Add
2. Subtract
3. Multiply
4. Exit
Enter your choice: Enter first number: Enter second number: Result: 30

Menu:
1. Add
2. Subtract
3. Multiply
4. Exit
Enter your choice: Exiting the calculator.

Sample Input 1

2
15
7
4

Sample Output 1

Menu:
1. Add
2. Subtract
3. Multiply
4. Exit
Enter your choice: Enter first number: Enter second number: Result: 8

Menu:
1. Add
2. Subtract
3. Multiply
4. Exit
Enter your choice: Exiting the calculator.

Sample Input 2

3
5
6
4

Sample Output 2

Menu:
1. Add
2. Subtract
3. Multiply
4. Exit
Enter your choice: Enter first number: Enter second number: Result: 30

Menu:
1. Add
2. Subtract
3. Multiply
4. Exit
Enter your choice: Exiting the calculator.

Change Theme Language Java 7

```
import java.util.Scanner;
class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
    public double add(double a, double b) {
        return a + b;
    }
    public int subtract(int a, int b) {
        return a - b;
    }
    public double subtract(double a, double b) {
        return a - b;
    }
    public int multiply(int a, int b) {
        return a * b;
    }
    public double multiply(double a, double b) {
        return a * b;
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Calculator calc = new Calculator();
        while (true) {
            System.out.println("Menu:");
```

Line: 72 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Polymorphism - Restaurant Food Delivery Tracker

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

The Restaurant Food Delivery Tracker is a software application designed to assist customers in tracking their food orders from various restaurants. In today's fast-paced world, customers often seek timely updates on their food delivery status. This application aims to enhance customer satisfaction by providing real-time updates based on different criteria: order ID, customer name, or a combination of both with the restaurant name. The application uses method overloading to cater to different user inputs efficiently.

In this scenario, we will create a Restaurant Food Delivery Tracker application that tracks food delivery orders based on various parameters. The application will utilize method overloading to handle different types of order queries.

Program Specifications

Class Name: FoodDelivery

Methods:

- void trackOrder(int orderId): This method tracks the delivery status of an order using the order ID.
- void trackOrder(String customerName): This method tracks all orders associated with a specific customer name.
- void trackOrder(String customerName, String restaurantName): This method tracks all orders for a specific customer from a specific restaurant.

Main Class: Main

- This class will interact with the user, allowing them to choose how they want to track their order.

Input Format

The user can choose to track an order using:

1. An order ID (integer).
2. A customer name (string).
3. A combination of customer name and restaurant name (two strings).

Constraints

NA

Output Format

The system will display the delivery status based on the input provided.

Sample Input 0

```
1
12345
```

Sample Output 0

```
Welcome to the Restaurant Food Delivery Tracker
Select tracking option:
1. Track by Order ID
2. Track by Customer Name
3. Track by Customer Name and Restaurant Name
Enter your choice (1-3): Enter the Order ID: Tracking Order ID: 12345
Order ID: 12345 is out for delivery.
```

Sample Input 1

2
Alice

Sample Output 1

Welcome to the Restaurant Food Delivery Tracker
Select tracking option:
1. Track by Order ID
2. Track by Customer Name
3. Track by Customer Name and Restaurant Name
Enter your choice (1-3): Enter the Customer Name: Tracking orders for customer: Alice
Customer Alice has 2 active orders.

Sample Input 2

3
Bob
Dominos

Sample Output 2

Welcome to the Restaurant Food Delivery Tracker
Select tracking option:
1. Track by Order ID
2. Track by Customer Name
3. Track by Customer Name and Restaurant Name
Enter your choice (1-3): Enter the Customer Name: Enter the Restaurant Name: Tracking orders for customer: Bob from restaurant: Dominos
Customer Bob has an order from Dominos that is ready for pickup.

Sample Input 3

5

Sample Output 3

Welcome to the Restaurant Food Delivery Tracker
Select tracking option:
1. Track by Order ID
2. Track by Customer Name
3. Track by Customer Name and Restaurant Name
Enter your choice (1-3): Invalid choice! Please select a valid option.

Change Theme Language Java 7

```
class FoodDelivery {
    void trackOrder(String customerName, String restaurantName) {
        System.out.println("Customer " + customerName + " has an order from "
                + restaurantName + " that is ready for pickup.");
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        FoodDelivery fd = new FoodDelivery();
```

```
27
28          System.out.println("Welcome to the Restaurant Food Delivery Tracker");
29          System.out.println("Select tracking option:");
30          System.out.println("1. Track by Order ID");
31          System.out.println("2. Track by Customer Name");
32          System.out.println("3. Track by Customer Name and Restaurant Name");
33          System.out.print("Enter your choice (1-3): ");
34
35          int choice = sc.nextInt();
36
37          switch (choice) {
38              case 1:
39                  System.out.print("Enter the Order ID: ");
40                  int orderId = sc.nextInt();
41                  fd.trackOrder(orderId);
42                  break;
43
```

Line: 65 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Polymorphism - Cricket Match Tracker

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

The Cricket Match Tracker application allows users to manage and display details about cricket matches. It leverages method overloading to provide different ways of displaying match-related information. This application is particularly useful for cricket fans, sports analysts, and event organizers who want to keep track of match schedules, venues, and outcomes.

Program Specifications

Class: Match Methods:

- void displayMatchDetails(Date matchDate): Displays the match date in MM-dd-yyyy format.
- void displayMatchDetails(String venue): Displays the stadium name and city separately.
- void displayMatchDetails(String winnerTeam, long runs): Displays the winner team and by how many runs they won.

Main Class: Main

- Contains the main method to test the Match class functionality.
- Utilizes user input to call the appropriate methods.

Input Format

The user can select an option to input match date, venue, or outcome.

The user provides:

1. Match date in dd/MM/yyyy format.
2. Venue in the format "Stadium Name, City".
3. Winning team name and the number of runs.

Constraints

NA

Output Format

The system displays the formatted match date, venue details, or match outcome based on user input.

Sample Input 0

```
1
15/08/2025
```

Sample Output 0

```
Menu
1. Match Date
2. Match Venue
3. Match Outcome
Enter your choice (1-3): Enter the date of the match (dd/MM/yyyy): Match Date: 08-15-2025
```

Sample Input 1

```
2
Eden Gardens, Kolkata
```

Sample Output 1

```
Menu
1. Match Date
2. Match Venue
3. Match Outcome
Enter your choice (1-3): Enter venue of the match (Stadium, City): Match Venue:
Stadium: Eden Gardens
City: Kolkata
```

Sample Input 2

```
3
India
45
```

Sample Output 2

```
Menu
1. Match Date
2. Match Venue
3. Match Outcome
Enter your choice (1-3): Enter the winner team of the match: Enter the number of runs: Match Outcome:
India won by 45 runs.
```

Change Theme Language Java 7

```
class Match {
    void displayMatchDetails(Date matchDate) {
        SimpleDateFormat sdf = new SimpleDateFormat("MM-dd-yyyy");
        System.out.println("Match Date: " + sdf.format(matchDate));
    }

    void displayMatchDetails(String venue) {
        String[] parts = venue.split(",");

        System.out.println("Match Venue:");
        System.out.println("Stadium: " + parts[0].trim());
        System.out.println("City: " + parts[1].trim());
    }

    void displayMatchDetails(String winnerTeam, long runs) {
        System.out.println("Match Outcome:");
        System.out.println(winnerTeam + " won by " + runs + " runs.");
    }
}

public class Main {
    public static void main(String[] args) throws ParseException {
        Scanner sc = new Scanner(System.in);
        Match match = new Match();

        System.out.println("Menu");
        System.out.println("1. Match Date");
        System.out.println("2. Match Venue");
```

Line: 78 Col: 1

Run Code Submit Code

Upload Code as File

Test against custom input

Polymorphism - Animal Sound Simulator

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

The Animal Sound Simulator application demonstrates the concept of method overriding in Java. It allows users to interact with different animal types and hear their unique sounds. This is particularly useful for educational purposes, helping children learn about animals and the sounds they make.

Animal Classes: Different animal classes (e.g., Dog, Cat) will extend a base class (Animal) and override the sound method to provide specific sounds for each animal. Sound Playback: Users can create objects of different animal types and call the overridden method to simulate the animal's sound.

Program Specifications

Class: Animal (Base Class)

- Method:
- void sound(): This method will be overridden in derived classes to provide specific sounds.

Derived Classes:

Dog: - Overrides: void sound(): Outputs "Bark".

Cat: - Overrides: void sound(): Outputs "Meow".

Main Class: Main

- Contains the main method to demonstrate method overriding.
- Uses user input to choose an animal and display the corresponding sound .

Input Format

The user is prompted to select an animal from a list (e.g., Dog or Cat).

Constraints

NA

Output Format

The system displays the sound produced by the selected animal.

Sample Input 0

```
1
```

Sample Output 0

```
Welcome to the Animal Sound Simulator!
Select an animal:
1. Dog
2. Cat
Enter your choice (1-2): Bark
```

Sample Input 1

```
2
```

Sample Output 1

```
Welcome to the Animal Sound Simulator!
Select an animal:
1. Dog
2. Cat
Enter your choice (1-2): Meow
```

Sample Input 2

```
3
```

Sample Output 2

```
Welcome to the Animal Sound Simulator!
Select an animal:
1. Dog
2. Cat
Enter your choice (1-2): Invalid choice.
```

Change Theme Language Java 7

```java

import java.util.Scanner;

class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Meow");
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

```

Line: 56 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Polymorphism - Cricket Player Profile with Role-Specific Details

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

A cricket team manager needs a system to keep track of the players on the team and their specific roles. Every player has basic information like their name, country, and the number of matches they've played. However, based on their specialization, each player type has additional details:

Batsman has a batting average and total runs scored. Bowler has a bowling average and total wickets taken. All-rounder has both batting and bowling averages. The manager wants to record player details and retrieve information about each player's statistics based on their roles, using inheritance to structure common and specific player attributes.

Program Specifications

Create a base class named Player:

Attributes:

- String name: Stores the player's name.
- String country: Stores the country the player represents.
- int matches: Stores the number of matches the player has played.

Constructors:

- Parameterized Constructor: Initializes name, country, and matches.

Method:

- public void displayPlayerInfo(): Displays basic player information (name, country, matches).

Create three subclasses Batsman, Bowler, and AllRounder:

Batsman:

- Additional Attributes: double battingAverage, int totalRuns.
- Constructor: Initializes all player details plus the batting average and total runs.
- public void displayPlayerInfo(): Overrides the base method to include batting details.

Bowler:

- Additional Attributes: double bowlingAverage, int totalWickets.
- Constructor: Initializes all player details plus the bowling average and total wickets.
- public void displayPlayerInfo(): Overrides the base method to include bowling details.

AllRounder:

- Additional Attributes: double battingAverage, double bowlingAverage.
- Constructor: Initializes all player details plus batting and bowling averages.
- public void displayPlayerInfo(): Overrides the base method to include both batting and bowling details.

Create a class named Main:

- Purpose: Capture user input for player type, instantiate the appropriate player subclass, and display player details.

Input Format

- Player Type (Batsman, Bowler, or AllRounder).
- Player's Name (String).
- Country Name (String).
- Number of Matches (Integer).
- Additional inputs based on player type:

1. For Batsman: Batting Average (Double), Total Runs (Integer).
2. For Bowler: Bowling Average (Double), Total Wickets (Integer).
3. For AllRounder: Batting Average (Double), Bowling Average (Double).

Constraints

NA

Output Format

Displays player details in a structured format, showing specific stats based on the player type.

Sample Input 0

```
Batsman
Virat Kohli
India
250
59.33
12000
```

Sample Output 0

```
Enter player type (Batsman/Bowler/AllRounder):
Enter player name:
Enter country:
Enter matches played:
Enter batting average:
Enter total runs:

Player Details:
Name: Virat Kohli
Country: India
Matches Played: 250
Player Type: Batsman
Batting Average: 59.33
Total Runs: 12000
```

Sample Input 1

```
Bowler
James Anderson
England
150
26.85
600
```

Sample Output 1

```
Enter player type (Batsman/Bowler/AllRounder):
Enter player name:
Enter country:
Enter matches played:
Enter bowling average:
Enter total wickets:

Player Details:
Name: James Anderson
Country: England
Matches Played: 150
Player Type: Bowler
Bowling Average: 26.85
Total Wickets: 600
```

Sample Input 2

```
AllRounder
Ben Stokes
England
80
35.50
29.75
```

Sample Output 2

```
Enter player type (Batsman/Bowler/AllRounder):
Enter player name:
Enter country:
Enter matches played:
Enter batting average:
Enter bowling average:

Player Details:
Name: Ben Stokes
Country: England
Matches Played: 80
Player Type: AllRounder
Batting Average: 35.5
Bowling Average: 29.75
```

Change Theme Language Java 7

```
1
2  import java.util.Scanner;
3
4  class Player {
5      String name;
6      String country;
7      int matches;
8
9      Player(String name, String country, int matches) {
10         this.name = name;
11         this.country = country;
12         this.matches = matches;
13     }
14
15     public void displayPlayerInfo() {
16         System.out.println("Name: " + name);
17         System.out.println("Country: " + country);
18         System.out.println("Matches Played: " + matches);
19     }
20 }
21
22 class Batsman extends Player {
23     double battingAverage;
24     int totalRuns;
25
26     Batsman(String name, String country, int matches,
27             double battingAverage, int totalRuns) {
```

Line: 142 Col: 1

Run Code Submit Code

Upload Code as File

Test against custom input

Polymorphism - Cricket Player Information Hub

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

The Cricket Player Information Hub is designed to help cricket teams manage and view player details based on specific criteria. Coaches and team managers can input player information, retrieve complete player lists, and filter players by their country. This system utilizes object-oriented programming principles, such as encapsulation and method overriding, for a clean and efficient design.

Program Specifications

Class: Player

Attributes:

- private String name
- private String country
- private String skill

Constructor: Initializes player details.

Getters and Setters: For each attribute.

Override:

- toString(): Formats the output to display player details.

Class: PlayerBO

Methods:

- void displayAllPlayerDetails(Player[] playerList): Displays the details of all players.
- void displaySpecificPlayerDetails(Player[] playerList, String countryName): Displays details of players from a specific country.

Class: Main

- Main Method: Tests the functionality by taking user input and invoking methods from PlayerBO.

Input Format

1. The user is prompted to enter the number of players and their details (name, country, and skill).
2. After entering the player details, the user can specify a country to filter players.

Constraints

NA

Output Format

- The system displays all player details and details of players filtered by country.
- Print the player details by using this spacing format(%-15s %-15s %s).

Sample Input 0

```
2
Virat Kohli
India
Batsman
Steve Smith
Australia
Batsman
India
```

Sample Output 0

```
Enter the number of players
Enter the player name
Enter the country name
Enter the skill
Enter the player name
Enter the country name
Enter the skill

Player Details
Virat Kohli     India          Batsman
Steve Smith     Australia      Batsman

Enter the country name for which players details to be known

Player Details
Virat Kohli     India          Batsman
```

Sample Input 1

```
3
MS Dhoni
India
Wicketkeeper
Kane Williamson
New Zealand
Batsman
Mitchell Starc
Australia
Bowler
Australia
```

Sample Output 1

```
Enter the number of players
Enter the player name
Enter the country name
Enter the skill
Enter the player name
Enter the country name
Enter the skill
Enter the player name
Enter the country name
Enter the skill

Player Details
MS Dhoni        India          Wicketkeeper
Kane Williamson New Zealand    Batsman
Mitchell Starc  Australia      Bowler

Enter the country name for which players details to be known

Player Details
Mitchell Starc  Australia      Bowler
```

Change Theme Language Java 7

```
2  import java.util.Scanner;
3
```

```
class Player {
    private String name;
    private String country;
    private String skill;

    public Player(String name, String country, String skill) {
        this.name = name;
        this.country = country;
        this.skill = skill;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCountry() {
        return country;
    }

    public void setCountry(String country) {
        this.country = country;
    }
```

Line: 103 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Polymorphism - Player Detail Comparison

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a sports management application, we need to manage and compare player details for cricket players. This system allows users to input player information and check whether two players have the same attributes (name, country, and skill). This feature is particularly useful for ensuring data consistency, especially when managing player registrations or rosters.

Program Specifications

Class: Player

Private Attributes:

- String name
- String country
- String skill

Getters:

- getName()
- getCountry()
- getSkill()

Setters:

- setName(String name)
- setCountry(String country)
- setSkill(String skill)

Override the equals() method to compare the current object with another Player object based on the values of name, country, and skill.

Class: Main

- In the main() method, prompt the user to input details for two players.
- After collecting the details, display the entered player information.
- Use the overridden equals() method to compare the two Player objects and display whether they are the same or not.

Input Format

The user will input player details, including name, country, and skill for both players.

Constraints

NA

Output Format

- Display the player details in the specified format.
- Indicate whether the details of the two players are the same.

Sample Input 0

```
Player 1:
Name: Virat
Country: India
Skill: Batsman

Player 2:
Name: Virat
```

Country: India
Skill: Batsman

Sample Output 0

Enter the player 1 details
Enter the player name
Enter the country name
Enter the skill
Player 1:----- Name: Virat------ Country: India

Enter the player 2 details
Enter the player name
Enter the country name
Enter the skill
Skill: Batsman----- ------ Player 2:
Both the player details are same.

Sample Input 1

Player 1:
Name: Rohit
Country: India
Skill: Batsman

Player 2:
Name: Jasprit
Country: India
Skill: Bowler

Sample Output 1

Enter the player 1 details
Enter the player name
Enter the country name
Enter the skill
Player 1:----- Name: Rohit------ Country: India

Enter the player 2 details
Enter the player name
Enter the country name
Enter the skill
Skill: Batsman----- ------ Player 2:
Both the player details are different.

Change Theme Language Java 7

```
1
2  import java.util.Scanner;
3  import java.util.Objects;
4
5  class Player {
6      private String name;
7      private String country;
8      private String skill;
9
10     public Player(String name, String country, String skill) {
11         this.name = name;
12         this.country = country;
```

```
13          this.skill = skill;
14      }
15
16      public String getName() { return name; }
17      public String getCountry() { return country; }
18      public String getSkill() { return skill; }
19
20      public void setName(String name) { this.name = name; }
21      public void setCountry(String country) { this.country = country; }
22      public void setSkill(String skill) { this.skill = skill; }
23
24      @Override
25      public boolean equals(Object obj) {
26          if (this == obj) return true;
27          if (!(obj instanceof Player)) return false;
```

Line: 92 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Abstraction - Vehicle Registration System

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a bustling city, a new vehicle registration system is needed to manage various types of vehicles. The city requires a streamlined way to register different vehicles such as cars and motorcycles, and it wants to ensure that each vehicle's details are captured correctly. To achieve this, an abstract class named Vehicle will be created to define the common properties and methods that all vehicles share. Specific vehicle types like Car and Motorcycle will extend this abstract class, allowing the system to be flexible and easily maintainable.

Program Specifications

Abstract Class: Vehicle

Attributes:

- String ownerName
- String registrationNumber

Constructor:

- Vehicle(String ownerName, String registrationNumber)

Methods:

- abstract void displayDetails()

Derived Class: Car

Additional Attribute:

- int numberOfDoors

Constructor:

- Car(String ownerName, String registrationNumber, int numberOfDoors)

Method:

- void displayDetails() (Overrides the abstract method)

Derived Class: Motorcycle

Additional Attribute:

- boolean hasSidecar

Constructor:

- Motorcycle(String ownerName, String registrationNumber, boolean hasSidecar)

Method:

- void displayDetails() (Overrides the abstract method)

Driver Class: Main

- The main method will prompt the user to input vehicle details and display the registered vehicle's information.

Input Format

1. Enter the type of vehicle (1 for Car, 2 for Motorcycle)
2. Enter the owner's name
3. Enter the registration number
4. For Car: Enter the number of doors
5. For Motorcycle: Enter whether it has a sidecar (true or false)

Constraints

NA

Output Format

Display the vehicle's details based on the inputs.

Sample Input 0

```
1
John Doe
ABC1234
4
```

Sample Output 0

```
Enter the type of vehicle (1 for Car, 2 for Motorcycle):
Enter owner's name:
Enter registration number:
Enter number of doors:
Vehicle Details:
Owner: John Doe
Registration Number: ABC1234
Number of Doors: 4
```

Sample Input 1

```
2
Jane Smith
XYZ5678
true
```

Sample Output 1

```
Enter the type of vehicle (1 for Car, 2 for Motorcycle):
Enter owner's name:
Enter registration number:
Does it have a sidecar? (true/false):
Vehicle Details:
Owner: Jane Smith
Registration Number: XYZ5678
Has Sidecar: true
```

Change Theme Language Java 7

```
1
2  import java.util.Scanner;
3
4  abstract class Vehicle {
5      String ownerName;
6      String registrationNumber;
7
8      Vehicle(String ownerName, String registrationNumber) {
9          this.ownerName = ownerName;
10         this.registrationNumber = registrationNumber;
11     }
12
13     abstract void displayDetails();
14 }
```

```
15
16  class Car extends Vehicle {
17      int numberOfDoors;
18
19      Car(String ownerName, String registrationNumber, int numberOfDoors) {
20          super(ownerName, registrationNumber);
```

Line: 79 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Abstraction - Shape Volume Calculator in a 3D Design Tool

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a 3D design tool used for educational or architectural purposes, designers often need to calculate the volume of different 3D shapes, such as spheres, cubes, and cuboids. This helps in estimating material usage and object scaling. Using Java, we can create an abstraction that allows users to calculate the volume of various shapes. By providing an abstract base class and concrete implementations, we ensure a modular and scalable structure that can be expanded to include other shapes in the future.

Program Specifications

Abstract Class: Volume

Protected Attribute:

- String name - the name of the shape.

Constructor:

- Initializes the name attribute.

Abstract Method:

- Float calculateVolume(): an abstract method that subclasses must override to calculate their specific volume.

Concrete Classes:

VolumeOfSphere:

- Private Attribute: Integer radius
- Constructor: Initializes name and radius.
- Method: calculateVolume() - calculates the volume of a sphere using the formula V=4/3πr3 (where pi = 3.14).

VolumeOfCube:

- Private Attribute: Integer side
- Constructor: Initializes name and side.
- Method: calculateVolume() - calculates the volume of a cube using the formula V=s3

VolumeOfCuboid:

- Private Attributes: Integer length, breadth, height
- Constructor: Initializes name, length, breadth, and height.
- Method: calculateVolume() - calculates the volume of a cuboid using the formula V=l×b× h.

Main Class:

- The main() method accepts the shape name and its respective dimensions as input.
- Based on the shape type, the method creates an instance of the appropriate subclass and calls the calculateVolume() method.
- The program should validate that all dimension values are non-negative; if any value is negative, it outputs "Invalid Input."

Input Format

NA

Constraints

NA

Output Format

NA

Sample Input 0

```
Sphere
3
```

Sample Output 0

```
Enter the shape (Sphere, Cube, or Cuboid):
Enter the radius of the sphere:
The volume of the sphere is 113.04
```

Sample Input 1

```
Cube
4
```

Sample Output 1

```
Enter the shape (Sphere, Cube, or Cuboid):
Enter the side of the cube:
The volume of the cube is 64.00
```

Change Theme Language Java 7

```java

import java.util.Scanner;

abstract class Volume {
    protected String name;

    Volume(String name) {
        this.name = name;
    }

    abstract float calculateVolume();
}

class VolumeOfSphere extends Volume {
    private int radius;

    VolumeOfSphere(String name, int radius) {
        super(name);
        this.radius = radius;
    }
```

Line: 114 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Blog | Scoring | Environment | FAQ | About Us | Helpdesk | Careers | Terms Of Service | Privacy Policy

Abstraction - Library Management System for Different Types of Books

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a library, books are categorized into different types, such as Reference Books and Regular Books. Each type of book has specific characteristics. For example, Reference Books are not available for borrowing, while Regular Books can be borrowed. Using abstraction, we can define a general Book class with common book attributes and methods, and then use specific subclasses for each type of book with their own behaviors.

Program Specifications

Abstract Class: Book

Protected Attributes:

- String title
- String author

Constructor:

- Initializes title and author attributes.

Abstract Method:

- void displayInfo(): Abstract method that will be implemented in each subclass to display the information based on the book type.

Subclasses:

- ReferenceBook: Implements displayInfo() to indicate that the book is only available for in-library use.
- RegularBook: Implements displayInfo() to show that the book can be borrowed.

Class: Library

- In the main() method, prompt the user to input details of the book and specify its type.
- Based on the selected book type, create an instance of the appropriate subclass.
- Call the displayInfo() method to display book details.

Input Format

The user inputs the book type (Reference or Regular), title, and author.

Constraints

NA

Output Format

The program displays information about the book, including its title, author, and availability status.

Sample Input 0

```
1
The Art of Computer Programming
Donald E. Knuth
```

Sample Output 0

```
Enter book type (1 for Reference, 2 for Regular):
Enter book title:
Enter author name:
Book Title: The Art of Computer Programming
```

Author: Donald E. Knuth
Availability: In-library use only

Sample Input 1

2
To Kill a Mockingbird
Harper Lee

Sample Output 1

Enter book type (1 for Reference, 2 for Regular):
Enter book title:
Enter author name:
Book Title: To Kill a Mockingbird
Author: Harper Lee
Availability: Available for borrowing

Change Theme Language Java 7

```
import java.util.*;

abstract class Book {
    protected String title;
    protected String author;

    Book(String title, String author) {
        this.title = title;
        this.author = author;
    }

    abstract void displayInfo();
}

class ReferenceBook extends Book {

    ReferenceBook(String title, String author) {
        super(title, author);
    }

```

Line: 68 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Abstraction - Online Payment System for Different Payment Methods

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In an online shopping platform, customers have multiple payment options, such as credit card, debit card, and digital wallets. To process these payments, each payment method has unique details and processes, but they all follow a general approach to initiate a payment. By using abstraction, we can define a general payment class and specific implementations for each payment method.

Program Specifications

Abstract Class: Payment

Protected Attributes:

- double amount

Constructor:

- Initializes the amount attribute.

Abstract Method:

- void makePayment(): Abstract method that will be overridden to specify the process for each payment type.

Subclasses:

- CreditCardPayment: Implements makePayment() to specify the process for credit card payments.
- DebitCardPayment: Implements makePayment() to specify the process for debit card payments.
- DigitalWalletPayment: Implements makePayment() to specify the process for digital wallet payments.

Class: PaymentSystem

- In the main() method, prompt the user to input the payment method, amount, and additional payment-specific information.
- Based on the chosen payment method, create an instance of the appropriate subclass.
- Call the makePayment() method to process the payment.

Input Format

1. The user inputs the payment method (credit card, debit card, or digital wallet).
2. The user enters the payment amount and, depending on the payment method, specific details (e.g., card number, wallet ID).

Constraints

NA

Output Format

The program displays confirmation with the amount and method of payment.

Sample Input 0

```
1
250.00
4321-8765-5678-1234
456
```

Sample Output 0

```
Select payment method:
1. Credit Card
2. Debit Card
3. Digital Wallet
Enter payment amount:
Enter credit card number:
Enter CVV:
Payment of $250.0 processed using Credit Card.
Card Number: 4321-8765-5678-1234
```

Sample Input 1

```
2
100.50
5678-1234-7890-4321
9876
```

Sample Output 1

```
Select payment method:
1. Credit Card
2. Debit Card
3. Digital Wallet
Enter payment amount:
Enter debit card number:
Payment of $100.5 processed using Debit Card.
Card Number: 5678-1234-7890-4321
```

Sample Input 2

```
3
200.00
wallet5678
```

Sample Output 2

```
Select payment method:
1. Credit Card
2. Debit Card
3. Digital Wallet
Enter payment amount:
Enter wallet ID:
Payment of $200.0 processed using Digital Wallet.
Wallet ID: wallet5678
```

Change Theme Language Java 7

```
1  import java.util.*;
2
3  abstract class Payment {
4      protected double amount;
5
6      Payment(double amount) {
7          this.amount = amount;
8      }
9
```

```
10      abstract void makePayment();
11  }
12
13  class CreditCardPayment extends Payment {
14      String cardNumber;
15      String cvv;
16
17      CreditCardPayment(double amount, String cardNumber, String cvv) {
18          super(amount);
19          this.cardNumber = cardNumber;
20          this.cvv = cvv;
```

Line: 101 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Abstraction - Library Membership Management System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Clara, a librarian, finds it challenging to track library member details, such as card numbers, book borrowing limits, and return dates. To simplify her workflow, we will design a basic Library Membership Management System that categorizes memberships into two types: Membership Card and LifeLong Membership Card. The system will allow her to enter member details and select the type of membership. Each membership type has different borrowing limits and return policies.

This solution uses Java to implement the classes and attributes Clara needs, including member details, borrowing limits, and return dates.

Program Specifications

Abstract Class: LibraryCard

Protected Attributes:

- String memberDetails: holds combined member information including name, card number, and expiry date.

Constructor:

- Initializes the memberDetails attribute.

Getters and Setters:

- Provides access to memberDetails.

Concrete Classes:

MembershipCard:

Private Attribute:

- Integer minimumBookToBorrow: minimum number of books the member can borrow.

Constructor:

- Takes memberDetails and minimumBookToBorrow.

Due Date Policy: Members with this card can keep books for 10 days.

LifeLongMembershipCard:

Private Attribute:

- Integer minimumBookToBorrow: minimum number of books the member can borrow.

Constructor:

- Takes memberDetails and minimumBookToBorrow.

Due Date Policy: Members with this card have an unlimited due date.

Main Class:

- Prompts Clara to choose a membership type and enter member details in the specified format.
- Based on the choice, it creates the appropriate membership type and displays details.

Input Format

1. First Input: Choice of membership type (Integer: 1 for LifeLong Membership Card, 2 for Membership Card).
2. Second Input: Member details in the format: Name | CardNumber | ExpiryDate (String).
3. Third Input: Minimum number of books to borrow (Integer).

Constraints

NA

Output Format

1. Display the member's name followed by "Membership Card Details".
2. On the next line, display "Card Number:" followed by the card number.
3. On the next line, display "Minimum books to borrow:" followed by the minimum number of books.
4. On the final line, display the "Due date to return the book:" with the respective return period.
    - For Membership Card: "10 days from the book taken".
    - For LifeLong Membership Card: "unlimited".
5. If the choice is invalid: Print " Invalid choice".

Sample Input 0

```
1
John Doe | 23456 | 01/05/2025
3
```

Sample Output 0

```
Here comes a choice to view
1. Lifelong Membership Card
2. Membership Card
Enter member details in format: Name | CardNumber | ExpiryDate
Enter the minimum number of books to borrow:
John Doe's Membership Card Details
Card Number: 23456
Minimum books to borrow: 3
Due date to return the book: unlimited
```

Sample Input 1

```
2
Alice Smith | 34567 | 22/07/2023
1
```

Sample Output 1

```
Here comes a choice to view
1. Lifelong Membership Card
2. Membership Card
Enter member details in format: Name | CardNumber | ExpiryDate
Enter the minimum number of books to borrow:
Alice Smith's Membership Card Details
Card Number: 34567
Minimum books to borrow: 1
Due date to return the book: 10 days from the book taken
```

Sample Input 2

```
3
Tom Baker | TB111 | 01-01-2027
4
```

Sample Output 2

```
Here comes a choice to view
1. Lifelong Membership Card
2. Membership Card
Enter member details in format: Name | CardNumber | ExpiryDate
Enter the minimum number of books to borrow:
Invalid choice
```

Change Theme Language Java 7

```
1
2  import java.io.*;
3  import java.util.*;
4
5  abstract class Librarycard {
6      protected String member;
7
8      Librarycard(String member) {
9          this.member = member;
10     }
11
12     // getter
13     public String getmember() {
14         return member;
15     }
16
17     // setter
18     public void setmember(String member) {
19         this.member = member;
20     }
```

Line: 98 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

- Sample Test case 0
- Sample Test case 1
- Sample Test case 2

Input (stdin) Download

```
1
John Doe | 23456 | 01/05/2025
3
```

Your Output (stdout)

```
Here comes a choice to view
1. Lifelong Membership Card
2. Membership Card
Enter member details in format: Name | CardNumber | ExpiryDate
Enter the minimum number of books to borrow:
John Doe's Membership Card Details
```

Blog | Scoring | Environment | FAQ | About Us | Helpdesk | Careers | Terms Of Service | Privacy Policy

Abstraction - Library Management System for Book Details and Borrowing Status

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a library management system, we need to handle different types of books and their borrowing status. Here, the concept of abstraction allows us to define a generic book class and abstract methods for tracking different borrowing statuses (e.g., available, reserved, or borrowed). This abstraction is useful because we can define general methods in the parent class and provide specific implementations in each subclass.

Program Specifications

Abstract Class: Book

Protected Attributes:

- String title
- String author
- int yearPublished

Constructor:

- Initializes the title, author, and yearPublished attributes.

Abstract Method:

- void displayStatus(): Abstract method that will display the borrowing status of the book. Each subclass will implement this to provide a specific status.

Concrete Method:

- void displayBookInfo(): Displays the basic information about the book (title, author, and publication year).

Subclasses:

- AvailableBook: Implements displayStatus() method to show that the book is currently available.
- ReservedBook: Implements displayStatus() method to indicate the book is reserved.
- BorrowedBook: Implements displayStatus() method to show that the book is borrowed.

Class: LibraryManagement

- In the main() method, prompt the user to input book information, the year of publication, and the book's status.
- Based on the chosen status, create an object of the appropriate subclass.
- Display the book information and its status.

Input Format

The user inputs the book title, author, publication year, and status.

Constraints

NA

Output Format

The program displays the book information, followed by the borrowing status.

Sample Input 0

```
Moby Dick
Herman Melville
```

```
1851
1
```

Sample Output 0

```
Enter the book title:
Enter the author:
Enter the year of publication:
Select book status:
1. Available
2. Reserved
3. Borrowed
Book Information:
Title: Moby Dick
Author: Herman Melville
Year Published: 1851
Status: The book is available.
```

Sample Input 1

```
The Great Gatsby
F. Scott Fitzgerald
1925
2
```

Sample Output 1

```
Enter the book title:
Enter the author:
Enter the year of publication:
Select book status:
1. Available
2. Reserved
3. Borrowed
Book Information:
Title: The Great Gatsby
Author: F. Scott Fitzgerald
Year Published: 1925
Status: The book is reserved.
```

Sample Input 2

```
1984
George Orwell
1949
3
```

Sample Output 2

```
Enter the book title:
Enter the author:
Enter the year of publication:
Select book status:
1. Available
2. Reserved
3. Borrowed
Book Information:
Title: 1984
Author: George Orwell
Year Published: 1949
Status: The book is borrowed.
```

Change Theme Language Java 7

```java
import java.util.*;

abstract class Book {
    protected String title;
    protected String author;
    protected int yearPublished;

    Book(String title, String author, int yearPublished) {
        this.title = title;
        this.author = author;
        this.yearPublished = yearPublished;
    }

    abstract void displayStatus();

    void displayBookInfo() {
        System.out.println("Book Information:");
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
        System.out.println("Year Published: " + yearPublished);
```

Line: 93 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Abstraction - Cricket Match Run Rate Calculator

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine a sports analytics app is being developed to provide real-time updates to fans watching a cricket match. The app includes a feature where users can check the required run rate and remaining balls for any ongoing game in one of the three formats: ODI, T20, or Test. The app calculates and displays the run rate based on the user's inputs for each game format.

Program Specifications

Abstract Class: Match

Attributes (private):

- int currentScore - The team's current score.
- float currentOver - The current over in the innings.
- int target - The target score to win.

Methods:

- Abstract method float calculateRunrate(): calculates the required run rate.
- Abstract method int calculateBalls(): calculates the remaining balls.
- Method void display(double reqRunRate, int balls): displays the number of balls remaining and the required run rate in a specified format.

Derived Classes:

- ODIMatch: A class extending Match with specific calculations for 50 overs (300 balls).
- T20Match: A class extending Match with calculations for 20 overs (120 balls).
- TestMatch: A class extending Match with calculations for 90 overs (540 balls).

Driver Class:

Main

- Creates instances of the ODIMatch, T20Match, and TestMatch classes and calculates the required run rate and remaining balls for each format.

Input Format

1. Cricket format (1 for ODI, 2 for T20, 3 for Test)
2. Current Score (int)
3. Current Over (float)
4. Target Score (int)

Constraints

NA

Output Format

Display requirements: Runs needed, balls remaining, and required run rate to two decimal places.

Sample Input 0

```
1
256
30
400
```

Sample Output 0

```
Enter the Cricket Format
1.ODI
2.T20
3. Test
Enter the Current Score
Enter the Current Over
Enter the Target Score
Requirements:
Need 144 Runs in 120 balls
Required Run Rate - 7.20
```

Sample Input 1

```
2
120
15
170
```

Sample Output 1

```
Enter the Cricket Format
1.ODI
2.T20
3. Test
Enter the Current Score
Enter the Current Over
Enter the Target Score
Requirements:
Need 50 Runs in 30 balls
Required Run Rate - 10.00
```

Sample Input 2

```
3
200
60
450
```

Sample Output 2

```
Enter the Cricket Format
1.ODI
2.T20
3. Test
Enter the Current Score
Enter the Current Over
Enter the Target Score
Requirements:
Need 250 Runs in 180 balls
Required Run Rate - 8.33
```

Change Theme Language Java 7

```
1  import java.util.*;
2
3  abstract class Match {
```

```
4       private int currentScore;
5       private float currentOver;
6       private int target;
7
8       Match(int currentScore, float currentOver, int target) {
9           this.currentScore = currentScore;
10          this.currentOver = currentOver;
11          this.target = target;
12      }
13
14      int getCurrentScore() {
15          return currentScore;
16      }
17
18      float getCurrentOver() {
19          return currentOver;
20      }
```

Line: 117 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Abstraction - Digital Banking Account Tracker

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

A bank provides different types of accounts to its customers, such as Savings and Current accounts. Each account type has its own rules for calculating interest and managing the minimum balance. The bank wants a system where basic account operations like checking balance, depositing, and withdrawing are handled, while the specific account rules (e.g., interest rates, minimum balance) are managed differently for each account type. This requires using abstraction to define common behaviors while allowing specific account types to implement their unique rules.

Program Specifications

Create an abstract class named BankAccount:

Attributes (Protected):

- String accountNumber: Stores the account number.
- double balance: Stores the balance in the account.

Constructors:

- Parameterized Constructor: Initializes accountNumber and balance.

Abstract Methods:

- public abstract void withdraw(double amount): Withdraws a specified amount.
- public abstract void deposit(double amount): Deposits a specified amount.
- public abstract double calculateInterest(): Calculates interest based on the account type.

Concrete Methods:

- public double getBalance(): Returns the current balance.
- public String getAccountNumber(): Returns the account number.

Create two subclasses SavingsAccount and CurrentAccount:

SavingsAccount:

- Implements calculateInterest() with a 4% interest rate.
- Minimum balance of $500 for withdrawals.

CurrentAccount:

- Implements calculateInterest() with no interest.
- Minimum balance of $1000 for withdrawals.

Create a class named Main:

- Purpose: Capture user input for account type, perform operations like deposit, withdraw, and display account details.

Input Format

1. Account Type (Savings or Current).
2. Account Number (String).
3. Initial Deposit (Double).
4. Deposit amount (Double).
5. Withdrawal amount (Double).

Constraints

NA

Output Format

Displays account details, including:

Account Type, Account Number, Balance, Interest (if any).

Sample Input 0

```
Savings
SAV001
1000.00
500.00
200.00
```

Sample Output 0

```
Enter account type (Savings/Current):
Enter account number:
Enter initial deposit:
Enter deposit amount:
Enter withdrawal amount:
Account Details:
Account Type: Savings
Account Number: SAV001
Balance: $1300.0
Interest: $52.00
```

Sample Input 1

```
Current
CUR002
2000.00
1000.00
1500.00
```

Sample Output 1

```
Enter account type (Savings/Current):
Enter account number:
Enter initial deposit:
Enter deposit amount:
Enter withdrawal amount:
Account Details:
Account Type: Current
Account Number: CUR002
Balance: $1500.0
```

Change Theme Language Java 7

```
1
2  import java.util.Scanner;
3
4  abstract class BankAccount {
5      protected String accountNumber;
6      protected double balance;
7
8      BankAccount(String accountNumber, double balance) {
9          this.accountNumber = accountNumber;
10         this.balance = balance;
11     }
```

```
12
13      public abstract void withdraw(double amount);
14
15      public abstract void deposit(double amount);
16
17      public abstract double calculateInterest();
18
19      public double getBalance() {
20          return balance;
```

Line: 125 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Sample Test case 1

Input (stdin) Download

```
Savings
SAV001
1000.00
500.00
200.00
```

Your Output (stdout)

```
Enter account type (Savings/Current):
Enter account number:
Enter initial deposit:
Enter deposit amount:
Enter withdrawal amount:
```

Interface - Animal Echoes

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Imagine a classroom where students are learning about different animals and the sounds they make. To make the learning process engaging, we will create an interface called Animal that defines the behavior of animals, specifically the ability to make a sound. Students will implement this interface in different classes representing various animals, allowing them to understand the concept of interfaces in Java.

Program Specifications

Interface: Animal

Methods:

- void makeSound(); - This method will be implemented by different animal classes to provide their specific sounds.

Implementing Classes:

Dog - Implements the Animal interface.

Methods:

- makeSound() - Outputs "Woof!" to represent the sound a dog makes.

Cat - Implements the Animal interface.

Methods:

- makeSound() - Outputs "Meow!" to represent the sound a cat makes.

Cow - Implements the Animal interface.

Methods:

- makeSound() - Outputs "Moo!" to represent the sound a cow makes.

Driver Class: Main

- The main method will create instances of each animal and call their makeSound() method to demonstrate polymorphism and interface implementation.

Input Format

No user input is required in this simple program; the program will automatically create and use instances of the animal classes.

Constraints

NA

Output Format

Display the sound made by each animal.

Sample Output 0

```
The dog goes: Woof!
The cat goes: Meow!
The cow goes: Moo!
```

Change Theme Language Java 7

```java
import java.util.*;
interface Animal{
    void makeSound();
}
class Dog implements Animal{
    public void makeSound(){
    System.out.println("The dog goes: Woof!");
    }
}
class Cat implements Animal{
    public void makeSound(){
    System.out.println("The cat goes: Meow!");
    }
}
class Cow implements Animal{
    public void makeSound(){
    System.out.println("The cow goes: Moo!");
}
}
public class Main{
    public static void main(String args[]){
        Dog d = new Dog();
        Cat c = new Cat();
        Cow b = new Cow();
        d.makeSound();
        c.makeSound();
        b.makeSound();
```

Line: 30 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Interface - Player Statistics Management System

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a sports management application, there is a need to keep track of players' statistics across various teams. This system allows sports analysts, coaches, and fans to view player performance details, making it easier to assess their contributions to the team. The application uses an interface to ensure that all player classes implement the required method for displaying statistics.

Program Specifications

Interface Creation:

- Create an interface named IPlayerStatistics with the method:

public void displayPlayerStatistics();

Class Implementation:

- Create a class named Player that implements the IPlayerStatistics interface.
- Include private data members:
  1. String name
  2. String teamName
  3. Integer noOfMatches
  4. Long totalRunsScored
  5. Integer noOfWicketsTaken

Include a constructor that initializes all the data members.

Implement the displayPlayerStatistics method to display the player details.

Main Class:

- Create a Main class with a main method to gather user input and display player statistics.

Input Format

1. Player name (String)
2. Team name (String)
3. Number of matches played (Integer)
4. Total runs scored (Long)
5. Number of wickets taken (Integer)

Constraints

NA

Output Format

Display player details in the specified format.

Sample Input 0

```
Virat Kohli
Royal Challengers Bangalore
200
6000
4
```

Sample Output 0

Enter player name: Enter team name: Enter number of matches played: Enter total runs scored: Enter number of wickets taken: Player Details:
Player name: Virat Kohli
Team name: Royal Challengers Bangalore
No of matches: 200
Total runs scored: 6000
No of wickets taken: 4

Sample Input 1

MS Dhoni
Chennai Super Kings
175
4500
0

Sample Output 1

Enter player name: Enter team name: Enter number of matches played: Enter total runs scored: Enter number of wickets taken: Player Details:
Player name: MS Dhoni
Team name: Chennai Super Kings
No of matches: 175
Total runs scored: 4500
No of wickets taken: 0

Change Theme Language Java 7

```
import java.util.*;
interface IPlayerStatistics{
    void displayPlayerStstistics();
}
class Player implements IPlayerStatistics{
    private String name;
    private String teamName;
    private int noOfMatches;
    private long totalRunsScored;
    private int noOfWicketsTaken;

    Player(String name , String teamname,int noOfMatches,long totalRunsScored,int noOfWickets
        this.name = name;
        this.teamName = teamname;
        this.noOfMatches=noOfMatches;
        this.totalRunsScored = totalRunsScored;
        this.noOfWicketsTaken = noOfWicketsTaken;
    }
    public void displayPlayerStstistics(){
        System.out.println("Player Details:");
        System.out.println("Player name: "+name);
        System.out.println("Team name: "+teamName);
         System.out.println("No of matches: "+noOfMatches);
          System.out.println("Total runs scored: "+totalRunsScored);
           System.out.println("No of wickets taken: "+noOfWicketsTaken);
    }
}
```

Line: 45 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Blog | Scoring | Environment | FAQ | About Us | Helpdesk | Careers | Terms Of Service | Privacy Policy

Interface - Vehicle Control System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a modern city, a transportation company needs a system to manage different types of vehicles, including cars and bikes. The system must allow for basic operations such as starting the vehicle, stopping the vehicle, and displaying vehicle details. To maintain flexibility and support multiple vehicle types, the company decides to implement this system using Java interfaces.

Program Specifications

Interface:

Create an interface named Vehicle with the following methods:

- void start();
- void stop();
- void fuelCapacity();

Classes:

Create two classes that implement the Vehicle interface:

Car

- Implements all methods of the Vehicle interface.
- Has a fuelCapacity of 50 liters.

Bike

- Implements all methods of the Vehicle interface.
- Has a fuelCapacity of 15 liters.

Main Class:

- Create a class named VehicleManagement that:
- Creates instances of Car and Bike.
- Demonstrates the usage of the interface methods .

Input Format

The user will choose which vehicle to start and stop.

Constraints

NA

Output Format

The program will display messages indicating the actions taken on the vehicle.

Sample Input 0

```
1
```

Sample Output 0

```
Choose a vehicle:
1. Car
2. Bike
Starting the car...
```

Fuel capacity: 50 liters.
The car has stopped.

Sample Input 1

2

Sample Output 1

Choose a vehicle:
1. Car
2. Bike
Starting the bike...
Fuel capacity: 15 liters.
The bike has stopped.

Change Theme Language Java 7

```java
import java.util.*;
interface Vehicle{
    void start();
    void stop();
    void fuelCapacity();
}
class Car implements Vehicle{
    public void start(){
    System.out.println("Starting the car...");
}
    public void fuelCapacity(){
        System.out.println("Fuel capacity: 50 liters.");
    }
    public void stop(){
        System.out.println("The car has stopped.");
    }
}
class Bike implements Vehicle{
    public void start(){
        System.out.println("Starting the bike...");
    }
    public void fuelCapacity(){
        System.out.println("Fuel capacity: 15 liters.");
    }
    public void stop(){
        System.out.println("The bike has stopped.");
    }
```

Line: 49 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Interface - Player Management System for Sports AnalyticsIntInt

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a sports analytics application, it is crucial to manage statistics for different types of players in various sports. The application helps coaches, analysts, and sports enthusiasts track player performance and make informed decisions based on player statistics. This scenario involves creating a system where details for both cricket and hockey players can be managed using an abstract class and interface.

Program Specifications

Interface Creation:

- Create an interface named IPlayerStatistics with the method:

public void displayPlayerStatistics();

Base Class Implementation:

- Create an abstract class named Player.
- Include protected data members:
  1. String name
  2. String teamName
  3. Integer noOfMatches
- Include a constructor with the following arguments: name, teamName, noOfMatches.

Derived Class for Cricket Players:

- Create a class named CricketPlayer that extends the Player class.
- Include private data members:
  1. Long totalRunsScored
  2. Integer noOfWicketsTaken
- Include a constructor that takes totalRunsScored and noOfWicketsTaken as arguments.
- Implement the displayPlayerStatistics method to display cricket player details.

Derived Class for Hockey Players:

- Create a class named HockeyPlayer that extends the Player class.
- Include private data members:
  1. String position
  2. Integer noOfGoals
- Include a constructor that takes position and noOfGoals as arguments.
- Implement the displayPlayerStatistics method to display hockey player details.

Main Class:

- Create a Main class with a main method to display a menu, gather user input, and show player statistics.

Input Format

1. Menu selection (1 for Cricket Player or 2 for Hockey Player)
2. Player name (String)
3. Team name (String)
4. Number of matches played (Integer)
5. Additional statistics (specific to the sport)

Constraints

NA

Output Format

Display player details in the specified format.

Sample Input 0

```
1
Virat Kohli
Royal Challengers Bangalore
120
4500
15
```

Sample Output 0

```
Menu
1. Cricket Player Details
2. Hockey Player Details
Enter choice: Enter player name: Enter team name: Enter number of matches played: Enter total runs scored: Enter total number of wickets taken: Pl
Player name: Virat Kohli
Team name: Royal Challengers Bangalore
No of matches: 120
Total runs scored: 4500
No of wickets taken: 15
```

Sample Input 1

```
2
Manpreet Singh
Punjab Warriors
55
Midfielder
12
```

Sample Output 1

```
Menu
1. Cricket Player Details
2. Hockey Player Details
Enter choice: Enter player name: Enter team name: Enter number of matches played: Enter the position: Enter total number of goals scored: Player [
Player name: Manpreet Singh
Team name: Punjab Warriors
No of matches: 55
Position: Midfielder
No of goals taken: 12
```

Change Theme Language Java 7

```
1
2  import java.util.*;
3
4  interface IPlayerStatistics {
5      public void displayPlayerStatistics();
6  }
```

```
}

abstract class Player {
    protected String name;
    protected String teamName;
    protected Integer noOfMatches;

    public Player(String name, String teamName, Integer noOfMatches) {
        this.name = name;
        this.teamName = teamName;
        this.noOfMatches = noOfMatches;
    }
}

class CricketPlayer extends Player implements IPlayerStatistics {
    private Long totalRunsScored;
    private Integer noOfWicketsTaken;

    public CricketPlayer(String name, String teamName, Integer noOfMatches,
                         Long totalRunsScored, Integer noOfWicketsTaken) {
        super(name, teamName, noOfMatches);
        this.totalRunsScored = totalRunsScored;
```

Line: 114 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Interface - Smart Home System

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

Context: In a smart home, different devices need to be controlled and monitored through a centralized system. Each device can perform common functions like turning on/off and providing status updates. The smart home system requires an interface to define these functions and allows for the implementation of multiple device types.

Program Specifications

Create an interface named SmartDevice:

- Define the following methods:
  1. void turnOn();
  2. void turnOff();
  3. String getStatus();

Create two classes that implement the SmartDevice interface

- Class SmartLight:
  1. Member variables: String name, boolean isOn
  2. Implement the turnOn, turnOff, and getStatus methods.
- Class SmartThermostat:
  1. Member variables: String name, int temperature, boolean isOn
  2. Implement the turnOn, turnOff, and getStatus methods.

Create a class named Main:

- In the main method, create instances of SmartLight and SmartThermostat.
- Demonstrate the functionality of turning on and off each device.
- Display the status of each device.

Input Format

1. Name of the smart light.
2. Name of the smart thermostat.
3. Initial temperature for the thermostat.

Constraints

NA

Output Format

- Actions performed on each device (turning on/off).
- Status of each device after the actions.

Sample Input 0

```
Bedroom Light
Main Thermostat
20
```

Sample Output 0

```
Enter the name of the smart light: Enter the name of the smart thermostat: Enter the initial temperature for the thermostat: Turning on the light: Bedr
Bedroom Light is now ON.
Turning off the light: Bedroom Light
Bedroom Light is now OFF.
Turning on the thermostat: Main Thermostat
Main Thermostat is now ON. Temperature set to 20��C.
Turning off the thermostat: Main Thermostat
Main Thermostat is now OFF.

Device Status:
Bedroom Light: OFF
Main Thermostat: OFF
```

Sample Input 1

```
Kitchen Light
Kitchen Thermostat
18
```

Sample Output 1

```
Enter the name of the smart light: Enter the name of the smart thermostat: Enter the initial temperature for the thermostat: Turning on the light: Kitch
Kitchen Light is now ON.
Turning off the light: Kitchen Light
Kitchen Light is now OFF.
Turning on the thermostat: Kitchen Thermostat
Kitchen Thermostat is now ON. Temperature set to 18��C.
Turning off the thermostat: Kitchen Thermostat
Kitchen Thermostat is now OFF.

Device Status:
Kitchen Light: OFF
Kitchen Thermostat: OFF
```

Change Theme Language Java 7

```java
import java.util.Scanner;

interface SmartDevice {
    void turnOn();
    void turnOff();
    String getStatus();
}

class SmartLight implements SmartDevice {
    String name;
    boolean isOn;

    SmartLight(String name) {
        this.name = name;
        isOn = false;
    }

    public void turnOn() {
        isOn = true;
        System.out.println(name + " is now ON.");
    }
```

```
22
23      public void turnOff() {
24          isOn = false;
25          System.out.println(name + " is now OFF.");
26      }
27
```

Line: 97 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Interface - University Management System

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In a university management system, both students and professors can enroll in courses and provide feedback. The system needs to manage these functionalities efficiently, allowing for multiple inheritance through interfaces.

Program Specifications

Create an interface named Enrollable:

- Define the following methods:
  1. void enrollInCourse(String courseName);
  2. void provideFeedback(String feedback);

Create another interface named Person:

- Define the following methods:
  1. String getName();
  2. String getID();

Create two classes that implement these interfaces:

- Class Student:
  1. Member variables: String name, String studentID
  2. Implement methods from Enrollable and Person.
- Class Professor:
  1. Member variables: String name, String employeeID
  2. Implement methods from Enrollable and Person.

Create a class named Main:

- In the main method, create instances of Student and Professor.
- Demonstrate the functionality of enrolling in courses and providing feedback.

Input Format

1. Student Name
2. Student ID
3. Course Name for Student
4. Feedback from Student
5. Professor Name
6. Professor ID
7. Course Name for Professor
8. Feedback from Professor

Constraints

NA

Output Format

Confirmation messages for enrollment and feedback provided by both student and professor.

Sample Input 0

John
S001
Chemistry
Very challenging course!
Dr. Smith
P100
Organic Chemistry
Course requires detailed study.

Sample Output 0

Enter Student Name: Enter Student ID: Enter Course Name for Student: Student John with ID S001 has enrolled in the course: Chemistry.
Enter Feedback from Student: Student John provides feedback: Very challenging course!

Enter Professor Name: Enter Professor ID: Enter Course Name for Professor: Professor Dr. Smith with ID P100 has enrolled in the course: Organic Ch
Enter Feedback from Professor: Professor Dr. Smith provides feedback: Course requires detailed study.

Sample Input 1

Emma
S002
Computer Science
Enjoyed the programming assignments.
Dr. Brown
P101
Data Structures
Students should focus on algorithms.

Sample Output 1

Enter Student Name: Enter Student ID: Enter Course Name for Student: Student Emma with ID S002 has enrolled in the course: Computer Science.
Enter Feedback from Student: Student Emma provides feedback: Enjoyed the programming assignments.

Enter Professor Name: Enter Professor ID: Enter Course Name for Professor: Professor Dr. Brown with ID P101 has enrolled in the course: Data Struct
Enter Feedback from Professor: Professor Dr. Brown provides feedback: Students should focus on algorithms.

Change Theme Language Java 7

```
1  import java.util.Scanner;
2
3  interface Enrollable {
4      void enrollInCourse(String courseName);
5      void provideFeedback(String feedback);
6  }
7
8  interface Person {
9      String getName();
10     String getID();
11 }
12
13 class Student implements Enrollable, Person {
14     String name;
15     String studentID;
16
17     Student(String name, String studentID) {
18         this.name = name;
```

```
19          this.studentID = studentID;
20      }
21
22      public String getName() {
23          return name;
24      }
25
26      public String getID() {
27          return studentID;
```

Line: 110 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Interface - E-commerce Payment System

CONTEST ENDS IN

1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

In the fast-paced world of e-commerce, customers need multiple options to make payments for their purchases. An interface called PaymentGateway will be created to define the methods that different payment methods (like Credit Card, PayPal, and Net Banking) should implement. This allows the system to support various payment methods while maintaining a consistent approach to handling payments.

Program Specifications

Interface: PaymentGateway

- Methods:
  1. void initiatePayment(double amount);
  2. boolean validatePayment();
  3. void displayPaymentStatus();

Implementing Classes:

- **CreditCardPayment**
  1. Implements the PaymentGateway interface.
- Attributes:
  1. String cardNumber
  2. String cardHolderName
- Methods:
  1. Implements all methods defined in the PaymentGateway interface.
- **PayPalPayment**
  1. Implements the PaymentGateway interface.
- Attributes:
  1. String email
- Methods:
  1. Implements all methods defined in the PaymentGateway interface.
- **NetBankingPayment**
  1. Implements the PaymentGateway interface.
- Attributes:
  1. String bankName
  2. String accountNumber
- Methods:
  1. Implements all methods defined in the PaymentGateway interface.

Driver Class: Main

- The main method will prompt the user to select a payment method and input the necessary details to process the payment.

Input Format

1. Enter the payment method (1 for Credit Card, 2 for PayPal, 3 for Net Banking)
2. Enter the payment amount
3. For Credit Card: Enter the card number and cardholder name
4. For PayPal: Enter the email
5. For Net Banking: Enter the bank name and account number

Constraints

NA

Output Format

Display the payment status based on the inputs.

Sample Input 0

```
1
120.75
1111-2222-3333-4444
Alice Johnson
```

Sample Output 0

```
Select Payment Method:
1. Credit Card
2. PayPal
3. Net Banking
Enter payment amount:
Enter card number:
Enter cardholder name:
Payment Status:
Payment of $120.75 using Credit Card is successful.
```

Sample Input 1

```
2
60.00
emma.brown@example.com
```

Sample Output 1

```
Select Payment Method:
1. Credit Card
2. PayPal
3. Net Banking
Enter payment amount:
Enter email:
Payment Status:
Payment of $60.00 using PayPal is successful.
```

Change Theme Language Java 7

```
9  class CreditCardPayment implements PaymentGateway {
12      double amount;
13
14      CreditCardPayment(String cardNumber, String cardHolderName) {
15          this.cardNumber = cardNumber;
16          this.cardHolderName = cardHolderName;
17      }
18
19      public void initiatePayment(double amount) {
20          this.amount = amount;
21      }
22
23      public boolean validatePayment() {
```

```
        return cardNumber.length() >= 12 && !cardHolderName.isEmpty();
    }

    public void displayPaymentStatus() {
        if (validatePayment()) {
            System.out.printf(
                "Payment of $%.2f using Credit Card is successful.%n",
                amount
            );
        } else {
            System.out.println("Payment failed.");
        }
    }
}

```

Line: 156 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Exception Handling - Student Grades Lookup System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

A college wants to create a simple lookup system where students can check their grades for different subjects by entering a subject index number. The system holds a fixed number of subjects with corresponding grades stored in an array. However, if a student enters an index that is outside the bounds of the array, an ArrayIndexOutOfBoundsException occurs. To handle this, the system should catch the exception and display an appropriate error message.

Write a Java program that:

1. Initializes an array of grades for a list of subjects.
2. Prompts the user to enter the index of the subject for which they want to view the grade.
3. Displays the grade for the chosen subject.
4. If the entered index is out of bounds, catches the ArrayIndexOutOfBoundsException and displays an error message: Error: Invalid subject index. Please enter a number between 0 and last valid index.
5. Allows the user to try again until a valid index is entered.

Input Format

1. Number of subjects (integer).
2. Grades for each subject (array of integers).
3. Subject index to look up (integer).

Constraints

NA

Output Format

- If a valid index is entered, display the grade for the chosen subject.
- If an invalid index is entered, display an error message and prompt the user to try again.

Sample Input 0

```
4
75 84 92 68
1
```

Sample Output 0

```
Enter the number of subjects:
Enter grades for each subject:
Enter the subject index to check grade:
Grade for subject 1: 84
```

Sample Input 1

```
6
80 95 70 88 76 91
6
5
```

Sample Output 1

```
Enter the number of subjects:
Enter grades for each subject:
Enter the subject index to check grade:
Error: Invalid subject index. Please enter a number between 0 and 5.
Enter the subject index to check grade:
Grade for subject 5: 91
```

Change Theme Language Java 7

```java

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the number of subjects: ");
        int n = sc.nextInt();

        int[] grades = new int[n];

        System.out.println("Enter grades for each subject: ");
        for (int i = 0; i < n; i++) {
            grades[i] = sc.nextInt();
        }

        while (true) {
            System.out.println("Enter the subject index to check grade: ");
            int index = sc.nextInt();

            try {
                System.out.println(
                    "Grade for subject " + index + ": " + grades[index]
                );
                break;
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println(
                    "Error: Invalid subject index. Please enter a number between 0 and "
                    + (n - 1) + "."
                );
            }
```

Line: 38 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Exception Handling - Budget Planner for Monthly Expenses

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

Sam wants to create a simple budget planner to manage monthly expenses. The planner will take inputs for the estimated monthly income and expected expenses. Sam may occasionally enter non-numeric values by mistake, which would cause a NumberFormatException. To handle this, Sam's budget planner needs to catch this exception, inform the user of the error, and prompt them to enter a valid number.

Problem Statement

Write a Java program that:

1. Prompts the user to enter the estimated monthly income and expected monthly expenses.
2. Calculates and displays the remaining balance by subtracting expenses from income.
3. If the user enters a non-numeric value for income or expenses, handle the NumberFormatException and display an error message: Error: Please enter a valid numeric value.
4. Re-prompt the user to enter valid values until they are correct.

Input Format

1. Monthly income (string that should be convertible to a double).
2. Monthly expenses (string that should be convertible to a double).

Constraints

NA

Output Format

- If both inputs are valid numbers, display the remaining balance after subtracting expenses from income.
- If an input is invalid, display an error message and re-prompt the user for valid input.

Sample Input 0

```
6000
3000
```

Sample Output 0

```
Enter your estimated monthly income:
Enter your expected monthly expenses:
Remaining balance: Rs.3000.00
```

Sample Input 1

```
4500
three thousand
2000
```

Sample Output 1

```
Enter your estimated monthly income:
Enter your expected monthly expenses:
Error: Please enter a valid numeric value.
Enter your expected monthly expenses:
Remaining balance: Rs.2500.00
```

Change Theme Language Java 7

```
1
2  import java.util.Scanner;
3
4  public class Main {
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7
8          double income = 0;
9          double expenses = 0;
10         boolean valid = false;
11
12         while (!valid) {
13             System.out.println("Enter your estimated monthly income: ");
14             String input = sc.nextLine();
15
16             try {
17                 income = Double.parseDouble(input);
18                 valid = true;
19             } catch (NumberFormatException e) {
20                 System.out.println(
21                     "Error: Please enter a valid numeric value."
22                 );
23             }
24         }
25
26         valid = false;
27
28         while (!valid) {
29             System.out.println("Enter your expected monthly expenses: ");
30             String input = sc.nextLine();
31
32             try {
```

Line: 49 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Exception Handling - Customer Feedback Manager for ShopEasy

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

ShopEasy is expanding its online shopping platform and has developed a feature where customers can leave feedback on the products they purchase. The feedbacks are stored in the system, and at times, customers might not provide feedback for certain items. When the system tries to display all feedback, it may encounter a NullPointerException if it tries to print a null feedback entry. To ensure the application runs smoothly, ShopEasy needs a solution to handle this exception gracefully, displaying "No feedback provided" whenever a null feedback entry is encountered.

Problem Statement

Write a Java program for ShopEasy that:

1. Initializes an array of feedback entries (strings) for products.
2. Displays each feedback entry, handling any NullPointerException that occurs if an entry is null.
3. If an entry is null, display "No feedback provided" instead of attempting to display the null entry.

Input Format

1. Number of products to enter feedback for (integer).
2. Feedback for each product (string or left blank).

Constraints

NA

Output Format

Display each feedback entry, with "No feedback provided" for any null entries.

Sample Input 0

```
4
Excellent quality!
(blank)
Fast delivery
(blank)
```

Sample Output 0

```
Enter the number of products:
Enter feedback for product 1:
Enter feedback for product 2:
Enter feedback for product 3:
Enter feedback for product 4:
Feedback for product 1: Excellent quality!
Feedback for product 2: (blank)
Feedback for product 3: Fast delivery
Feedback for product 4: (blank)
```

Sample Input 1

```
2
(blank)
```

Not satisfied

Sample Output 1

```
Enter the number of products:
Enter feedback for product 1:
Enter feedback for product 2:
Feedback for product 1: (blank)
Feedback for product 2: Not satisfied
```

Change Theme Language Java 7

```
1
2  import java.util.Scanner;
3
4  public class Main {
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7
8          System.out.println("Enter the number of products: ");
9          int n = Integer.parseInt(sc.nextLine());
10
11         String[] feedback = new String[n];
12
13         for (int i = 0; i < n; i++) {
14             System.out.println("Enter feedback for product " + (i + 1) + ": ");
15             String input = sc.nextLine();
16
17             if (!input.trim().isEmpty()) {
18                 feedback[i] = input;
19             }
20         }
21
22         for (int i = 0; i < n; i++) {
23             try {
24                 System.out.println(
25                     "Feedback for product " + (i + 1) + ": "
26                     + feedback[i].toString()
27                 );
28             } catch (NullPointerException e) {
29                 System.out.println(
30                     "Feedback for product " + (i + 1)
31                     + ": No feedback provided"
32                 );
```

Line: 39 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Exception Handling - Custom Exception Handling for Age Verification

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

A developer is tasked with creating a program that verifies a user's age for a certain application. The program should throw a custom exception if the user is under the required age of 18. It should handle this exception with a try-catch block. Regardless of whether an exception occurs, a finally block should execute to print a message indicating the end of the age verification process.

This exercise will teach you:

1. How to create and throw custom exceptions.
2. How to handle exceptions using try, catch, and finally blocks.

Write a Java program that:

1. Prompts the user to enter their age.
2. If the age is less than 18, throw a custom exception named AgeNotValidException.
3. Use a try-catch block to catch the exception and display an appropriate message.
4. If the age is valid (18 or older), print a success message.
5. Include a finally block that executes regardless of whether an exception was caught.

Input Format

A single integer representing the user's age.

Constraints

NA

Output Format

- If the age is less than 18, output a message indicating that the age is not valid.
- If the age is valid, output a success message.
- Output a final message indicating the completion of the verification process in all cases.

Sample Input 0

```
17
```

Sample Output 0

```
Enter your age:
Exception: Age is not valid. You must be at least 18 years old.
End of Age Verification Process.
```

Sample Input 1

```
18
```

Sample Output 1

```
Enter your age:
Age verification successful. Welcome!
End of Age Verification Process.
```

Change Theme Language Java 7

```java

import java.util.Scanner;

class AgeNotValidException extends Exception {
    public AgeNotValidException(String message) {
        super(message);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter your age:");
        int age = sc.nextInt();

        try {
            if (age < 18) {
                throw new AgeNotValidException(
                    "Age is not valid. You must be at least 18 years old."
                );
            }

            System.out.println("Age verification successful. Welcome!");
        } catch (AgeNotValidException e) {
            System.out.println("Exception: " + e.getMessage());
        } finally {
            System.out.println("End of Age Verification Process.");
        }

        sc.close();
    }
```

Line: 34 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Exception Handling - String Splitter with Exception Handling

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

A programmer is creating a tool that splits a user-input string based on the / (slash) symbol and displays each split part with its respective index. If the string length is too short (2 characters or fewer), a NullPointerException should be triggered to indicate that splitting is not applicable. Additionally, regardless of any exceptions, the program should execute a final cleanup message.

This exercise teaches the concepts of:

1. Handling exceptions with custom conditions.
2. Using a finally block to execute code unconditionally.

Write a Java program that:

1. Prompts the user to enter a string.
2. If the string length is greater than 2, calls a user-defined method splitString() to split the string by /.
3. Displays each split part with its index in the format Splitted string at index X is: Y.
4. If the string length is 2 or less, throws a NullPointerException.
5. Regardless of any exception occurrence, executes a finally block to print "Inside finally block".

Input Format

A single string (containing words separated by /).

Constraints

NA

Output Format

- If the string length is greater than 2, output each part of the split string with its index.
- If the string length is 2 or less, output a NullPointerException message.
- Output "Inside finally block" in all cases.

Sample Input 0

```
Java/Programming/Language
```

Sample Output 0

```
Enter a string: Splitted string at index 0 is: Java
Splitted string at index 1 is: Programming
Splitted string at index 2 is: Language
Inside finally block
```

Sample Input 1

```
Hi
```

Sample Output 1

```
Enter a string: Exception: java.lang.NullPointerException: String length is too short to split.
Inside finally block
```

Change Theme Language Java 7

```java

import java.util.Scanner;

public class Main {

    public static void splitString(String str) {
        String[] parts = str.split("/");

        for (int i = 0; i < parts.length; i++) {
            System.out.println(
                "Splitted string at index " + i + " is: " + parts[i]
            );
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        try {
            if (str.length() <= 2) {
                throw new NullPointerException(
                    "String length is too short to split."
                );
            }

            splitString(str);

        } catch (NullPointerException e) {
            System.out.println("Exception: " + e);
```

Line: 40 Col: 1

Upload Code as File Test against custom input Run Code Submit Code

Exception Handling - Online Quiz Scoring System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

An online quiz platform allows students to take quizzes and receive scores based on their answers. After taking the quiz, students enter the total score they obtained. However, a score is only valid if it falls within the range of 0 to 100. If a score outside this range is entered, the system should throw a custom exception, InvalidScoreException, and prompt the student to enter a valid score.

The program should:

1. Catch the invalid score exception and display an error message.
2. Ensure all resources, such as the input scanner, are properly closed, regardless of whether the score is valid or invalid, by using a finally block.

Problem Statement

Write a Java program that:

1. Prompts the user to enter their quiz score.
2. Checks if the entered score is between 0 and 100.
3. If the score is outside this range, throws an InvalidScoreException with an appropriate message.
4. Catches this exception, displays the error message, and prompts the user to enter a valid score.
5. Closes resources using a finally block to ensure proper cleanup, even if an exception occurs.

Input Format

Quiz score (integer).

Constraints

NA

Output Format

- If the score is valid, display a success message.
- If the score is invalid, display an error message and prompt for a valid score.
- Finally, display a confirmation message that resources have been closed.

Sample Input 0

```
105
85
```

Sample Output 0

```
Enter your quiz score:
Error: Score must be between 0 and 100. Please enter a valid score.
Enter your quiz score:
Your score of 85 has been recorded.
Resources have been closed.
```

Sample Input 1

```
50
```

Sample Output 1

```
Enter your quiz score:
Your score of 50 has been recorded.
Resources have been closed.
```

Change Theme Language Java 7

```
1
2   import java.util.Scanner;
3
4   class InvalidScoreException extends Exception {
5       public InvalidScoreException(String message) {
6           super(message);
7       }
8   }
9
10  public class Main {
11      public static void main(String[] args) {
12          Scanner sc = new Scanner(System.in);
13
14          try {
15              while (true) {
16                  System.out.println("Enter your quiz score: ");
17                  int score = sc.nextInt();
18
19                  try {
20                      if (score < 0 || score > 100) {
21                          throw new InvalidScoreException(
22                              "Error: Score must be between 0 and 100. Please enter a valid sc
23                          );
24                      }
25
26                      System.out.println(
27                          "Your score of " + score + " has been recorded."
28                      );
29                      break;
30
31                  } catch (InvalidScoreException e) {
                        System.out.println(e.getMessage());
```

Line: 41 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Exception Handling - Bank Account Withdrawal System

CONTEST ENDS IN
1 day

Problem Submissions Leaderboard

RATE THIS CHALLENGE

★ ★ ★ ★ ★

A bank provides a simple system where customers can withdraw money from their accounts. For security reasons, the bank needs to enforce a minimum balance requirement of ₹1000 in each account. If a withdrawal request would leave the account balance below this minimum, an exception should be raised, and the transaction should be denied. The program should handle this situation by throwing an exception and providing a clear error message.

Write a Java program that:

1. Prompts the user to enter their account balance and the amount they want to withdraw.
2. Checks if the withdrawal is possible without violating the minimum balance requirement of ₹1000.
3. If the withdrawal amount would leave the account balance below ₹1000, the program should throw a custom exception, InsufficientBalanceException.
4. Catch this exception and display an error message: Error: Withdrawal denied. Minimum balance of ₹1000 must be maintained.
5. If the withdrawal is successful, display the remaining balance.

Input Format

1. Initial account balance (double).
2. Amount to withdraw (double).

Constraints

NA

Output Format

- If the withdrawal is valid, display the remaining balance.
- If the withdrawal violates the minimum balance, display an error message.

Sample Input 0

```
2500
1600
```

Sample Output 0

```
Enter your account balance:
Enter amount to withdraw:
Error: Withdrawal denied. Minimum balance of Rs.1000 must be maintained.
```

Sample Input 1

```
4500
3000
```

Sample Output 1

```
Enter your account balance:
Enter amount to withdraw:
```

Transaction successful. Remaining balance: Rs.1500.00

Change Theme Language Java 7

```
import java.util.Scanner;

class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter your account balance: ");
        double balance = sc.nextDouble();

        System.out.println("Enter amount to withdraw: ");
        double amount = sc.nextDouble();

        try {
            if (balance - amount < 1000) {
                throw new InsufficientBalanceException(
                    "Error: Withdrawal denied. Minimum balance of Rs.1000 must be maintained.
                );
            }

            double remainingBalance = balance - amount;

            System.out.printf(
                "Transaction successful. Remaining balance: Rs.%.2f%n",
                remainingBalance
            );
```

Line: 41 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code